```
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">
<title>Spaceship Command</title>

<!-- ============================================================
     HTA APPLICATION SETTINGS
     Declares this as a standalone desktop window, not a browser
     page. contextmenu="no" kills the native IE right-click menu
     so right-click can be used as the game's primary input verb.
     ============================================================ -->
<hta:application
    id="SpaceshipCommand"
    applicationname="Spaceship Command"
    border="thin"
    borderstyle="normal"
    caption="yes"
    contextmenu="no"
    innerborder="no"
    maximizebutton="yes"
    minimizebutton="yes"
    navigable="no"
    scroll="no"
    scrollflat="yes"
    selection="no"
    showintaskbar="yes"
    singleinstance="yes"
    sysmenu="yes"
    version="1.0"
    windowstate="normal">

<style type="text/css">

/* ============================================================
   SECTION 1 : BASE / RESET
   Palette is hardcoded hex throughout. CSS custom properties
   are NOT supported by Trident, so no var() anywhere.
     white      #ffffff     panel      #faf7f2
     panel2     #f4efe6     panel3     #efe8dc
     border     #e4dbcb     borderdark #cbb89c
     coffee     #8c6f4e     coffeedark #6b5338
     coffeelite #b89b76     text       #3b3630
     muted      #8a7f72     terracotta #b05a42
     sage       #7f9161     amber      #c08a3e
   ============================================================ */

/* UNIVERSAL BORDER-BOX. Padding and borders are drawn INSIDE a
   declared width, which is the single rule that makes a fluid
   layout safe here: width:100% plus padding can no longer burst
   its parent, and percentage columns add up to exactly 100.
```

```
   Every fixed pixel size in this file is therefore an OUTER size. */
* { margin:0; padding:0;
    box-sizing:border-box;
    -moz-box-sizing:border-box;
    -webkit-box-sizing:border-box; }

html, body {
    width:100%; height:100%;
    overflow:hidden;
    background:#ffffff;
    color:#3b3630;
    font-family:"Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    font-size:12px;
    cursor:default;
}

.num {
    font-family:Consolas, "Lucida Console", "Courier New", monospace;
}

/* ============================================================
   SECTION 2 : WINDOW FRAME
   Absolute positioning is used instead of flexbox. Trident's
   flex support is unreliable; top/left/right/bottom anchoring
   is rock solid all the way back to IE8 standards mode.

   THE FLUID RULE OF THIS FILE:
   the side panel keeps a fixed width, and #content pins itself to
   left:166px AND right:0. That pair IS calc(100% - 166px), without
   needing calc or flexbox - the content area simply grows and
   shrinks with the window in both directions.
   ============================================================ */

#taskbar {
    position:absolute; top:0; left:0; right:0; height:35px;
    background:#f4efe6;
    border-bottom:1px solid #cbb89c;
    overflow:hidden;
    white-space:nowrap;
}

/* 30% narrower than the original 236px. The panel is a glance
   strip now, not a reading column: the three roster blocks fold
   away and only open when the player wants them. */
#sidepanel {
    position:absolute; top:36px; left:0; bottom:93px; width:165px;
    background:#faf7f2;
    border-right:1px solid #e4dbcb;
    overflow-y:auto;
    overflow-x:hidden;
}
```

```
/* Pinned. The ship functions and the build marker must not drift
   up and down as the blocks above them fold open and shut, and
   must not scroll away either - so they live outside the
   scrolling panel, anchored to the bottom of the window. */
#sidefoot {
    position:absolute; left:0; bottom:23px; width:165px; height:70px;
    background:#faf7f2;
    border-right:1px solid #e4dbcb;
    border-top:1px solid #e4dbcb;
    overflow:hidden;
}

#content {
    position:absolute; top:36px; left:166px; right:0; bottom:23px;
    background:#ffffff;
    overflow:auto;
}

.st-last { color:#5c5145; }
#statusbar {
    position:absolute; left:0; right:0; bottom:0; height:22px;
    background:#f4efe6;
    border-top:1px solid #e4dbcb;
    color:#8a7f72;
    font-size:11px;
    line-height:22px;
    padding:0 10px;
    overflow:hidden;
}

/* ============================================================
   SECTION 3 : TASKBAR TABS
   ============================================================ */

.tab {
    display:inline-block;
    height:26px;
    line-height:25px;
    margin:6px 2px 0 0;
    padding:0 10px 0 12px;
    background:#efe8dc;
    border:1px solid #e4dbcb;
    border-bottom:none;
    color:#6b6157;
    font-size:12px;
    cursor:pointer;
    vertical-align:top;
    max-width:210px;
    overflow:hidden;
}
.tab:hover     { background:#f7f2e9; color:#3b3630; }
.tab-active    { background:#ffffff; border-color:#cbb89c; color:#3b3630; font-weight:bold;
```

```
height:29px; line-height:27px; margin-top:6px; }
.tab-home      { border-left:3px solid #8c6f4e; }
.tab-kind      { color:#a2947f; font-weight:normal; padding-right:5px; }
.tab-close {
   display:inline-block; width:14px; height:14px; line-height:13px;
   text-align:center; margin-left:8px; color:#a2947f;
   font-size:13px; font-weight:normal;
}
.tab-close:hover { background:#e3d3c0; color:#b05a42; }
/* reserved: unread / changed marker on a tab (Phase: event engine) */
.tab-mark {
   display:inline-block; width:6px; height:6px; margin-left:7px;
   background:#7f9161; vertical-align:middle;
}

/* ============================================================
  SECTION 4 : SIDE PANEL
  ============================================================ */

.sp-block  { border-bottom:1px solid #e4dbcb; padding:9px 10px 10px 10px; }
.sp-title  {
   font-size:10px; letter-spacing:1px; text-transform:uppercase;
   color:#8c6f4e; font-weight:bold; margin-bottom:8px;
}

/* --- folding blocks ------------------------------------- */
/* A header the player can click. Closed by default: the roster
  is reference material, not something to stare at all watch. */
.sp-drop {
   font-size:10px; letter-spacing:1px; text-transform:uppercase;
   color:#8c6f4e; font-weight:bold;
   cursor:pointer; overflow:hidden; padding:1px 0;
}
.sp-drop:hover { color:#6b5338; }
.sp-drop-open  { margin-bottom:7px; }
/* CSS triangle, no image and no font glyph: right when closed,
  down when open */
.sp-caret {
   float:right; width:0; height:0; margin:4px 1px 0 0;
   border-top:4px solid transparent; border-bottom:4px solid transparent;
   border-left:5px solid #a2947f;
}
.sp-caret-open {
   margin:5px 0 0 0;
   border-left:4px solid transparent; border-right:4px solid transparent;
   border-top:5px solid #8c6f4e; border-bottom:none;
}
.sp-row       { padding:2px 0; overflow:hidden; }
.sp-hr        { height:0; border-top:1px solid #ece5d8; margin:6px 0 5px 0; }
.sp-long      { padding:3px 0 4px 0; }
.sp-longk     { color:#8c6f4e; font-size:10px; letter-spacing:.4px; }
.sp-longv     { color:#6b6157; font-size:10px; line-height:14px;
```

```
            padding-top:1px; }
.sp-key      { float:left; color:#6b6157; }
.sp-val      { float:right; color:#3b3630; }
.sp-val-dim  { float:right; color:#8a7f72; }

.sp-ship     { font-size:14px; font-weight:bold; color:#3b3630; margin-bottom:2px; }
.sp-sub      { color:#8a7f72; font-size:11px; line-height:15px; }

/* --- cryptic indicators : deliberately unlabelled ----------
   Standing lines that fill from the BOTTOM UP, one per process
   the ship has running. No symbols, no captions, no numbers -
   the player learns to read the shape of the group.

   GREEN SHRINKS  - work the player committed to. The line drains
                    as the process nears completion, so an empty
                    slot means something is ready to collect.
   RED GROWS      - something building on its own. A climbing red
                    line is the only warning the ship gives.

   Half the size of the old horizontal tracks, and they no longer
   take a row each: the whole set is one strip. */
.ind-strip {
   position:relative; height:104px; overflow:hidden;
}
.ind-line {
   position:absolute; bottom:0; width:3px;
   background:#7f9161;
}
.ind-grow { background:#b05a42; }
/* the unused remainder of a line, so the eye can see how far a
   process still has to travel */
.ind-ghost {
   position:absolute; top:0; width:3px; background:#ece5d8;
}
.ind-none { color:#b3a794; font-size:10px; line-height:104px; }

/* --- ship functions : symbols, pinned bottom left ----------
   Every symbol is built out of plain divs. No image files (the
   .hta must stay a single file) and no font glyphs (a Trident
   host may not have one and would render a box). */
/* --- the events drop-down ------------------------------ */
.sp-evt {
   padding:4px 0; border-bottom:1px solid #f0e9dd; overflow:hidden;
}
.sp-evt-kind { float:left; color:#5c5145; font-size:11px; line-height:19px; }
.sp-evt-btn {
   float:right; padding:1px 8px; font-size:10px; cursor:pointer;
   background:#f4efe6; border:1px solid #cbb89c; color:#6b5338;
}
.sp-evt-btn:hover { background:#e9dcc6; border-color:#8c6f4e; }
.sp-dim { color:#b3a794; font-size:11px; }
```

```
.sp-icons  { text-align:center; padding:8px 0 0 0; font-size:0; }
.sp-ico {
    display:inline-block; position:relative;
    width:26px; height:26px; margin:0 3px;
    background:#f4efe6; border:1px solid #e4dbcb;
    cursor:pointer; vertical-align:top;
}
.sp-ico:hover { background:#ffffff; border-color:#8c6f4e; }
.ico-part { position:absolute; background:#6b5338; }

/* ship hull : a wide shallow trapezoid drawn with borders, mast
   and sail above it. Wide and flat, or it reads as an arrowhead. */
.ico-hull {
    left:3px; top:15px; width:12px; height:0; background:none;
    border-left:4px solid transparent;
 border-right:4px solid transparent;
    border-top:6px solid #6b5338;
}
.ico-mast { left:12px; top:5px;  width:2px; height:10px; }
.ico-sail { left:14px; top:6px;  width:6px; height:6px; background:#a2947f; }

/* log : a short stack of rules */
.ico-line1 { left:6px; top:8px;  width:14px; height:2px; }
.ico-line2 { left:6px; top:12px; width:14px; height:2px; }
.ico-line3 { left:6px; top:16px; width:9px;  height:2px; }

/* disk : body, shutter, label - save and load are one door.
   The shutter sits off-centre and the label is short, or the two
   cut-outs meet in the middle and the icon reads as a letter H. */
.ico-disk  { left:5px;  top:5px;  width:16px; height:16px; }
.ico-shut  { left:13px; top:6px;  width:4px;  height:5px; background:#faf7f2; }
/* the label is PRINTED on the disk, not cut out of it - a
   background-coloured block here reads as the letter H */
.ico-lab   { left:8px;  top:14px; width:10px; height:7px; background:#c9b79a; }

/* cog : a ring with four teeth */
.ico-ring {
    left:7px; top:7px; width:12px; height:12px; background:none;
    border:3px solid #6b5338;
    -moz-border-radius:9px; -webkit-border-radius:9px; border-radius:9px;
}
.ico-t1 { left:11px; top:3px;  width:4px; height:4px; }
.ico-t2 { left:11px; top:19px; width:4px; height:4px; }
.ico-t3 { left:3px;  top:11px; width:4px; height:4px; }
.ico-t4 { left:19px; top:11px; width:4px; height:4px; }

.sp-build {
    text-align:center; color:#b3a794; font-size:10px;
    padding:6px 6px 0 6px; line-height:13px;
}

/* ============================================================
```

```
   SECTION 5 : IDLE SCREEN  (the free screen - costs nothing)
   ========================================================== */

#screen-idle { padding:16px 18px 30px 18px; }

/* FLUID. The rail spans the whole content area; each cell is told
   its share as a percentage by the builder, so adding a fifth
   gauge later needs no CSS change at all. The gutter is padding
   rather than margin so it lives inside the percentage. */
.rail       { width:100%; margin:0 0 12px 0; overflow:hidden; }
.rail-cell  { float:left; padding-right:8px; }
.rail-last  { padding-right:0; }

.stat {
   background:#faf7f2; border:1px solid #e4dbcb;
   padding:7px 10px 8px 10px; width:100%;
}
.stat:hover  { border-color:#cbb89c; background:#f7f2e9; }
.stat-label  { font-size:10px; letter-spacing:.7px; text-transform:uppercase; color:#8a7f72; }
.stat-value  { font-size:17px; color:#3b3630; padding:1px 0 4px 0; }
.stat-unit   { font-size:11px; color:#8a7f72; }

.bar    { height:6px; background:#efe8dc; border:1px solid #e4dbcb; overflow:hidden; }
.bar-in { height:6px; }

.info {
   background:#faf7f2; border:1px solid #e4dbcb;
   padding:7px 10px 8px 10px; width:100%; min-height:48px;
}
.info:hover  { border-color:#cbb89c; background:#f7f2e9; }
.info-label  { font-size:10px; letter-spacing:.7px; text-transform:uppercase; color:#8a7f72; }
.info-value  { font-size:13px; color:#3b3630; padding-top:3px; }

/* --- the ship diagram ------------------------------------ */
/* CENTRE STAGE - DELIBERATELY NOT FLUID. The hull is a scale
   drawing, not a table: every room coordinate in ROOMS is
   measured against this 836 x 344 stage, so it keeps its exact
   size and is centred in whatever space the window gives it.
   This is the house rule for every middle-of-screen graphic in
   the project; see STAGE in the script. Everything around it
   moves, the drawing itself does not. */
#stage {
   position:relative; width:1160px; height:460px;
   margin:0 auto; background:#ffffff;
   border:1px solid #e4dbcb;
}
/* --- the silhouette : one rule per hull part, all drawn from
   the HULL table in stage coordinates ---------------------- */
.hpart  { position:absolute; background:#fdfbf7;
          border:2px solid #8c6f4e; }
.hp-body { -moz-border-radius:0 60px 60px 0; -webkit-border-radius:0 60px 60px 0;
          border-radius:0 60px 60px 0; }
```

```
.hp-spine{ background:#faf7f2;
        -moz-border-radius:14px 14px 0 0; -webkit-border-radius:14px 14px 0 0;
        border-radius:14px 14px 0 0; }
.hp-pod  { background:#faf7f2;
        -moz-border-radius:0 0 14px 14px; -webkit-border-radius:0 0 14px 14px;
        border-radius:0 0 14px 14px; }
.hp-lens { background:#fdfbf7;
        -moz-border-radius:50%; -webkit-border-radius:50%; border-radius:50%; }
.hp-nacelle { background:#efe8dc;
        -moz-border-radius:15px; -webkit-border-radius:15px; border-radius:15px; }
.hp-pylon{ background:#e4dbcb; border:2px solid #8c6f4e; }
.nac-bar { position:absolute; background:#cbb89c; }
.conduit { position:absolute; background:#f7f2e9;
        border-top:1px dashed #ddd2bf; border-bottom:1px dashed #ddd2bf; }
.nozzle {
   position:absolute; left:-15px; width:15px; height:38px;
   background:#efe8dc; border:2px solid #8c6f4e; border-right:none;
}
.room {
   position:absolute;
   border:1px solid #cbb89c;
   overflow:hidden;
}
.room:hover { border-color:#8c6f4e; background:#f6efe3 !important; }
.room-name {
   position:absolute; left:7px; top:5px;
   font-size:11px; font-weight:bold; color:#5c5145;
   letter-spacing:.3px;
}
.room-sys {
   position:absolute; left:7px; top:20px; right:5px;
   font-size:10px; color:#9c9082; line-height:13px;
}
.room-tag {
   position:absolute; right:6px; bottom:4px;
   font-size:9px; color:#b3a794; letter-spacing:.5px;
}
/* a numbered shell: name only, centred, because there is nothing
  else in there for it to line up with */
.room-small .room-name { top:50%; margin-top:-8px; left:0; right:0;
                         text-align:center; font-weight:normal;
                         color:#8a7f72; }
.room-small .room-tag  { bottom:2px; right:4px; font-size:8px; }
.corridor {
   position:absolute; background:#f7f2e9;
   border:1px dashed #d8cab4;
}
.divider {
   position:absolute; width:0; border-left:1px dashed #ddd2bf;
}

/* --- crew dots : cosmetic ambient drift only ---------- */
```

```
.dot {
   position:absolute; width:15px; height:15px;
   -moz-border-radius:8px; -webkit-border-radius:6px; border-radius:6px;
   border:2px solid #ffffff;
}
.dot:hover { border-color:#3b3630; }
.dot-label {
   position:absolute; font-size:9px; color:#8a7f72;
   white-space:nowrap;
}

.stage-note {
   width:100%; max-width:1160px; margin:6px auto 0 auto;
   font-size:11px; color:#a2947f; text-align:right;
}

/* --- event log / activity feed ----------------------------- */
#feedwrap { width:100%; margin:14px 0 0 0; }
.feed-head {
   font-size:10px; letter-spacing:1px; text-transform:uppercase;
   color:#8c6f4e; font-weight:bold; padding-bottom:6px;
}
/* no fixed height: layoutIdle() gives the feed whatever vertical
  space is left over, so it grows as the window grows */
#feed {
   min-height:62px; overflow-y:auto;   /* matches FEED.min in the script */
   background:#faf7f2; border:1px solid #e4dbcb;
   padding:6px 10px;
}
.feed-line   { padding:2px 0; color:#5c5145; line-height:15px; }
.feed-time   { color:#b3a794; padding-right:8px; }

/* ============================================================
  SECTION 6 : TASK WINDOWS
  ============================================================ */

.task-screen { padding:18px 22px 30px 22px; }
.task-head   { border-bottom:1px solid #cbb89c; padding-bottom:9px; margin-bottom:14px; }
.task-kind   { font-size:10px; letter-spacing:1.2px; text-transform:uppercase; color:#8c6f4e; }
.task-title  { font-size:19px; color:#3b3630; padding-top:2px; }
.task-sub    { font-size:12px; color:#8a7f72; padding-top:3px; }

.panel       { background:#faf7f2; border:1px solid #e4dbcb; margin-bottom:12px; }
.panel-head  {
   background:#f4efe6; border-bottom:1px solid #e4dbcb;
   padding:6px 11px; font-size:10px; letter-spacing:1px;
   text-transform:uppercase; color:#6b5338; font-weight:bold;
}
.panel-body  { padding:11px 13px; }

.kv      { overflow:hidden; padding:3px 0; border-bottom:1px solid #f0e9dd; }
.kv-k    { float:left; width:150px; color:#6b6157; }
```

```
.kv-v     { display:block; overflow:hidden; color:#3b3630; }

.act {
   display:block; width:100%; text-align:left;
   padding:7px 11px; margin-bottom:5px;
   background:#f4efe6; border:1px solid #e4dbcb;
   color:#9c9082; font-size:12px;
   font-family:"Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
   cursor:default;
}
/* a wired, spendable action - visibly alive next to the dead ones */
.act-live {
   background:#ffffff; border:1px solid #8c6f4e;
   color:#3b3630; cursor:pointer;
}
.act-live:hover   { background:#f6efe3; border-color:#6b5338; }
.act-live .act-cost { color:#6b5338; border-color:#cbb89c; background:#f4efe6; }
.act-cost {
   float:right; font-size:10px; color:#a2947f;
   border:1px solid #ddd2bf; padding:1px 6px; background:#faf7f2;
}
.notice {
   background:#f7f2e9; border:1px solid #ddd2bf;
   border-left:3px solid #8c6f4e;
   padding:9px 12px; color:#6b6157; line-height:17px;
}

.col-l { float:left; width:52%; }
.col-r { float:left; width:46%; margin-left:2%; }
.clear { clear:both; height:0; overflow:hidden; }

/* ============================================================
  SECTION 7 : FLOATING POPUP WINDOWS
  Side panel items open here so the idle screen stays visible
  behind them, per manifesto 1.5.
  ============================================================ */

.pop {
   position:absolute; width:380px;
   background:#ffffff; border:1px solid #8c6f4e;
   z-index:40;
}
.pop-head {
   background:#f4efe6; border-bottom:1px solid #cbb89c;
   padding:6px 9px; font-size:11px; font-weight:bold; color:#5c5145;
   cursor:move; overflow:hidden;
}
.pop-x {
   float:right; width:15px; height:15px; line-height:14px;
   text-align:center; color:#8a7f72; font-weight:normal; cursor:pointer;
}
.pop-x:hover { background:#e3d3c0; color:#b05a42; }
```

```
.pop-body    { padding:11px 13px; max-height:400px; overflow:auto; }

/* ============================================================

   SECTION 8 : PROTAGONIST MOUSE
   The cursor is the player's body in the world. A spent action
   leaves a mark exactly where the hand was. Drawn as two bars
   rather than a glyph so no font can ever fail us.
   ============================================================ */

/* pointer-events:none IS THE WHOLE FIX for clicks lost when
   clicking quickly. The mark is an 11px element at z-index 120
   dropped exactly under the cursor and left there for over half
   a second - so the NEXT click in the same spot hit the mark
   instead of the button and was silently thrown away. It must be
   scenery, never a target. Same family as Entry 015 and the
   position ring in Entry 023. */
.amark   { position:absolute; width:11px; height:11px; z-index:120;
           pointer-events:none; }
.amark-h { position:absolute; left:0; top:4px; width:11px; height:2px;
           background:#7ec63f; box-shadow:0 0 5px #b6f07d;
           pointer-events:none; }
.amark-v { position:absolute; left:4px; top:0; width:2px; height:11px;
           background:#7ec63f; box-shadow:0 0 5px #b6f07d;
           pointer-events:none; }

/* ============================================================
   SECTION 9 : PROGRESS BUTTONS
   A committed process shows its progress as a filling bar and
   never as a number. The player learns to read the bar, not to
   count down from nine.
   ============================================================ */

.pbtn {
   position:relative; display:block; width:100%;
   height:31px; margin-top:4px;
   background:#ffffff; border:1px solid #8c6f4e;
   overflow:hidden; cursor:pointer;
}
.pbtn:hover      { background:#f6efe3; }
.pbtn-fill {
   position:absolute; left:0; top:0; height:100%; width:0;
   background:#dff0cc; border-right:1px solid #7ec63f;
}
.pbtn-text {
   position:relative; line-height:29px; text-align:center;
   color:#3b3630; font-size:12px;
}
/* running: inert and quiet, the bar does the talking */
.pbtn-run        { border-color:#cbb89c; cursor:default; background:#faf7f2; }
.pbtn-run:hover  { background:#faf7f2; }
.pbtn-run .pbtn-text { color:#8a7f72; }
```

```
/* finished: alive again, waiting to be acknowledged */
.pbtn-done       { border-color:#7ec63f; background:#ffffff; cursor:pointer; }
.pbtn-done .pbtn-text { color:#3b3630; font-weight:bold; }
.pbtn-off        { border-color:#e4dbcb; cursor:default; }
.pbtn-off:hover  { background:#ffffff; }
.pbtn-off .pbtn-text { color:#b3a794; }

/* ============================================================
   SECTION 10 : NAVIGATION SCREEN
   Flow layout, not absolute positioning. Panels size themselves
   to their content and there are no scrollbars anywhere - the
   Scan Data column in particular must be free to grow downward
   without limit as sensors improve.
   ============================================================ */

/* FLUID. The nav screen no longer has a width of its own - it is
   as wide as the content area, which is as wide as the window. */
.navwrap { width:100%; margin:0; }

/* overflow:hidden on a row is the IE-safe way to contain floats */
.navrow  { overflow:hidden; }

.navpanel {
    background:#faf7f2; border:1px solid #e4dbcb;
    margin-bottom:12px;
}
.navpanel-head {
    background:#f4efe6; border-bottom:1px solid #e4dbcb;
    padding:5px 10px; font-size:10px; letter-spacing:1px;
    text-transform:uppercase; color:#6b5338; font-weight:bold;
}
/* no height, no overflow - the panel is as tall as its content */
.navpanel-body {
    padding:9px 11px;
    color:#3b3630; line-height:17px;
}
.navpanel-body .dim { color:#a2947f; }

/* PERCENTAGE COLUMNS. Under border-box the 1px panel border is
   inside the percentage, so the arithmetic is simply
   32 + 2 + 32 + 2 + 32 = 100 and nothing has to be shaved.

   The LAST column of every row floats RIGHT, not left. Trident
   rounds each percentage to whole pixels and the accumulated
   1px error would eventually wrap the final column onto its own
   line; a right float can never collide with the ones before it. */
/* THE FRAME. The chart is the centrepiece of this window, so the
   panels are arranged AROUND it and never in front of it: a
   stacked column on the left, the chart in the middle, Scan Data
   on the right.

   Three display:table-cells, not floats. Floats cannot give the
```

```
   side columns the height of the chart and cannot centre the
   chart vertically against them; table cells do both for free and
   are honoured by every document mode an HTA can land in. This is
   layout by table BEHAVIOUR, not a <table> element - the markup
   stays divs.

   The side columns are fixed pixel widths and the middle takes
   whatever is left, so the chart keeps as much room as the window
   can spare and the reading panels stay a readable, steady width
   instead of stretching on a wide monitor. */
.navmain   { display:table; width:100%; table-layout:fixed;
             margin-bottom:12px; }
.navcell   { display:table-cell; }
/* ALL THREE ANCHOR TO THE TOP. The chart sits centre-top, in the
   same place the ship slice sits on the idle screen, and the two
   reading columns start level with it. Consistency across task
   windows matters more than filling the lower half: a panel that
   floats to a different height depending on how much text is in
   it is a panel the eye has to hunt for. */
.navcell-l { width:214px; vertical-align:top; padding-right:12px; }
.navcell-c {               vertical-align:top; }
.navcell-r { width:252px; vertical-align:top; padding-left:12px;  }

/* the chart and the two panels under it travel together and stay
   centred in the middle column, so the panels always line up with
   the chart edges however wide the window gets */
.navcenter { max-width:900px; margin:0 auto; }

/* last panel in a stack - nothing below it to space away from */
.np-last { margin-bottom:0; }

/* under the map: 49 + 2 + 49 = 100 */
.np-half      { float:left;  width:49%; margin-right:2%; }
.np-half-last { float:right; width:49%; }

.np-full { width:100%; }

.navname   { font-size:17px; color:#3b3630; line-height:21px; }
.navsub    { color:#8a7f72; padding-top:2px; font-size:11px; }
.navfield  { padding:2px 0; overflow:hidden; }
.navfield-k{ float:left; width:104px; color:#6b6157; }
/* no fixed width - the value takes whatever the panel has left */
.navfield-v{ display:block; overflow:hidden; color:#3b3630; }

/* a sentence of explanation under the fields of a panel */
.navnote   { color:#8a7f72; font-size:11px; line-height:15px;
             padding-top:7px; margin-top:6px;
             border-top:1px solid #efe8dc; }
.scanline  { padding:2px 0 4px 0; }
.scanlock  { padding:2px 0 4px 0; color:#b8ab98; }

/* --- the map ---------------------------------------------- */
```

```
/* The raster is a CSS background, not DOM elements: two
   linear-gradients drawing a 1px line each, tiled by
   background-size. Solid colour is declared first as the
   fallback for any document mode without gradient support. */
/* FLUID UP TO A CEILING. The chart grows with the window but
   stops at STAGE.map (900 x 400) and centres itself in the rest -
   the same treatment the ship slice gets. See the CENTRE STAGE
   house rule in the script. Bodies live in a fixed 638 x 278
   world and are projected onto whatever size the box settles at,
   so the stars never drift off their routes.

   The perspective raster is painted by applyRaster() as a stack
   of gradient layers, not declared here, because every line has
   to be recomputed from the live box size. The flat colour below
   is the fallback if gradients are unavailable. */
#map {
    position:relative; width:100%; max-width:900px;
    min-height:240px; margin:0 auto 12px auto;
    background-color:#ece2d1;
    background-repeat:no-repeat;
    background-size:100% 100%;
    background-position:0 0;
    border:1px solid #8c6f4e;
    overflow:hidden;
    cursor:move;        /* the whole chart is draggable */
}

/* Zero-sized anchors, NOT sheets. A full-size transparent layer
   swallows every click meant for the bodies underneath. Do not
   give .maplayer a width or height. */
.maplayer { position:absolute; left:0; top:0; width:0; height:0; }

/* routes are drawn as dot trails - no transforms, no canvas,
   works in every document mode an HTA can land in */
.rdot     { position:absolute; width:2px; height:2px; background:#d3c3a8; }
.rdot-sel { background:#b9846b; width:3px; height:3px; }
/* authored lanes on the wide layers - deliberately weaker than a
   plotted course, they are context and not a route you can fly */
.rdot-faint { background:#e3d8c6; }
/* cosmetic star clusters on the galaxy layer. They live in the
   routes layer, underneath everything, so they can never take a
   click meant for a marker. */
.cdot     { position:absolute; width:2px; height:2px; background:#cabca2; }

.mapdot {
    position:absolute; background:#6b5338;
    -moz-border-radius:8px; -webkit-border-radius:8px; border-radius:8px;
    border:2px solid #ffffff; cursor:pointer;
}
.mapdot:hover     { background:#b05a42; }
.mapdot-station   { background:#8c6f4e; -moz-border-radius:0; -webkit-border-radius:0; border-
radius:0; }
```

```
/* a whole star system, seen from the sector scale: a light disc
   inside a coffee ring, so it reads as a container of things
   rather than as one object */
.mapdot-sys       { background:#faf7f2; border-color:#8c6f4e; }
/* a galactic region: same idea, squared off, because a region is
   an area on a chart and not a place you can park at */
.mapdot-region    { background:#f4efe6; border-color:#6b5338;
                    -moz-border-radius:0; -webkit-border-radius:0; border-radius:0; }
.mapdot-sel       { background:#b05a42; border-color:#3b3630; }
.maphere {
    position:absolute; border:2px solid #7f9161; background:transparent;
    -moz-border-radius:14px; -webkit-border-radius:14px; border-radius:14px;
}
.maplabel {
    position:absolute; font-size:10px;
 color:#6b5338;
    white-space:nowrap; cursor:pointer;
}
/* ambient furniture - cosmetic only */
.rock   { position:absolute; background:#c9b99f;
          -moz-border-radius:2px; -webkit-border-radius:2px; border-radius:2px; }
.sat    { position:absolute; width:3px; height:3px; background:#7f9161; }
.beacon { position:absolute; width:4px; height:4px; background:#b05a42; }

.mapzoom  { position:absolute; right:7px; top:7px; z-index:5; }
/* directly below the + / - pair. Own container rather than shared,
   so the zoom buttons keep their 21px square and these keep a
   width that fits a word. */
.maplayers { position:absolute; right:7px; top:62px; z-index:5; }
.lbtn {
    display:block; width:64px; height:19px; line-height:17px;
    text-align:center; margin-bottom:4px;
    background:#faf7f2; border:1px solid #8c6f4e; color:#6b5338;
    font-size:9px; letter-spacing:1px; cursor:pointer;
    font-family:"Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
}
.lbtn:hover { background:#f0e7d8; }
.lbtn-on    { background:#ffffff; border-color:#6b5338; color:#3b3630; font-weight:bold; }
.zbtn {
    display:block; width:21px; height:21px; line-height:19px;
    text-align:center; margin-bottom:4px;
    background:#faf7f2; border:1px solid #8c6f4e; color:#3b3630;
    font-size:14px; cursor:pointer;
    font-family:"Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
}
.zbtn:hover { background:#f0e7d8; }
/* Both are scenery. Text painted over the chart must never be
   the thing a click lands on - Entry 015 / 023 / 026, same trap
   three times. They are children of #map, so a drag that starts
   on them still bubbles and still pans. */
.mapscale   { position:absolute; left:8px;  top:7px;    font-size:10px;
              color:#9c8f7c; z-index:5; pointer-events:none; }
```

```
.maphint    { position:absolute; right:8px; bottom:6px; font-size:10px;
              color:#9c8f7c; z-index:5; pointer-events:none; }

/* ============================================================
   SECTION 10-B : THE ENGINE MACHINE
   A fixed 650x390 illustration made of positioned divs. It never
   scales - centre-stage house rule - and every piece of it is
   scenery except the condition meter. Circles are border-radius,
   angles are Math.cos/sin scattering small dots: Trident gives us
   no transform and no canvas worth trusting, so a needle is three
   dots on a radius and a turbine blade is a dot on a circle.
   ============================================================ */

#machine {
    position:relative; margin:0 auto 12px auto;
    background:#fdfbf7;
    border:1px solid #8c6f4e;
    overflow:hidden;
}
/* scenery must never be the thing a click lands on */
.mach-part {
    position:absolute; pointer-events:none;
    font-family:"Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
}

.mach-pipe   { background:#efe8dc; border:1px solid #cbb89c; }
.mach-valve  { background:#f4efe6; border:1px solid #8c6f4e; }
.mach-rod    { background:#cbb89c; }
.mach-spoke  { background:#cbb89c; }
.mach-brk    { background:#cbb89c; }
.mach-mount  { background:#cbb89c; border:1px solid #a89476; }
.mach-rivet  { background:#cbb89c;
               -moz-border-radius:2px; -webkit-border-radius:2px; border-radius:2px; }
.mach-flow   { background:#a89476;
               -moz-border-radius:2px; -webkit-border-radius:2px; border-radius:2px; }
.mach-fin    { background:#ece5d8; border-left:1px solid #cbb89c; }
.mach-tick   { background:#cbb89c;
               -moz-border-radius:3px; -webkit-border-radius:3px; border-radius:3px; }
.mach-nd     { background:#8c6f4e;
               -moz-border-radius:2px; -webkit-border-radius:2px; border-radius:2px; }
.mach-scale  { background:#ddd2bf; }

/* dial faces and the indicator bus */
.mach-dial   { border:1px solid #8c6f4e; background:#fdfbf7;
               -moz-border-radius:50%; -webkit-border-radius:50%; border-radius:50%; }
.mach-hub2   { background:#cbb89c; border:1px solid #8c6f4e;
               -moz-border-radius:50%; -webkit-border-radius:50%; border-radius:50%; }
.mach-lamp   { background:#efe8dc; border:1px solid #cbb89c;
               -moz-border-radius:50%; -webkit-border-radius:50%; border-radius:50%; }
.mach-lamp-on  { background:#7f9161; border-color:#6b7d50; }
.mach-lamp-amb { background:#c08a3e; border-color:#a8762f; }
```

```
/* the core, as rings inside rings */
.mach-ring {
   border:2px solid #8c6f4e; background:#faf7f2;
   -moz-border-radius:50%; -webkit-border-radius:50%; border-radius:50%;
}
.mach-ring2 {
   border:1px dashed #cbb89c;
   -moz-border-radius:50%; -webkit-border-radius:50%; border-radius:50%;
}
.mach-hub {
   background:#8c6f4e; border:1px solid #6b5338;
   -moz-border-radius:50%; -webkit-border-radius:50%; border-radius:50%;
}
.mach-core {
   background:#fdfbf7;
   -moz-border-radius:50%; -webkit-border-radius:50%; border-radius:50%;
}
.mach-bolt {
   background:#f4efe6; border:1px solid #8c6f4e;
   -moz-border-radius:50%; -webkit-border-radius:50%; border-radius:50%;
}
.mach-wheel {
   border:2px solid #8c6f4e; background:#f7f2e9;
   -moz-border-radius:50%; -webkit-border-radius:50%; border-radius:50%;
}

.mach-box    { background:#faf7f2; border:1px solid #e4dbcb; }
.mach-cyl    { background:#efe8dc; border:1px solid #cbb89c; }

/* the small print: statistics, captions, gauge labels */
.mach-mini   { font-size:8px; letter-spacing:1px; color:#a2947f; }
.mach-mini2  { font-size:8px; color:#b3a794; }
.mach-cap    { font-size:8px; letter-spacing:1px; color:#a2947f;
               text-align:center; }
.mach-stat   { font-size:9px; overflow:hidden; color:#6b6157;
               border-bottom:1px solid #f0e9dd; }
.mach-sk     { float:left; }
.mach-sv     { float:right; color:#3b3630;
               font-family:Consolas, "Courier New", monospace; }

.mach-track  { background:#ece5d8; }
.mach-fillx  { background:#cbb89c; }

/* THE ONE LIVE READING */
.mach-track2 { background:#ece5d8; border:1px solid #ddd2bf; }
.mach-fill2  { background:#7f9161; }
.mach-pct    { font-size:26px; color:#3b3630; text-align:right;
               font-family:Consolas, "Courier New", monospace; }
.mach-word   { font-size:10px; letter-spacing:2px; line-height:14px;
               text-align:right; }
.mach-plate  { font-size:8px; letter-spacing:1px; color:#b3a794;
               font-family:Consolas, "Courier New", monospace; }
```

```
/* ============================================================
   SECTION 10-C : THE FILTER BANK
   A fixed 600x360 illustration. Scenery is .o2-part and takes no
   clicks; the three install buttons sit in .o2-btnwrap, which is
   the only thing in the box that does.
   ============================================================ */
#o2mach {
   position:relative; margin:0 auto 12px auto;
   background:#fdfbf7; border:1px solid #8c6f4e; overflow:hidden;
}
.o2-part   { position:absolute; pointer-events:none;
          font-family:"Segoe UI", Tahoma, Geneva, Verdana, sans-serif; }
.o2-btnwrap{ position:absolute; z-index:5; }

.o2-duct   { background:#efe8dc; border:1px solid #cbb89c; }
.o2-louvre { background:#cbb89c; }
.o2-flow   { background:#e4dbcb; border:1px solid #cbb89c; }
.o2-mini   { font-size:8px; letter-spacing:1px; color:#a2947f; }

/* the cabin air: stale by default, clean while the bank runs */
.o2-air       { background:#efe7d8; border:1px solid #ddd2bf; }
.o2-air-clean { background:#f1f6ea; border-color:#cfdcc0; }
.o2-dust   { position:absolute; width:3px; height:3px; background:#b9a893; }
.o2-streak { position:absolute; width:72px; height:2px; background:#cfdcc0; }
.o2-airword{ position:absolute; right:10px; top:6px;
          font-size:11px; letter-spacing:2px; }
.o2-airsub { position:absolute; right:10px; top:19px;
          font-size:8px; color:#a2947f; }
.o2-airtrack { position:absolute; left:8px; top:32px;
            width:544px; height:14px;
            background:#e6ddcd; border:1px solid #d3c6b1; }
.o2-airfill  { position:absolute; left:9px; top:33px; height:12px; }
.o2-airnotch { position:absolute; top:32px; width:1px; height:14px;
            background:#fdfbf7; }
.o2-airlab { position:absolute; left:8px; top:8px;
          font-size:8px; letter-spacing:1px; color:#a2947f; }

/* a slot housing and what sits in it */
.o2-slot   { background:#faf7f2; border:1px solid #cbb89c; }
.o2-well   { background:#f4efe6; border:1px solid #cbb89c; }
.o2-well-bare { background:#fdfbf7; border:1px dashed #cbb89c; }
.o2-can    { background:#e4dbcb; border:1px solid #8c6f4e;
          -moz-border-radius:8px; -webkit-border-radius:8px;
          border-radius:8px; }
.o2-can-live { background:#dfeccd; border-color:#7f9161; }
.o2-neck   { background:#cbb89c; border:1px solid #8c6f4e; }
.o2-state  { font-size:10px; letter-spacing:1.5px; text-align:center; }
.o2-track  { background:#ece5d8; border:1px solid #ddd2bf; }
.o2-fill   { background:#7f9161; }

/* the proportion bar used inside the reading panels */
```

```
.navbar    { height:8px; background:#ece5d8; border:1px solid #ddd2bf;
             margin:7px 0 3px 0; }
.navbar-in { height:6px; }

/* ============================================================
   SECTION 10-D : CREW QUARTERS
   A 560x300 hallway and a sixteen-row sprite built out of five
   pixel divs. Scenery is .cq-part and takes no clicks; the
   roster rows, the job select and the order toggles are the
   live parts and sit outside the stage.
   ============================================================ */
#hallway {
    position:relative; margin:0 auto 12px auto;
    background:#fdfbf7; border:1px solid #8c6f4e; overflow:hidden;
}
.cq-part  { position:absolute; pointer-events:none;
            font-family:"Segoe UI", Tahoma, Geneva, Verdana, sans-serif; }
.cq-wall  { background:#faf7f2; }
.cq-floor { background:#f0e9dd; border-top:1px solid #cbb89c; }
.cq-board { background:#e4dbcb; }
.cq-skirt { background:#e9e0d0; border-top:1px solid #d8cab4; }
.cq-light
 { background:#f7f2e9; border:1px solid #cbb89c; }
.cq-glow  { background:#f4efe6; }
.cq-door  { background:#f4efe6; border:2px solid #8c6f4e; }
.cq-dpane { background:#faf7f2; border:1px solid #cbb89c; }
.cq-dhand { background:#cbb89c; }
.cq-lock  { background:#f4efe6; border:1px solid #8c6f4e; }
.cq-lvent { background:#ddd2bf; }
.cq-lhand { background:#8c6f4e; }
.cq-lnum  { font-size:8px; letter-spacing:1px; color:#b3a794; }
.cq-mini  { font-size:8px; letter-spacing:1px; color:#a2947f;
            text-align:center; }
.cq-shadow{ background:#e4dbcb;
            -moz-border-radius:50%; -webkit-border-radius:50%;
            border-radius:50%; }
.cq-scene { position:absolute; left:0; top:0; width:560px; height:300px;
            pointer-events:none; }
.cq-px    { position:absolute; }

/* --- the roster ------------------------------------------- */
.cq-pay   { background:#faf7f2; border:1px solid #efe8dc;
            padding:5px 7px 6px 7px; margin-bottom:6px; }
.cq-payh  { font-size:9px; letter-spacing:1px; color:#8c6f4e;
            font-weight:bold; padding-bottom:3px; }
.cq-payrow{ overflow:hidden; padding:1px 0; }
.cq-payk  { float:left;  font-size:10px; color:#8a7f72; }
.cq-payv  { float:right; font-size:10px; color:#3b3630;
            font-family:Consolas, "Courier New", monospace; }
.cq-row   { padding:5px 6px; border-bottom:1px solid #f0e9dd;
            cursor:pointer; overflow:hidden; }
.cq-row:hover   { background:#f7f2e9; }
```

```
.cq-row-sel     { background:#f4efe6; border-left:3px solid #8c6f4e;
                  padding-left:3px; }
.cq-dot   { display:inline-block; width:9px; height:9px; margin-right:6px;
            -moz-border-radius:5px; -webkit-border-radius:5px;
            border-radius:5px; }
.cq-nm    { color:#3b3630; font-size:12px; }
.cq-flag  { float:right; font-size:9px; color:#8c6f4e;
            background:#f4efe6; border:1px solid #cbb89c;
            padding:0 4px; letter-spacing:.5px; }
.cq-rl    { color:#a2947f; font-size:10px; padding:2px 0 0 15px; }

/* --- the order pad ---------------------------------------- */
.cq-sub   { font-size:13px; color:#3b3630; padding-bottom:6px; }
.cq-lab   { font-size:9px; letter-spacing:1px; text-transform:uppercase;
            color:#8c6f4e; padding:8px 0 4px 0; }
.cq-sel   { width:100%; font-size:12px; font-family:inherit;
            color:#3b3630; border:1px solid #cbb89c; background:#ffffff; }
.cq-cost  { font-size:10px; color:#a2947f; padding-top:3px; }
.cq-ord   { padding:4px 6px; margin-top:3px; cursor:pointer;
            border:1px solid #e4dbcb; background:#ffffff;
            overflow:hidden; font-size:11px; color:#6b6157; }
.cq-ord:hover { background:#f7f2e9; border-color:#cbb89c; }
.cq-ord-on    { background:#f4efe6; border-color:#8c6f4e; color:#3b3630; }
.cq-box   { display:inline-block; width:12px; height:12px; line-height:11px;
            text-align:center; font-size:10px; color:#8c6f4e;
            border:1px solid #cbb89c; background:#ffffff; margin-right:7px; }
.cq-ordl  { display:inline-block; }
.cq-ordc  { float:right; color:#a2947f; font-size:10px; }
.cq-num   { display:inline-block; width:34px;
            font-family:Consolas, "Courier New", monospace; color:#3b3630; }

.cq-row-busy    { background:#faf7f2; }
.cq-row-busy .cq-nm { color:#a2947f; }
.cq-row-busy .cq-rl { color:#bfb3a1; font-style:italic; }
.cq-flag-busy   { color:#6b6157; background:#efe8dc; border-color:#ddd2bf;
                  font-family:Consolas, "Courier New", monospace; }
.cq-closed      { padding:10px 2px; color:#8a7f72; font-size:12px;
                  line-height:18px; }
.cq-ord-off { opacity:.5; filter:alpha(opacity=50); cursor:default;
              background:#faf7f2; }
.cq-ord-off:hover { background:#faf7f2; border-color:#e4dbcb; }
.cq-ordn  { color:#a2947f; font-size:9px; padding:2px 0 0 19px; }
.cq-sub2  { float:right; color:#8c6f4e;
            font-family:Consolas, "Courier New", monospace; }
.cq-talkrow { overflow:hidden; }
.cq-talkbtn { float:left; width:210px; padding:7px 0; text-align:center;
              border:1px solid #8c6f4e; background:#ffffff;
              color:#3b3630; font-size:12px; cursor:pointer; }
.cq-talkbtn:hover { background:#f6efe3; }
.cq-talkbtn-off   { border-color:#e4dbcb; color:#b3a794; cursor:default;
                    background:#faf7f2; }
.cq-talkbtn-off:hover { background:#faf7f2; }
```

```
.cq-says  { margin-left:224px; padding:8px 0 0 0; color:#5c5145;
          font-size:12px; font-style:italic; }

/* --- the target action band -------------------------------
   One centred row of task buttons at the very bottom of the
   screen, one per thing that can be done TO the selected body.
   Inline-block rather than float, so the row stays centred
   whatever number of buttons it ends up holding - travel and
   scan today, repair or hail later, no layout change needed.
   font-size:0 on the row kills the whitespace gap Trident leaves
   between inline-blocks; the cell puts the size back. */
.actionbar  { text-align:center; padding:2px 0 6px 0; }
.actionrow  { text-align:center; font-size:0; }
.actioncell { display:inline-block; vertical-align:top;
            width:200px; margin:0 5px; font-size:12px; }
.actionnote { text-align:center; padding-top:8px; }

/* --- idle screen navigation strip ---------------------------- */
#navstrip {
    width:100%; margin:12px 0 0 0;
    background:#faf7f2; border:1px solid #e4dbcb;
    padding:8px 10px; overflow:hidden;
}
#navstrip:hover { border-color:#cbb89c; background:#f7f2e9; }
.ns-label { font-size:10px; letter-spacing:.7px; text-transform:uppercase; color:#8a7f72; }
.ns-main  { font-size:13px; color:#3b3630; padding-top:3px; }
.ns-right { float:right; text-align:right; color:#a2947f; font-size:11px; padding-top:9px; }

/* temporary test room marker */
.room-temp { border:1px dashed #b05a42 !important; }

</style>
</head>

<body>

<div id="taskbar"></div>
<div id="sidepanel"></div>
<div id="sidefoot"></div>
<div id="content"></div>
<div id="statusbar"></div>

<script type="text/javascript">
/* ================================================================
   SPACESHIP COMMAND
   Version 1 - INTERFACE SHELL

   SCOPE OF THIS BUILD
   This file builds the window, the idle screen, the tab system and
   the side panel only. It deliberately contains NO action economy,
   NO event engine, NO quests, NO stations, NO economy and NO
   save/load. Those arrive in later phases and have reserved,
```

```
   clearly marked sections at the bottom of this file.

   ABSOLUTE RULE ALREADY ENFORCED HERE
   The idle screen is free. Nothing on it consumes an action and
   nothing on it advances game state. The only thing that moves is
   cosmetic dot drift, which touches no data the game cares about.

   LANGUAGE CONSTRAINT
   Trident / IE engine. ES5 only. var only, no let or const, no
   arrow functions, no template literals, no promises, no
   Array.forEach / map / indexOf, no classList, no addEventListener
   without fallback. Plain for loops throughout.
   ================================================================ */


/* ================================================================
   SECTION A : SHIP DEFINITION DATA
   Everything the diagram draws is read from these arrays. Adding a
   room later means adding one object here, not editing the render
   code. Geometry is in pixels relative to the hull box.
   ================================================================ */

var SHIP = {
    name     : "SC Tessellate",
    className: "Kestrel-series light hauler",
    registry : "TX-4471",
    commander: "Commander",
    /* spec sheet - read by the Vessel drop-down in the sidebar */
    built    : "2387",
    yard     : "Halvan Reach Yards, slip 9",
    refit    : "2391 - spine and dorsal pod added",
    length   : "212 m",
    beam     : "64 m",
    height   : "48 m",
    mass     : "9,400 t laden",
    decks    : "4",
    berths   : "14",
    drive    : "Twin nacelle, single primary core"
};

/* ----------------------------------------------------------------
   THE HULL, AS DATA
   A side cross-section of a ship built around a long main body:
   a dorsal spine riding on top, a ventral pod slung underneath,
   two nacelles on stub pylons, and a command lens at the bow.
   Recognisably of the same family as the ships everyone pictures
   when you say starship - and deliberately not one of them: the
   nacelles sit ABOVE and BELOW the axis rather than side by side,
   and the lens is a bow section, not a saucer bolted on a neck.
   Rooms are positioned in STAGE coordinates, not hull-relative,
   because there is no longer one hull to be relative to.
   ---------------------------------------------------------------- */
```

```
var HULL = [
    { id:"nac-d",  cls:"hp-nacelle", x:140, y:6,   w:420, h:30  },
    { id:"pyl-d1", cls:"hp-pylon",   x:214, y:36,  w:18,  h:12  },
    { id:"pyl-d2", cls:"hp-pylon",   x:468, y:36,  w:18,  h:12  },
    { id:"spine",  cls:"hp-spine",   x:200, y:46,  w:580, h:58  },
    { id:"body",   cls:"hp-body",    x:90,  y:102, w:810, h:256 },
    { id:"pod",    cls:"hp-pod",     x:240, y:358, w:480, h:58  },
    { id:"pyl-v1", cls:"hp-pylon",   x:300, y:416, w:18,  h:8   },
    { id:"pyl-v2", cls:"hp-pylon",   x:642, y:416, w:18,  h:8   },
    { id:"nac-v",  cls:"hp-nacelle", x:230, y:424, w:420, h:30  },
    { id:"lens",   cls:"hp-lens",    x:850, y:150, w:290, h:160 }
];

/* Rooms carried over from the small ship keep their names and
   their jobs. Everything the enlargement added is a numbered
   shell - a room that exists, can be opened and read, and has
   nothing in it yet. They are named Room 1..15 on purpose: an
   empty room with an invented name is a promise, an empty room
   with a number is an inventory. */
var ROOMS = [
    /* --- MAIN DECK A ---------------------------------- */
    { id:"engine", name:"Engine Room", x:100, y:112, w:160, h:94,

    systems:"Propulsion / Power", tag:"AFT", tone:"#f4efe6" },
    { id:"o2", name:"O2 Filter Room", x:268, y:112, w:126, h:94,
      systems:"Life support - filters and canister rack",
      tag:"DECK A", tone:"#f4efe6" },
    { id:"quarters", name:"Crew Quarters", x:402, y:112, w:130, h:94,
      systems:"Habitation - shared bunks", tag:"DECK A", tone:"#faf7f2" },
    { id:"mess", name:"Mess Hall", x:540, y:112, w:142, h:94,
      systems:"Galley / Rations", tag:"DECK A", tone:"#faf7f2" },
    { id:"r1", name:"Room 1", x:690, y:112, w:150, h:94,
      systems:"Unassigned", tag:"DECK A", tone:"#fdfbf7", blank:true },

    /* --- MAIN DECK B ---------------------------------- */
    { id:"cargo", name:"Cargo Hold", x:100, y:240, w:308, h:94,
      systems:"Storage - 3 bays", tag:"DECK B", tone:"#f4efe6" },
    { id:"science", name:"Science Room", x:416, y:240, w:160, h:94,
      systems:"Survey records / scan logs", tag:"DECK B", tone:"#faf7f2" },
    /* TEMPORARY. Test harness only - the one room allowed to hold
       a click-farm button. Delete this entry and the "test" branch
       in describeTarget to remove it cleanly. */
    { id:"test", name:"Test Room", x:584, y:240, w:90, h:94,
      systems:"Diagnostic harness", tag:"TEMP", tone:"#f7efe9", temp:true },
    { id:"r2", name:"Room 2", x:682, y:240, w:158, h:94,
      systems:"Unassigned", tag:"DECK B", tone:"#fdfbf7", blank:true },

    /* --- DORSAL SPINE --------------------------------- */
    { id:"r3", name:"Room 3", x:206, y:54, w:106, h:44, small:true,
      systems:"", tag:"SPINE", tone:"#faf7f2", blank:true },
    { id:"r4", name:"Room 4", x:320, y:54, w:106, h:44, small:true,
      systems:"", tag:"SPINE", tone:"#fdfbf7", blank:true },
```

```
    { id:"r5", name:"Room 5", x:434, y:54, w:106, h:44, small:true,
      systems:"", tag:"SPINE", tone:"#faf7f2", blank:true },
    { id:"r6", name:"Room 6", x:548, y:54, w:106, h:44, small:true,
      systems:"", tag:"SPINE", tone:"#fdfbf7", blank:true },
    { id:"r7", name:"Room 7", x:662, y:54, w:106, h:44, small:true,
      systems:"", tag:"SPINE", tone:"#faf7f2", blank:true },

    /* --- VENTRAL POD ---------------------------------- */
    { id:"r8", name:"Room 8", x:252, y:366, w:106, h:44, small:true,
      systems:"", tag:"POD", tone:"#faf7f2", blank:true },
    { id:"r9", name:"Room 9", x:366, y:366, w:106, h:44, small:true,
      systems:"", tag:"POD", tone:"#fdfbf7", blank:true },
    { id:"r10", name:"Room 10", x:480, y:366, w:106, h:44, small:true,
      systems:"", tag:"POD", tone:"#faf7f2", blank:true },
    { id:"r11", name:"Room 11", x:594, y:366, w:106, h:44, small:true,
      systems:"", tag:"POD", tone:"#fdfbf7", blank:true },

    /* --- COMMAND LENS ---------------------------------
       The lens is an ELLIPSE, so a room near its top or bottom
       must be short AND narrow or its corners hang outside the
       curve. Checked corner by corner against the ellipse. */
    { id:"r12", name:"Room 12", x:940, y:160, w:110, h:32, small:true,
      systems:"", tag:"LENS", tone:"#fdfbf7", blank:true },
    { id:"bridge", name:"Bridge", x:890, y:196, w:210, h:68,
      systems:"Navigation / Comms", tag:"FWD", tone:"#f7f2e9" },
    { id:"r13", name:"Room 13", x:940, y:268, w:110, h:32, small:true,
      systems:"", tag:"LENS", tone:"#fdfbf7", blank:true }
];
var CORRIDOR = { x:100, y:210, w:740, h:26 };

/* Crew. Three, scrappy, none of them experts - manifesto 1.9.
   stats here are display values only; nothing mutates them yet. */
var CREW = [
    {
       id:"rell", name:"Rell Aventine", role:"Pilot / Navigation",
       room:"bridge", activity:"Bridge watch", colour:"#8c6f4e",
       sex:"f", hair:"#c9a85c", hair2:"#e2c793", skin:"#f2dcc0", hairStyle:"bun",
       health:96, morale:71, fatigue:34, hunger:22,
       traits:"Steady, quietly stubborn"
    },
    {
       id:"jax", name:"Jax Moreno", role:"Engineer",
       room:"engine", activity:"Inspecting coolant line", colour:"#b05a42",
       sex:"m", hair:"#5c4436", hair2:"#7d6049", skin:"#d2a781", hairStyle:"cowlick",
       health:88, morale:64, fatigue:52, hunger:41,
       traits:"Blunt, fixes things twice",
       /* JOB ASSIGNMENT. A crew member holding the maintenance job
          services the drive on his own initiative - but only once
          condition has fallen to jobAt. An untrained hand leaves it
          far too late. Training at a station, or hiring a better
          engineer, raises jobAt. Assigning the job will be done in
          the crew member's own task window. */
```

```
      job:"maintenance", skill:"Untrained", jobAt:30
    },
    {
      id:"bo", name:"Bo Ferreira", role:"Life Support",
      room:"o2", activity:"Checking the canister rack", colour:"#c08a3e",
      sex:"m", hair:"#8a857e", hair2:"#a8a39c", skin:"#c69070", hairStyle:"thin",
      health:91, morale:69, fatigue:37, hunger:28,
      traits:"Methodical, hoards spares",
      /* JOB ASSIGNMENT - the second one in the game.
         jobAt is read by the JOB, not by the engine: for the
         maintenance job it is a condition percentage, for the
         filter job it is an AIR STAGE (4 clean .. 0 foul). An
         untrained hand waits for stage 1 - BAD - which means
         the crew have already been labouring for ten actions
         before he thinks to do anything about it. Training, or
         a better technician, raises that to 3 or 2. */
      job:"filters", skill:"Untrained", jobAt:1
    },
    {
      id:"suri", name:"Suri Tal", role:"General Duties",
      room:"mess", activity:"Preparing rations", colour:"#7f9161",
      sex:"f", hair:"#2f2a26", hair2:"#4a423b", skin:"#e8c8a4", hairStyle:"long",
      health:93, morale:78, fatigue:28, hunger:15,
      traits:"Cheerful, notices small things"
    }
];

/* ---------------------------------------------------------------
   EVENTS - what the ship has noticed but nobody has looked at yet.
   The sidebar shows the KIND and nothing else. What it actually
   is, what it will cost and what it leads to lives behind the
   task window, which is deliberately still a shell.
   Data-driven per the manifesto: a new event is a row here, not
   a change to any function.
   --------------------------------------------------------------- */
var EVENTS = [
    { id:"ev-1", kind:"Mechanical", source:"Hull sensors",  state:"unexamined" },
    { id:"ev-2", kind:"Contact",    source:"Comms array",   state:"unexamined" },
    { id:"ev-3", kind:"Crew",       source:"Mess hall",     state:"unexamined" }
];

/* ---------------------------------------------------------------
   THE PRICE LIST IS THE COMMANDER'S, NOT MINE.
   Ordered 2026-09-30, verbatim:
     "more jobs that cost lest actions to get, cleaner cost one
      actions, engineer cost 2 actions, Navigation 3 actions,
      Filter 3 actions"
     "as more important the work is as more it will cost"
     "Send to DOC: cost 1 actions but be aware he might not come
      back for 7 actions"
     "Learn and train: depends on what and where, online cost
      less than direclty in a spacestaion ... learning
```

```
      engineering cost more than learning cleaning"
     "Fire. yeah, that one cost 5 actions."
   These figures are not to be re-based, rounded, merged or
   "simplified" as a side effect of fixing something else. An
   order costs what it says here, and the time it takes to carry
   out is that same number - ONE number per order, so the sheet
   can never again show a price and a duration that disagree.
   The medical bay is the single exception: 1 action to send
   them, DOC_AWAY actions before they are back.
   ---------------------------------------------------------------
   JOBS AND ORDERS
   A job is a standing assignment a crew member acts on by
   himself. jobAt is read BY THE JOB: a condition percentage for
   maintenance, an air stage for filters. trainStep/trainCap say
   what a session of training is worth and where it stops.
   cloth/dark are what the sprite in the quarters wears.
   ---------------------------------------------------------------- */
var JOBS = [
    { id:"",           label:"None",       unique:false, cost:1, wage:0,
      learnOn:0,  learnSt:0, feeOn:0, feeSt:0,  trainStep:0,  trainCap:0,  unit:"",
      cloth:"#a2947f", dark:"#857a68" },
    { id:"cleaner",    label:"Cleaner",    unique:false, cost:1, wage:5,
      learnOn:1,  learnSt:2, feeOn:10, feeSt:20,  trainStep:1,  trainCap:5,  unit:"",
      cloth:"#7f9161", dark:"#66754d" },
    { id:"maintenance", label:"Engineer",   unique:true,  cost:2, wage:10,
      learnOn:3,  learnSt:5, feeOn:30, feeSt:50,  trainStep:15, trainCap:75, unit:"%",
      cloth:"#b05a42", dark:"#8f4735" },
    { id:"navigation",  label:"Navigation",  unique:true,  cost:3, wage:15,
      learnOn:4,  learnSt:6, feeOn:40, feeSt:60,  trainStep:1,  trainCap:5,  unit:"",
      cloth:"#8c6f4e", dark:"#6b5338" },
    { id:"filters",    label:"Filters",    unique:true,  cost:3, wage:15,
      learnOn:4,  learnSt:6, feeOn:40, feeSt:60,  trainStep:1,  trainCap:3,  unit:" air",
      cloth:"#c08a3e", dark:"#9c6f2f" }
];
/* THE PRICE IS THE POINT. A cleaner is a word and a mop; an
   engineer is a week of somebody's attention; navigation and
   life support are the two jobs that kill people when they are
   done badly, so they cost the most to hand over and the most
   to teach. Nothing here is free - the bill simply arrives at
   the bottom of the screen instead of at the click. */
var ORDERS = [
    { id:"doc",    label:"Send to Doc",    note:"away up to 7 actions" },
    { id:"learnOn", label:"Learn - aboard",  note:"cheap, one step" },
    { id:"learnSt", label:"Learn - station", note:"two steps, needs a dock" },
    { id:"fire",   label:"Lay off",        note:"off the payroll, stays aboard" }
];
/* An order's price is also how
 long the sheet takes to carry
   out: the bottom button counts the lot down as one bar, and
   nobody takes new orders until it is empty. Giving the order
   costs the commander one action, like every other button.
   The medical bay is the exception - it keeps a body for
```

```
   DOC_AWAY actions AFTER the sheet is finished. */
var DOC_AWAY = 7;

/* ---------------------------------------------------------------
   PAYROLL
   A job is not a favour - the hand holding it draws a wage, and
   the wage is the job's own price times five. Nobody unemployed
   costs anything, and nobody unemployed does anything either:
   that is the whole bargain. Payday lands every PAY.every
   actions, measured on the same counter as everything else.
   If the purse cannot cover the bill the ship pays DOWN the list,
   dearest first - life support and navigation keep the crew
   alive, the cleaner is what a short captain stops paying for -
   and anyone left unpaid comes off the payroll on the spot.
   --------------------------------------------------------------- */
var PAY = {
    every  : 100,   /* actions between paydays */
    hit    : 12     /* morale lost by a hand nobody could pay */
};
      /* changing a job */

/* Hull-exterior stat readouts - manifesto 1.5. All right-clickable. */
var STATS = [
    { id:"hull",   label:"Hull Integrity", value:100, unit:"%", colour:"#7f9161" },
    { id:"shield", label:"Shields",        value:62,  unit:"%", colour:"#c08a3e" },
    { id:"energy", label:"Energy",         value:88,  unit:"%", colour:"#7f9161" },
    { id:"fuel",   label:"Fuel",           value:74,  unit:"%", colour:"#8c6f4e" }
];

var INFOS = [
    { id:"location", label:"Current Location", value:"Kessler Drift - open space" },
    { id:"contacts", label:"Nearby Objects",   value:"No contacts in scan range" },
    { id:"life",     label:"Life Support",     value:"Nominal - atmosphere stable" }
];

/* A resource line is either a plain value or DERIVED from one of
   the stores below. Derived lines cannot drift out of step with
   the room that holds the stuff - there is one copy of the truth
   and the sidebar reads it. */
var RESOURCES = [
    { label:"Shiplets",    from:"purse" },
    { label:"Fuel cells",  value:"12" },
    { label:"Rations",     value:"18" },
    { label:"Repair kits", value:"3" },
    { label:"O2 tanks",    from:"o2" },
    { label:"Goods",       from:"goods" },
    { label:"Survey data", from:"survey" }
];

/* ---------------------------------------------------------------
   STORE 1 : OXYGEN                  (lives in the O2 Filter Room)
   Canisters are the stock; filters are what actually breathes for
```

```
   the crew. A filter loses charge slowly as actions are spent
   anywhere on the ship, and a canister has to be carried over and
   loaded into it BY HAND. Nothing here is automatic - that is the
   point of the room.
   The drain itself belongs to the event/economy phase; the data
   and the room are seeded here so it has somewhere to land.
   ---------------------------------------------------------------- */
var LIFE = {
    canisters : 6,
    capacity  : 10,
    /* THREE SLOTS, THREE STATES.
       empty     - nothing seated, the air gets no help
       installed - a canister is in and the machine is ready
       active    - the machine is running on it
       A canister leaves the rack the moment an install is
       COMMITTED, not when it finishes: it is in the player's
       hands from that point on. */
    slots     : [
       { id:"s0", name:"Slot 1", state:"empty" },
       { id:"s1", name:"Slot 2", state:"empty" },
       { id:"s2", name:"Slot 3", state:"empty" }
    ],
    lifeMax   : 200,    /* a full charge, in actions */
    decay     : 4       /* lost per action spent anywhere aboard */
};

/* ----------------------------------------------------------------
   THE AIR ITSELF
   When the bank dies the cabin does not go foul on the spot - the
   air already in the room holds for a while and then declines in
   four visible steps, five actions each. Twenty actions from a
   dead bank to unbreathable.
   EVERYTHING BAD AIR DOES IS IN THIS ONE TABLE. Index is the
   stage: 4 is clean, 0 is foul. A future effect - fatigue, morale,
   a medical event - is another column here, not a new branch
   somewhere in the engine.
      slow    : multiplier on work done BY THE CREW. The commander
                is not slowed; he is the one giving the orders and
                he can hold his breath. Crew work takes longer.
      jobDrop : how far a crew member's own trigger threshold
                slips. An engineer who waits for 30% waits for
                20% when the air is bad - he is slower to notice
                and slower to care.
   ---------------------------------------------------------------- */
var AIR = {
    holdMax : 20,   /* actions of breathable air after the bank dies */
    step    : 5     /* actions per step of decline                   */
};
var AIR_STAGES = [
    { word:"FOUL",    tone:"#b05a42", slow:2.00, jobDrop:10, harm:5  }, /* 0 */
    { word:"BAD",     tone:"#b05a42", slow:1.50, jobDrop:10, harm:10 }, /* 1 */
    { word:"THIN",    tone:"#c08a3e", slow:1.25, jobDrop:5,  harm:0  }, /* 2 */
```

```
    { word:"FADING",  tone:"#bda43f", slow:1.00, jobDrop:0,  harm:0  }, /* 3 */
    { word:"OPTIMAL", tone:"#7f9161", slow:1.00, jobDrop:0,  harm:0  }  /* 4 */
];
/* harm = one point of health off every crew member every N
   actions. Zero means the air is not hurting anyone yet. */

/* ---------------------------------------------------------------
   STORE 2 : GOODS                    (lives in the Cargo Hold)
   Bought low at one station, sold high at another, or hauled for
   someone else's quest. Volume is the real constraint, not value -
   the hold is a fixed number of cubic metres and a fat cheap lot
   can cost more in space than it earns. Unregistered lots are
   worth the most and are the reason customs exists.
   --------------------------------------------------------------- */
var CARGO = { capacity:60 };

var GOODS = [
    { id:"ore",    name:"Refined ore",         vol:14, value:280,
      kind:"Bulk metal",     risk:false },
    { id:"parts",  name:"Drive components",    vol:6,  value:840,
      kind:"Components",     risk:false },
    { id:"greens", name:"Hydroponic produce",  vol:9,  value:210,
      kind:"Perishable",     risk:false },
    { id:"amps",   name:"Unmarked ampoules",   vol:3,  value:1450,
      kind:"Unregistered",   risk:true  }
];

/* ---------------------------------------------------------------
   STORE 3 : SURVEY DATA              (lives in the Science Room)
   Earned, never bought: filed by a completed deep scan, pulled out
   of debris fields and abandoned hulls, or handed over for finishing
   an exploration job. Packaged here, sold at a station for shiplets.
   --------------------------------------------------------------- */
var SURVEY = [
    { id:"sv1", name:"Coldwell debris field", src:"Salvage",
      value:260, state:"filed" },
    { id:"sv2", name:"Tolme approach traffic", src:"Contract",
      value:90,  state:"packaged" }
];

/* what a deep scan of each class of body is worth as data */
var SURVEY_WORTH = { star:140, station:90, origin:60, system:0, region:0 };

/* ---------------------------------------------------------------
   NAVIGATION BODIES
   World coordinates inside the map viewport. Distance is NOT
   authored - it is derived from the gap between where the ship
   is and where it is going, so the chart stays honest after the
   ship actually moves. See navMetrics().
   --------------------------------------------------------------- */
var MAP = {
    /* w/h/cx/cy are the WORLD, a fixed 638 x 278 drawing board.
```

```
      Body coordinates and every derived distance live in it and
      never change, whatever the window does. */
   w:638, h:278, cx:319, cy:139,
   /* vw/vh are the live pixel size of the map box, re-measured on
      every build and every resize. Rendering scales world -> box;
      navMetrics() ignores them, so stretching the window can
      never alter a travel time. */
   vw:638, vh:278,
   zoom:1, zmin:0.6, zmax:2.0, zstep:0.2,
   here:"kessler",
   layer:"system",      /* which map is on screen - see LAYERS */
   grid:45,             /* raster pitch in px at zoom 1 */
   /* perspective raster: the vanishing point sits above the top
      edge by this fraction of the height, so the chart shows the
      fan running away from the viewer and never the knot where
      the lines meet */
   vpy:0.35,
   rungs:9,             /* depth lines at zoom 1 */
   rungPow:2.1,         /* >1 bunches them toward the horizon */
   pxPerAction:16,      /* map pixels per action of travel */
   lyPerAction:0.42     /* light years per action */
};

/* A deep scan costs one action to commit and this many actions
   spent elsewhere before the readout is ready. Data, not logic -
   better sensors will change this number, not the engine. */
var SCAN_COST = 5;

var BODIES = [
   {
      id:"kessler", type:"origin", name:"Kessler Drift",
      x:210, y:150,
      klass:"Open space - no anchorage",
      details:[
         ["Age",        "Not surveyed"],
         ["Atmosphere", "None - hard vacuum"],
         ["Day length", "Not applicable"],
         ["Gravity",    "Negligible"],
         ["Mean temp",  "-268 C"]
      ],
      scan:["Empty volume. Weak stellar wind.",
            "Nothing here worth stopping for."],
      scanLocked:["Deep gravimetric sweep", "Long-range wake analysis"]
   },
   {
      id:"halvan", type:"star", name:"Halvan's Reach",
      x:100, y:66,
      klass:"Binary pair, unstable",
      details:[
         ["Age",        "1.2 billion years"],
         ["Atmosphere", "Stripped - trace helium"],
         ["Day length",
```

```
"9 h 40 m"],
        ["Gravity",    "1.44 g"],
        ["Mean temp",  "+310 C sunward"]
      ],
      scan:["Heavy radiation wash across the inner orbit.",
          "Salvage reported in the debris ring.",
          "Two derelict hulls, neither answering."],
      scanLocked:["Hull registry identification", "Ore density mapping",
              "Radiation shadow plotting"]
    },
    {
      id:"sirrus", type:"star", name:"Sirrus Minor",
      x:395, y:62,
      klass:"Dim orange dwarf",
      details:[
        ["Age",        "4.1 billion years"],
        ["Atmosphere", "Thin CO2 on the inner body"],
        ["Day length", "31 h 12 m"],
        ["Gravity",    "0.87 g"],
        ["Mean temp",  "-40 C"]
      ],
      scan:["One scorched body in close orbit.",
          "No registered traffic. Quiet."],
      scanLocked:["Subsurface water survey", "Atmospheric composition detail"]
    },
    {
      id:"tolme", type:"station", name:"Tolme Anchorage",
      x:545, y:205,
      klass:"Independent station, class C",
      details:[
        ["Commissioned", "61 years ago"],
        ["Atmosphere",   "Regulated, 1.0 atm"],
        ["Day length",   "24 h standard, artificial"],
        ["Gravity",      "0.95 g by rotation"],
        ["Mean temp",    "+21 C"]
      ],
      scan:["Market, berths and a tavern.",
          "Docking fees quoted on approach.",
          "Traffic moderate. Two ships at berth."],
      scanLocked:["Berth manifest", "Market price index", "Security posture"]
    },
    {
      id:"coldwell", type:"star", name:"The Coldwell",
      x:78, y:245,
      klass:"Collapsed system, cold",
      details:[
        ["Age",        "11.8 billion years"],
        ["Atmosphere", "Frozen solid at surface"],
        ["Day length", "406 h"],
        ["Gravity",    "2.10 g"],
        ["Mean temp",  "-231 C"]
      ],
```

```
      scan:["No stellar output. Long transit.",
            "Three unanswered distress relays, all old."],
      scanLocked:["Relay decryption", "Thermal anomaly trace",
                  "Deep structure scan"]
    }
];

/* ----------------------------------------------------------------
   MAP LAYER : SECTOR
   The neighbours of the system the ship is in. Same coordinate
   world as BODIES (638 x 278) so the projection maths is shared,
   but nothing here is reachable: range:false puts every one of
   them outside sensor and drive range. They exist to be looked at
   and to make the chart feel like it has an outside.
   ---------------------------------------------------------------- */
var SECTOR_BODIES = [
    {
      id:"sec-kessler", type:"system", name:"Kessler System",
      x:300, y:150, home:true, range:false,
      klass:"Current system - five charted bodies",
      details:[["Charted bodies", "5"], ["Traffic", "Negligible"]],
      scan:[], scanLocked:[]
    },
    {
      id:"sec-vance", type:"system", name:"Vance Cluster",
      x:120, y:70, range:false,
      klass:"Binary system - heavy industry",
      details:[["Charted bodies", "9"], ["Traffic", "Heavy"]],
      scan:[], scanLocked:[]
    },
    {
      id:"sec-orrin", type:"system", name:"Orrin Gate",
      x:470, y:60, range:false,
      klass:"Single star - registered waypoint",
      details:[["Charted bodies", "4"], ["Traffic", "Moderate"]],
      scan:[], scanLocked:[]
    },
    {
      id:"sec-pell", type:"system", name:"Pell Hollow",
      x:540, y:180, range:false,
      klass:"Collapsed star - salvage rights disputed",
      details:[["Charted bodies", "2"], ["Traffic", "Unlawful"]],
      scan:[], scanLocked:[]
    },
    {
      id:"sec-bryce", type:"system", name:"Bryce Anchorage",
      x:190, y:240, range:false,
      klass:"Triple star - shipyard and berths",
      details:[["Charted bodies", "11"], ["Traffic", "Heavy"]],
      scan:[], scanLocked:[]
    },
    {
```

```
      id:"sec-marrow", type:"system", name:"Marrow Field",
      x:420, y:235, range:false,
      klass:"Dead system - debris belt, no anchorage",
      details:[["Charted bodies", "Unknown"], ["Traffic", "None"]],
      scan:[], scanLocked:[]
    }
];

/* faint navigation lines between systems - authored, not derived,
   because a lane is a thing people agreed on, not a straight line
   between two points */
var SECTOR_LINKS = [
    ["sec-kessler", "sec-vance"], ["sec-kessler", "sec-orrin"],
    ["sec-kessler", "sec-pell"],  ["sec-kessler", "sec-bryce"],
    ["sec-kessler", "sec-marrow"],
    ["sec-orrin", "sec-pell"],    ["sec-bryce", "sec-marrow"]
];

/* ----------------------------------------------------------------
   MAP LAYER : GALAXY
   Regions, not objects. Nothing at this scale is a destination in
   Version 1 - it is the shape of the world the ship is small in.
   cluster:n scatters n cosmetic star dots around the marker.
   ---------------------------------------------------------------- */
var GALAXY_BODIES = [
    {
      id:"gal-core", type:"region", name:"Galactic Core",
      x:319, y:128, range:false, cluster:16,
      klass:"Dense core - navigation hazardous",
      details:[["Charted systems", "Uncountable"], ["Authority", "Core Compact"]],
      scan:[], scanLocked:[]
    },
    {
      id:"gal-inner", type:"region", name:"Inner Rim",
      x:170, y:76, home:true, range:false, cluster:11,
      klass:"Settled space - contains the Kessler System",
      details:[["Charted systems", "Thousands"], ["Authority", "Rim Assembly"]],
      scan:[], scanLocked:[]
    },
    {
      id:"gal-outer", type:"region", name:"Outer Reach",
      x:500, y:68, range:false, cluster:8,
      klass:"Thin settlement - long crossings",
      details:[["Charted systems", "Hundreds"], ["Authority", "Contested"]],
      scan:[], scanLocked:[]
    },
    {
      id:"gal-verge", type:"region", name:"The Verge",
      x:118, y:212, range:false, cluster:6,
      klass:"Edge of charted space",
      details:[["Charted systems", "Few"], ["Authority", "None"]],
      scan:[], scanLocked:[]
```

```
    },
    {
      id:"gal-shoal", type:"region", name:"Cradle Shoal",
      x:488, y:216, range:false, cluster:9,
      klass:"Deep space shallows - old wreck fields",
      details:[["Charted systems", "Unsurveyed"], ["Authority", "None"]],
      scan:[], scanLocked:[]
    }
];

var GALAXY_LINKS = [
    ["gal-core", "gal-inner"], ["gal-core", "gal-outer"],
    ["gal-core", "gal-shoal"], ["gal-inner", "gal-verge"],
    ["gal-outer", "gal-shoal"]
];

/* ---------------------------------------------------------------
   THE LAYER TABLE
   Each layer is a map in its own right and remembers its own zoom,
   its own pan and its own selection, so switching back and forth
   does not reset the player's place. Only SYSTEM is in range, and
   only SYSTEM can be travelled - it is the only layer whose
   coordinates the distance maths is measured in.
   --------------------------------------------------------------- */
var LAYERS = {
    system : { id:"system", label:"SYSTEM", name:"System",
               bodies:BODIES,        links:[],
               inRange:true,  zoom:1, px:0, py:0, sel:null },
    sector : { id:"sector", label:"SECTOR", name:"Sector",
               bodies:SECTOR_BODIES, links:SECTOR_LINKS,
               inRange:false, zoom:1, px:0, py:0, sel:null },
    galaxy : { id:"galaxy", label:"GALAXY", name:"Galaxy",
               bodies:GALAXY_BODIES, links:GALAXY_LINKS,
               inRange:false, zoom:1, px:0, py:0, sel:null }
};

var LAYER_ORDER = ["system", "sector", "galaxy"];

var OUT_OF_RANGE =
    '<span class="dim">Distance too great. Target out of sensor range.</span>';


/* Cryptic side-panel indicators - manifesto 1.5. No labels, no
   numbers, no tooltips. The player learns these by playing. In this
   build they are static; the event engine will drive them later. */
/* AMBIENT PROCESSES. Things the ship is doing to itself that the
   player did not start - the only kind that CLIMBS. Placeholders
   until the event engine owns them; each will become a real
   countdown then. Unlabelled on purpose: the player is meant to
   notice a line rising, not read a caption. */
var INDICATORS = [
    { pct:22 },
```

```
    { pct:48 },
    { pct:66 },
    { pct:11 }
];


/* ================================================================
   SECTION B : RUNTIME STATE
   One global object, so that serialising it later is trivial.
   In this build it holds interface state only.
   ================================================================ */

/* THE HEARTBEAT.
   Single source of truth for elapsed game time. Deliberately a
   plain global, not a STATE field, so there is exactly one place
   it can be written. The save serialiser will read this directly
   when persistence is built. */
var actionCounter = 0;

var STATE = {
    freshTabs : {},     /* tabs created since the last tab render */
    activeTab : "home",
    tabs      : [],         /* open task tabs */
    feed      : [],         /* activity log lines */
    popups    : 0,          /* running count, used for offsetting */
    spOpen    : { vessel:false, resources:false, crew:false },
    ongoing   : [],         /* ongoing tasks - see Section D-4 */
    navTarget : null,       /* selected map body id, free to change */
    scanned   : {},         /* bodyId -> true once a readout was read */

    /* THE ENGINE. The first system that decays on its own. It
       does not decay with TIME - nothing in this game does - it
       decays with ATTENTION: every fourth action spent anywhere
       aboard costs the drive one percent. */
    engineCondition : 100,
    wearTick        : 0,    /* actions counted toward the next 1% */

    /* LIFE SUPPORT. Running charge of the filter
  bank, in
       actions. 0 means the air is stale and the slots are bare;
       LIFE.lifeMax means it has just been switched on. Like the
       engine it decays with ATTENTION, not with time. */
    filterLife      : 0,

    /* Breathable air left in the cabin itself, in actions. Full
       at boot - the ship has been flying - and it drains from the
       moment there is no bank running to replace it. */
    airHold         : 20,
    rackWarned      : false,/* so an empty rack is reported once */

    /* CREW QUARTERS. Who is being looked at, and what has been
       written on the order sheet but not yet handed out. Staging
```

```
      is free; the whole sheet is paid for in one go. */
    crewSel        : "rell",
    pendingOrders  : {},
    crewTick       : 0,    /* actions counted toward slow crew effects */
    shiplets       : 340,  /* the purse. Nothing earns yet - see Ideas */
    payTick        : 0,    /* actions counted toward the next payday */
    docked         : false /* station training needs this */
};

/* One place to tune the whole wear loop. 4 actions per percent
   means a full 100 -> 0 cycle costs the player 400 actions. */
var ENGINE = {
    wearEvery  : 4,     /* actions per step           */
    wearStep   : 1,     /* percent lost per step      */
    warnBelow  : 50,    /* sidebar bar appears here   */
    perAction  : 10     /* 1 action of work per 10% missing */
};


/* ============================================================
   SECTION C : UTILITIES
   Small IE-safe helpers. No modern DOM API used anywhere.
   ============================================================ */

function $(id) { return document.getElementById(id); }

function make(tag, cls, html) {
    var el = document.createElement(tag);
    if (cls)  { el.className = cls; }
    if (html !== undefined && html !== null) { el.innerHTML = html; }
    return el;
}

function on(el, type, fn) {
    if (el.addEventListener) { el.addEventListener(type, fn, false); }
    else if (el.attachEvent) { el.attachEvent("on" + type, fn); }
}

function rightClick(el, fn) {
    el.oncontextmenu = function () { fn(); return false; };
}

function clear(el) { while (el.firstChild) { el.removeChild(el.firstChild); } }

function pad2(n) { return (n < 10 ? "0" : "") + n; }

function stamp() {
    var d = new Date();
    return pad2(d.getHours()) + ":" + pad2(d.getMinutes()) + ":" + pad2(d.getSeconds());
}

function findById(list, id) {
```

```
    var i;
    for (i = 0; i < list.length; i++) {
        if (list[i].id === id) { return list[i]; }
    }
    return null;
}

function bar(pct, colour) {
    var w = pct; if (w < 0) { w = 0; } if (w > 100) { w = 100; }
    return '<div class="bar"><div class="bar-in" style="width:' + w +
          '%;background:' + colour + ';"></div></div>';
}


/* ================================================================
   SECTION C-2 : FLUID LAYOUT
   The window frame is fluid through CSS alone. Two things cannot
   be: the activity feed and the star map, because both are empty
   boxes that have no content to push them open. They are measured
   and sized here instead, on build and on every resize.

   The method is the same for both: park the element at its
   minimum, measure how tall everything ELSE on the screen is,
   and hand the leftover to the element. No magic numbers, so
   panels can be added or removed later without touching this.
   ================================================================ */

/* ----------------------------------------------------------------
   HOUSE RULE : CENTRE STAGE GRAPHICS
   A graphic in the middle of a screen does NOT stretch to fill the
   window. It has a ceiling and is centred in whatever space it is
   given; only the panels and the text around it are fluid. A hull
   drawing or a star chart blown up to 1800px reads as a poster,
   not as an instrument.

   The ship slice set the precedent - fixed at 836 x 344 - and the
   chart follows it with a ceiling instead of a fixed size, so it
   can still shrink on a small window. Every future diagram,
   viewport or animation belongs in this table.
   ---------------------------------------------------------------- */
var STAGE = {
    ship   : { w:1160, h:460 }, /* fixed - see #stage in the CSS   */
    map    : { w:900, h:400 },  /* ceiling - mirrors #map max-width */
    engine : { w:650, h:390 },  /* fixed - the machine never scales */
    o2     : { w:600, h:360 },  /* fixed - the filter bank         */
    crew   : { w:560, h:300 }   /* fixed - the quarters hallway    */
};

var RESIZE = { timer:null };

/* Trident fires onresize continuously while a window is dragged.
   Coalesce into one relayout, or the map redraws hundreds of
```

```
  times for a single drag. */
function onWindowResize() {
   if (RESIZE.timer) { window.clearTimeout(RESIZE.timer); }
   RESIZE.timer = window.setTimeout(applyLayout, 90);
}

function applyLayout() {
   RESIZE.timer = null;
   if ($("map"))  { layoutNav();  return; }
   if ($("feed")) { layoutIdle(); }
}

/* how much room the content area has, minus the screen's padding */
function contentSpace() {
   var host = $("content");
   return host ? host.clientHeight : 0;
}

/* The feed gave up its height to the ship. It is a glance at the
  last few lines now, not a reading panel - the bigger hull needs
  the room more than the log does. Leftover space is left where
  it falls, exactly as on the nav screen: nothing stretches. */
var FEED = { min:62, max:132 };

function layoutIdle() {
   var host = $("content"), f = $("feed"), scr = $("screen-idle"), other, h;
   if (!host || !f || !scr) { return; }
   f.style.height = FEED.min + "px";
   other = scr.offsetHeight - f.offsetHeight;   /* everything but the feed */
   h = contentSpace() - other - 4;
   if (h > FEED.max) { h = FEED.max; }
   if (h < FEED.min) { h = FEED.min; }
   f.style.height = h + "px";
}

function layoutNav() {
   var host = $("content"), m = $("map"), scr, other, h, cap;
   if (!m) { return; }

   scr = m.parentNode;
   while (scr && scr.className !== "task-screen") { scr = scr.parentNode; }

   /* the sizing step is optional - if the screen shape ever
      changes the chart must still be measured and drawn */
   if (host && scr) {
      m.style.height = "240px";
      other = scr.offsetHeight - m.offsetHeight;   /* all but the map */
      h = contentSpace() - other - 4;

      /* CENTRE STAGE: take the spare height, but never more than
         the ceiling, and never so much that a narrow chart turns
         into a square - the world is roughly 2:1 and the stars
```

```
        would visibly squash. Width is capped by max-width in
        the CSS and the box centres itself. */
      measureMap();
      cap = STAGE.map.h;
      if (Math.round(MAP.vw / 2) < cap) { cap = Math.round(MAP.vw / 2); }
      if (h > cap) { h = cap; }
      if (h < 240) { h = 240; }
      m.style.height = h + "px";
    }

    measureMap();
    applyRaster();
    drawMapRoutes();
    drawMapBodies();
}

/* NOTHING IS STRETCHED TO FILL.
   An earlier pass tried two ways of using the leftover height
   below the chart: padding the frame down to centre it (which
   made the whole screen visibly drop when Navigation was opened
   on a big window), then stretching the frame row to fill the
   content area (which forced Scan Data to full height and left
   the screen exactly as tall as its container - one pixel of
   rounding and a scrollbar appeared).

   Both are gone. The chart is sized, capped and centred
   horizontally; everything simply flows from the top; Scan Data
   is as tall as the text in it and no taller. The leftover sits
   at the bottom of the window where it costs nothing. This is
   also what the idle screen does with the ship slice, so the two
   task windows now agree with each other.
   ---------------------------------------------------------- */

/* The one place the rendered size of the chart is read. */
function measureMap() {
    var m = $("map");
    if (!m) { return; }
    MAP.vw = m.clientWidth  || MAP.w;    /* clientWidth excludes the border */
    MAP.vh = m.clientHeight || MAP.h;
}

/* ----------------------------------------------------------------
   THE PERSPECTIVE RASTER
   A square grid reads as a sheet of graph paper laid flat. This
   one is drawn as a plane running away from the viewer instead:
   depth rungs that bunch toward the top, and a fan of lines
   converging on a vanishing point held ABOVE the top edge, so the
   chart shows the run-away and never the knot where they meet.
   The rungs also fade with depth, which is most of the illusion.

   It is still pure CSS background - no DOM elements, no canvas,
   no transforms, all of which are unreliable across document
```

```
   modes. Each line is one gradient layer sized to the whole box.

   The maths for one line: take the line through the vanishing
   point, take its PERPENDICULAR as the gradient axis, turn that
   axis into a CSS angle, then place a 1px stop where the axis
   crosses the line. Layer count is bounded (16 worst case) by
   widening the fan pitch on wide charts.
   ------------------------------------------------------------- */

function rasterInk(a) { return "rgba(139,111,78," + a + ")"; }

/* one hairline, as a gradient layer. t is measured along the
   gradient axis from where that axis enters the box. */
function rasterLayer(ang, t, ink) {
    var p = Math.round(t);
    return "linear-gradient(" + ang + "deg," +
           "transparent " + p + "px," + ink + " " + p + "px," +
           ink + " " + (p + 1) + "px,transparent " + (p + 1) + "px)";
}

/* the hairline that passes through (vx,vy) and (bx,by) */
function rasterThrough(W, H, vx, vy, bx, by, ink) {
    var ux = bx - vx, uy = by - vy;
    var len = Math.sqrt(ux * ux + uy * uy), dx, dy, ang, rad, L, t;
    if (len <
 0.01) { return ""; }

    dx =  uy / len;                    /* perpendicular of the line */
    dy = -ux / len;

    /* two decimals, not whole degrees: on a 900px chart a rounding
       of half a degree walks the far end of the line about 6px off
       its anchor and the fan stops converging cleanly. */
    ang = Math.round(Math.atan2(dx, -dy) * 18000 / Math.PI) / 100;

    rad = ang * Math.PI / 180;          /* re-derive from the rounded */
    dx  = Math.sin(rad);                /* angle so t cannot drift    */
    dy  = -Math.cos(rad);

    L = Math.abs(W * dx) + Math.abs(H * dy);
    t = (vx - W / 2) * dx + (vy - H / 2) * dy + L / 2;
    return rasterLayer(ang, t, ink);
}

function applyRaster() {
    var m = $("map");
    if (!m) { return; }

    var W = MAP.vw, H = MAP.vh;
    var vx = W / 2, vy = -H * MAP.vpy;    /* vanishing point, off the top */
    var pitch = MAP.grid * MAP.zoom;
    var layers = [], g = "linear-gradient(to bottom", n, k, j, f, y, a;
```

```
    /* depth rungs - one layer, one stop pair per line. Zooming in
       means fewer, wider rungs, same as a bigger grid cell. */
    n = Math.round(MAP.rungs / MAP.zoom);
    if (n < 4)  { n = 4; }
    if (n > 14) { n = 14; }
    for (j = 1; j <= n; j++) {
        f = Math.pow(j / (n + 1), MAP.rungPow);
        y = Math.round(f * H);
        a = (0.06 + 0.12 * f).toFixed(3);        /* far faint, near solid */
        g += ",transparent " + y + "px," + rasterInk(a) + " " + y + "px," +
             rasterInk(a) + " " + (y + 1) + "px,transparent " + (y + 1) + "px";
    }
    layers[layers.length] = g + ")";

    /* the fan. Widening the pitch on a wide chart keeps the number
       of gradient layers bounded whatever the window does. */
    if (pitch < W / 13) { pitch = W / 13; }
    n = Math.ceil((W * 0.62) / pitch) + 1;
    if (n > 7) { n = 7; }
    for (k = -n; k <= n; k++) {
        layers[layers.length] =
            rasterThrough(W, H, vx, vy, vx + k * pitch, H, rasterInk(0.13));
    }

    m.style.backgroundImage = layers.join(",");
}


/* ================================================================
   SECTION D : ACTIVITY FEED
   Informational only. Writing a line does not advance anything.
   ================================================================ */

function logLine(text) {
    STATE.feed[STATE.feed.length] = { t: stamp(), s: text };
    if (STATE.feed.length > 200) { STATE.feed.shift(); }
    renderFeed();
    refreshLogPopup();
    setStatus();     /* so an event logged mid-tick reaches the bar */
}

function renderFeed() {
    var box = $("feed");
    if (!box) { return; }
    /* NEWEST FIRST. The box is four lines tall on a good day, and
       with the oldest at the top the thing that just happened sat
       at the bottom of a two-hundred line scroll. The latest line
       is now the first line and no scrolling is needed to see it. */
    var html = "", i, f;
    for (i = STATE.feed.length - 1; i >= 0; i--) {
        f = STATE.feed[i];
```

```
        html += '<div class="feed-line"><span class="feed-time num">' + f.t +
              '</span>' + f.s + '</div>';
    }
    box.innerHTML = html;
    box.scrollTop = 0;
}


/* ================================================================
   SECTION D-2 : THE ACTION ECONOMY
   The heartbeat of the entire game. One action = one tick of
   everything that is counting.

   HARD RULE: performAction() may only ever be called from inside
   a task window. Nothing on the idle screen may call it, directly
   or indirectly. Looking is free, and that is enforced by simply
   never wiring this to anything on the idle screen.
   ================================================================ */

function performAction(detail) {
    actionCounter++;

    /* every countdown, chain and pending outcome advances here,
       exactly once, before anything is reported to the player */
    tickWorld();

    /* THE FEED IS NOT A CLICK COUNTER. Writing "Action taken -
       Total actions: N" on every single action buried the things
       that actually happened: in a feed four lines deep, the
       engineer starting work or the filter hand throwing the
       switch scrolled away before the player could look at it.
       The tally lives in the status bar, where it belongs. Only
       the DETAIL - which is only ever set for a real commitment
       - reaches the log. */
    if (detail) { logLine(detail + "."); }
    setStatus();
}

/* The event engine hangs off this single call so that nothing
   else ever needs to know how time passes. */
function tickWorld() {
    /* wear BEFORE the task sweep, so a maintenance job finishing
       on this very action still leaves the engine at 100 */
    wearEngine();
    tickFilters();
    tickCrew();
    tickPayroll();
    tickOngoing();
    /* and only THEN may the engineer take the job on himself -
       added before tickOngoing, his own task would lose a step to
       the very action that spawned it */
    autoMaint();
```

```
    autoFilters();
    /* future: chains, persistent conditions, crew needs - all
       advance from here, once per action */
}

/* ---------------------------------------------------------------
   ENGINE WEAR
   Counted in actions, never in seconds. The accumulator lives in
   STATE rather than being derived from actionCounter so that a
   future save file restores mid-step wear exactly as it was.
   ------------------------------------------------------------- */
function wearEngine() {
    STATE.wearTick++;
    if (STATE.wearTick < ENGINE.wearEvery) { return; }

    STATE.wearTick = 0;
    STATE.engineCondition -= ENGINE.wearStep;
    if (STATE.engineCondition < 0) { STATE.engineCondition = 0; }

    paintIndicators();
    paintEngine();
}

/* ---------------------------------------------------------------
   THE FILTER BANK
   Three slots, one charge, two hundred actions of clean air. The
   bank only burns while it is RUNNING - a seated canister sitting
   idle costs nothing. When the charge is gone the slots come out
   bare and the room has to be worked again.
   ------------------------------------------------------------- */
function tickFilters() {
    var i, was;

    /* --- the bank is running: burn the charge ---------------- */
    if (STATE.filterLife > 0) {
        STATE.filterLife -= LIFE.decay;
        if (STATE.filterLife > 0) { refreshLifeInfo(); paintO2(); return; }

        STATE.filterLife = 0;
        for (i = 0; i < LIFE.slots.length; i++) { LIFE.slots[i].state = "empty"; }
        /* the room is still full of good air - it holds, and then
           it goes. This action does not also spend a step of it. */
        STATE.airHold = AIR.holdMax;
        logLine("Filter bank spent. The cabin air will hold for " +
              AIR.holdMax + " actions.");
        refreshLifeInfo();
        paintIndicators();
        paintO2();
        return;
    }

    /* --- no bank: the air in the room is what is left ------- */
```

```
    if (STATE.airHold <= 0) { return; }

    was = airStage();
    STATE.airHold--;
    if (airStage() !== was) {
        /* only the bottom is announced. The steps above it are a
           colour on a bar and a line in the sidebar - the player
           is expected to look, not to be told. */
        if (airStage() === 0) {
            logLine("The cabin air has gone foul. The crew are labouring.");
        }
        paintIndicators();
    }
    refreshLifeInfo();
    paintO2();
}

function slotsIn(state) {
    var i, n = 0;
    for (i = 0; i < LIFE.slots.length; i++) {
        if (LIFE.slots[i].state === state) { n++; }
    }
    return n;
}
function bankReady() {   /* every slot seated, nothing running yet */
    return slotsIn("installed") === LIFE.slots.length;
}
function bankRunning() {
    return STATE.filterLife > 0;
}
/* 4 down to 0. A running bank holds the top step; with no bank
   the stage follows the air left in the room. */
function airStage() {
    if (bankRunning()) { return 4; }
    var st = Math.ceil(STATE.airHold / AIR.step);
    if (st > 4) { st = 4; }
    if (st < 0) { st = 0; }
    return st;
}
function airSpec()  { return AIR_STAGES[airStage()]; }
function airWord()  { return airSpec().word; }
function airTone()  { return airSpec().tone; }
/* how full the green indicator above the slots stands, 0-100 */
function airPercent() {
    if (bankRunning()) { return 100; }
    var p = Math.round((STATE.airHold / AIR.holdMax) * 100);
    if (p < 0) { p = 0; }
    return p;
}

/* ---- what the air does to the people breathing it ---------- */
function crewSlow() { return airSpec().slow; }
```

```
function crewCost(n) {
    var c = Math.ceil(n * crewSlow());
    return c < 1 ? 1 : c;
}
/* a crew member's own trigger point, after the air has had its
   say. Never pushed below 5 - nobody ignores a dead engine. */
function effectiveJobAt(w) {
    var v = w.jobAt - airSpec().jobDrop;
    return v < 5 ? 5 : v;
}
function airPenalty() {
    var sp = airSpec();
    if (sp.slow === 1 && sp.jobDrop === 0) { return "None - the crew work normally"; }
    return "Crew work x" + sp.slow.toFixed(2) + ", thresholds -" + sp.jobDrop;
}

/* one canister leaves the rack per commitment, and the two
   actions that seat it are spent anywhere aboard */
function startInstall(e, i) {
    var sl = LIFE.slots[i];
    if (!sl || sl.state !== "empty") { return; }
    if (findOngoing("o2s" + i))      { return; }
    if (LIFE.canisters <= 0) {
        logLine("No canisters left in the rack.");
        return;
    }
    fireAction(e, "Canister carried to " + sl.name);
    LIFE.canisters--;
    paintResources();
    /* NOT passed as the task target: addOngoing stores
       (target || null) and slot 0 is a perfectly good zero that
       would be thrown away. The index rides in its own field. */
    var task = addOngoing("o2s" + i, "Seating - " + sl.name, 2,
                          "o2slot", sl.id);
    if (task)
 { task.slot = i; }
    refreshLifeInfo();
    paintO2();
}
function installFn(i) {
    return function (e) { startInstall(e, i); };
}

/* THE REWARD. One ordinary action to throw the switch, and then
   ten actions of progress that cost nothing and wear nothing. */
function turnOnFilter(e) {
    if (!bankReady() || bankRunning()) { return; }
    fireAction(e, "Filter bank started");
    runBank("Filters activated. Air purified. +10 Pro-Action.");
}

/* The machine does not care whose hand is on the switch: the
```

```
  burst is the machine's, not the person's. Only the feed line
  differs, and the commander pays an action for his. */
function runBank(line) {
    var i;
    for (i = 0; i < LIFE.slots.length; i++) { LIFE.slots[i].state = "active"; }
    STATE.filterLife = LIFE.lifeMax;

    proAction(10);
    logLine(line);
    refreshLifeInfo();
    paintIndicators();
    paintO2();
}

/* ---------------------------------------------------------------
  THE CREW, ONE ACTION AT A TIME
  Nobody is harmed quickly. Bad air takes a point of health off
  everyone every few actions; a hand doing work he was never
  taught loses heart slowly; a body in the medical bay is simply
  not aboard for a while. Hunger and fatigue have their fields
  and no drain yet - they belong to the needs phase.
  --------------------------------------------------------------- */
/* ---- WAGES ------------------------------------------------ */
function wageOf(c) {
    var j = findById(JOBS, c.job || "");
    return (j && j.wage) ? j.wage : 0;
}
function payrollBill() {
    var i, n = 0;
    for (i = 0; i < CREW.length; i++) { n += wageOf(CREW[i]); }
    return n;
}
function payrollDue() {
    var left = PAY.every - STATE.payTick;
    return left < 0 ? 0 : left;
}

/* Payday. Dearest first, because the jobs that cost most are the
  ones keeping everybody breathing. A hand the purse cannot
  reach loses the job there and then - and walks to the
  corridor, where the player can see exactly what he could not
  afford. */
function tickPayroll() {
    STATE.payTick++;
    if (STATE.payTick < PAY.every) { return; }
    STATE.payTick = 0;

    var list = [], i, j, c, paid = 0, spent = 0, cut = 0;
    for (i = 0; i < CREW.length; i++) {
        if (wageOf(CREW[i]) > 0) { list[list.length] = CREW[i]; }
    }
    if (list.length === 0) {
```

```
      logLine("Payday. Nobody on the payroll - nothing owed.");
      return;
    }
    /* plain insertion sort, dearest first - no Array.sort callback
       worth trusting across Trident document modes */
    for (i = 1; i < list.length; i++) {
      c = list[i];
      for (j = i - 1; j >= 0 && wageOf(list[j]) < wageOf(c); j--) {
        list[j + 1] = list[j];
      }
      list[j + 1] = c;
    }

    for (i = 0; i < list.length; i++) {
      c = list[i];
      if (STATE.shiplets >= wageOf(c)) {
        STATE.shiplets -= wageOf(c);
        spent += wageOf(c);
        paid++;
      } else {
        layOff(c, "could not be paid");
        cut++;
      }
    }
    logLine("Payday. " + spent + " sh to " + paid + " hand" +
        (paid === 1 ? "" : "s") + ". " + STATE.shiplets + " sh left.");
    if (cut > 0) {
      logLine("The purse came up short. " + cut + " hand" +
          (cut === 1 ? " is" : "s are") + " off the payroll.");
    }
    paintIndicators();
    paintCrewScreen();
}

/* OFF THE PAYROLL IS NOT OFF THE SHIP. There is nowhere to put a
   man in deep space: he keeps his bunk, keeps his name on the
   roster, and loses only the work and the wage. */
function layOff(c, why) {
    if (!c.job) { return; }
    var j = findById(JOBS, c.job);
    c.job      = "";
    c.morale   = Math.max(0, c.morale - PAY.hit);
    c.activity = "Idle in the corridor";
    logLine(c.name + " - " + why + " - is off the " +
        (j ? j.label.toLowerCase() : "") + " job.");
}

function tickCrew() {
    var i, c, sp = airSpec();
    STATE.crewTick++;

    /* the air gets into people */
```

```
    if (sp.harm > 0 && STATE.crewTick % sp.harm === 0) {
      for (i = 0; i < CREW.length; i++) {
        CREW[i].health = Math.max(0, CREW[i].health - 1);
      }
      logLine("The air is telling on the crew.");
    }

    /* an untrained hand holding a job loses heart */
    if (STATE.crewTick % 10 === 0) {
      for (i = 0; i < CREW.length; i++) {
        c = CREW[i];
        if (c.job && (c.skill === "Untrained" || !c.skill)) {
          c.morale = Math.max(0, c.morale - 1);
        }
      }
    }
}

function assignedFilterHand() {
    var i;
    for (i = 0; i < CREW.length; i++) {
      if (CREW[i].job === "filters") { return CREW[i]; }
    }
    return null;
}

/* ----------------------------------------------------------------
   THE FILTER HAND
   Same shape as the engineer: a job held on a crew record, acted
   on without being asked, and acted on TOO LATE because nobody
   has trained him. He seats one canister at a time - he has two
   hands - and throws the switch himself once all three are in.
   Everything he does is slowed by the very air he is fixing.
   ---------------------------------------------------------------- */
function autoFilters() {
    var w = assignedFilterHand(), i, sl;
    if (!w || bankRunning())       { return; }
    if (crewBusy(w))               { return; }   /* carrying out orders */
    if (airStage() > w.jobAt)      { return; }   /* not bad enough yet */

    if (bankReady()) {             /* all three in - throw the switch */
      runBank(w.name + " started the filter bank. Air purified. " +
            "+10 Pro-Action.");
      return;
    }
    if (LIFE.canisters <= 0) {
      if (!STATE.rackWarned) {
        logLine(w.name + " went for a canister. The rack is empty.");
        STATE.rackWarned = true;
      }
      return;
    }
```

```
    STATE.rackWarned = false;

    /* one slot at a time, and only if he is not already on one */
    for (i = 0; i < LIFE.slots.length; i++) {
      if (findOngoing("o2s" + i)) { return; }
    }
    for (i = 0; i < LIFE.slots.length; i++) {
      sl = LIFE.slots[i];
      if (sl.state !== "empty") { continue; }
      LIFE.canisters--;
      paintResources();
      var t = addOngoing("o2s" + i, "Seating - " + sl.name,
                         crewCost(2), "o2slot", sl.id);
      if (t) { t.slot = i; t.by = w.name; }
      logLine(w.name + " is seating a canister in " + sl.name + ".");
      refreshLifeInfo();
      paintIndicators();
      paintO2();
      return;
    }
}

function refreshLifeInfo() {
    var n = findById(INFOS, "life");
    if (n) { n.value = lifeSummary(); }
}

/* The assigned engineer starts maintenance HIMSELF once the drive
   has fallen to his threshold. It costs the player no click - only
   the ordinary drain of actions spent elsewhere, like any other
   running work. An untrained hand waits until 30%, which is well
   past the warning line: the ship is already limping by then. */
/* Set while the commander's own click is in flight. startMaint
   spends the action BEFORE it registers the task, and spending an
   action runs tickWorld - which used to let the engineer grab the
   job in that gap and leave TWO maintenance tasks on one engine. */
var MAINT_CLAIM = false;

function autoMaint() {
    var w = assignedEngineer();
    if (MAINT_CLAIM) { return; }          /* the commander is mid-click */
    if (!w || maintRunning()) { return; }
    if (crewBusy(w))          { return; }   /* carrying out orders */
    if (STATE.engineCondition > effectiveJobAt(w)) { return; }
    if (STATE.engineCondition >= 100)              { return; }

    /* bad air makes him both later and slower */
    var t = addOngoing("maint-eng", "Maintenance - " + w.name,
                       crewCost(maintCost()), "maint", "engine");
    if (t) { t.by = w.name; }
    logLine(w.name + " has started maintenance without being asked.");
    paintIndicators();
```

```
    paintEngine();
}

/* 1 action of work per 10% missing, rounded up. A perfect engine
   costs 0, which is the same as saying it cannot be maintained. */
function maintCost() {
    return Math.ceil((100 - STATE.engineCondition) / ENGINE.perAction);
}

/* only one maintenance job at a time, whoever is holding the
   spanner */
function maintRunning() {
    return findOngoing("maint") || findOngoing("maint-eng");
}

/* the Engineer counts as present only if he is actually IN the
   engine room - a crew member elsewhere cannot be ordered here */
function assignedEngineer() {
    var i;
    for (i = 0; i < CREW.length; i++) {
        if (CREW[i].job === "maintenance") { return CREW[i]; }
    }
    return null;
}


/* ================================================================
   SECTION D-4 : ONGOING TASK ENGINE
   A committed process does not run on a clock. It runs on the
   player's attention: it advances only when an action is spent
   somewhere else in the game.
   ================================================================ */

/* ----------------------------------------------------------------
   TASK TYPES - the dispatch table.
   The engine does not know what a task MEANS. It counts down, and
   then asks this table what that meant. Two hooks:
     onDone        - the counter just hit zero, nobody has looked
     onAcknowledge - the player clicked the finished button
   A repair, a hail, a refit or a random encounter is a row here
   plus a button that starts it. The engine itself never changes.
   ---------------------------------------------------------------- */
var TASK_TYPES = {
    travel : {
        onDone       : null,
        onAcknowledge : function (t) { if (t.target) { arriveAt(t.target); } }
    },
    scan : {
        onDone       : null,
        onAcknowledge : function (t) { revealScan(t.target); }
    },
    /* Maintenance has nothing to hand
```

```
over: the engine is simply
      whole again. autoClear tells the engine to retire the task
      the moment it finishes, so the button returns to its idle
      label instead of waiting to be acknowledged. */
    /* a crew member carrying out the orders written for them.
      The payload rides on the task; the effects land when the
      work is finished, not when it was ordered. */
    /* THE SHEET. Only a gate and a bar: it holds the order pad
      shut until all the work on it has been paid for. It does
      not carry anybody's orders any more. */
    crewOrder : {
       autoClear : true,
       quiet     : true,  /* it names itself when it lands */
       onDone    : function (t) {
          logLine("The order sheet is discharged.");
          paintCrewScreen();
       },
       onAcknowledge : null
    },
    /* ONE PAIR OF HANDS. Their own share of the sheet, their own
      countdown, their own release - a one-action errand is over
      in one action even if the sheet around it runs to fifteen. */
    crewJob : {
       autoClear : true,
       quiet     : true,  /* it names itself when it lands */
       onDone    : function (t) {
          var c = findById(CREW, t.who);
          if (c) { applyOrderPayload(c, t.orders); }
          paintCrewScreen();
       },
       onAcknowledge : null
    },
    /* a body in the medical bay. Its own countdown, because the
      doc keeps them long after the order sheet is finished. */
    crewDoc : {
       autoClear : true,
       quiet     : true,  /* it names itself when it lands */
       onDone    : function (t) {
          var c = findById(CREW, t.who);
          if (c) {
             c.health   = Math.min(100, c.health + 12);
             c.activity = "Back on station";
             logLine(c.name + " is back from the medical bay.");
          }
          paintCrewScreen();
       },
       onAcknowledge : null
    },

    /* seating a canister: no readout to collect, so it clears
      itself the moment the second action lands */
    o2slot : {
```

```
        autoClear : true,
        quiet     : true,  /* it names itself when it lands */
        onDone    : function (t) {
           var sl = LIFE.slots[t.slot];
           if (sl) { sl.state = "installed"; }
           logLine("Canister seated in " + (sl ? sl.name : "a slot") + ".");
           refreshLifeInfo();
           paintO2();
        },
        onAcknowledge : null
     },

     maint : {
        autoClear : true,
        quiet     : true,  /* it names itself when it lands */
        onDone    : function (t) {
           STATE.engineCondition = 100;
           STATE.wearTick        = 0;
           logLine("Engine maintenance completed by " +
                 (t.by || "the commander") + ".");
           paintIndicators();
           paintEngine();
        },
        onAcknowledge : null
     }
};

function taskHook(t, name) {
   var h = TASK_TYPES[t.type];
   return (h && h[name]) ? h[name] : null;
}

/* type defaults to the id, so a one-off task needs no extra
   argument; target is whatever the task is being done TO. */
function addOngoing(id, label, total, type, target) {
   if (findOngoing(id)) { return null; }
   var t = { id:id, label:label, total:total, left:total, state:"running",
             type:(type || id), target:(target || null) };
   STATE.ongoing[STATE.ongoing.length] = t;
   paintIndicators();          /* a new line appears in the strip */
   return t;
}

function findOngoing(id) {
   var i;
   for (i = 0; i < STATE.ongoing.length; i++) {
      if (STATE.ongoing[i].id === id) { return STATE.ongoing[i]; }
   }
   return null;
}

function clearOngoing(id) {
```

```
    var i, keep = [];
    for (i = 0; i < STATE.ongoing.length; i++) {
        if (STATE.ongoing[i].id !== id) { keep[keep.length] = STATE.ongoing[i]; }
    }
    STATE.ongoing = keep;
    paintIndicators();          /* its line leaves the strip */
}

/* Some finished tasks have no readout to hand over. Rather than
   leave a "Complete" button the player must clear for no reason,
   the type can declare autoClear and the engine retires it here -
   after the countdown loop, never during it. */
function sweepAutoTasks() {
    var i, t, keep = [], spec;
    for (i = 0; i < STATE.ongoing.length; i++) {
        t    = STATE.ongoing[i];
        spec = TASK_TYPES[t.type];
        if (t.state === "done" && spec && spec.autoClear) { continue; }
        keep[keep.length] = t;
    }
    STATE.ongoing = keep;
}

/* Called once per action, from tickWorld only. */
/* Move every running task forward by n and fire the hooks of
   anything that lands on zero. Split out of tickOngoing because
   an action is no longer the only thing that can advance work:
   the filter bank grants a burst of ten. */
function advanceTasks(n) {
    var i, t, hook, k, spec, done = 0, quiet = 0, list = [];

    /* Walk a SNAPSHOT. An onDone hook is allowed to queue work -
       the medical bay does exactly that - and a task created
       half way through this loop must not be docked a step by
       the very tick that gave birth to it. It also means a hook
       may clear a task without the loop losing its place. */
    for (i = 0; i < STATE.ongoing.length; i++) {
        list[list.length] = STATE.ongoing[i];
    }
    for (i = 0; i < list.length; i++) {
        t = list[i];
        if (t.state !== "running") { continue; }
        t.left -= n;
        if (t.left <= 0) {
            t.left  = 0;
            t.state = "done";
            hook = taskHook(t, "onDone");   /* the engine asks, it does not decide */
            if (hook) { hook(t); }
            done++;
            spec = TASK_TYPES[t.type];
            if (!spec || !spec.quiet) { quiet++; }
        }
```

```
    }
    sweepAutoTasks();          /* retire anything with nothing to collect */

    /* Deliberately vague. The main screen never says what
       finished - the player goes and looks. */
    /* The vague ping exists so the player must go and LOOK - it
       is right for a scan or a crossing. A task that already
       announces itself by name ("Canister seated in Slot 2")
       does not need it as well: two lines for one event is how
       the feed drowned in the first place. */
    for (k = 0; k < quiet; k++) {
        logLine("A ship process has completed.");
    }
    return done;
}

function tickOngoing() {
    var any = (advanceTasks(1) > 0);
    repaintQueue(any);
}

/* every screen that draws the queue, refreshed in one place */
function repaintQueue(any) {
    paintIndicators();          /* every line moved */
    /* a finished task can change what the nav buttons should say,
       so rebuild them rather than leave a stale label on screen */
    if (any && $("nav-actions")) { paintNavPanels(); }
    if ($("eng-actions")) { paintEngine(); }
    if ($("o2-actions"))  { paintO2(); }
    if ($("cq-band"))     { paintCrewScreen(); }
    paintResources();
    updateProgressUI();
}

/* ----------------------------------------------------------------
   PRO-ACTION
   A burst of progress that is NOT an action. The counter does not
   move, the engine does not wear, no red bar advances - only the
   player's own committed work jumps forward. This is the whole
   reward of the filter bank, and the first thing in the game that
   gives time back instead of taking it.
   ---------------------------------------------------------------- */
function proAction(n) {
    var any = (advanceTasks(n) > 0);
    repaintQueue(any);
}

function ongoingPercent(t) {
    if (!t || !t.total) { return 0; }
    return Math.floor(((t.total - t.left) / t.total) * 100);
}
```

```
/* ================================================================
   SECTION D-5 : PROGRESS BUTTON COMPONENT
   Three states: idle (click to commit), running (inert, bar
   fills), done (click to acknowledge and reset). No numbers,
   ever - the bar is the only readout.
   ================================================================ */

var PROGRESS_UI = {};   /* taskId -> live DOM refs for the current screen */

function buildProgressButton(taskId, idleLabel, startFn, enabled, doneLabel) {
   var wrap = make("div", "pbtn");
   var fill = make("div", "pbtn-fill");
   var text = make("div", "pbtn-text", idleLabel);
   wrap.appendChild(fill);
   wrap.appendChild(text);

   PROGRESS_UI[taskId] = {
      wrap:wrap, fill:fill, text:text,
      idleLabel:idleLabel, doneLabel:(doneLabel || "Complete"),
      enabled:(enabled !== false)
   };

   on(wrap, "click", function (e) { onProgressClick(e, taskId, startFn); });
   paintProgress(taskId);
   return wrap;
}

function onProgressClick(e, taskId, startFn) {
   var ui = PROGRESS_UI[taskId];
   var t  = findOngoing(taskId);
   var hook;

   if (t && t.state === "running") { return; }          /* inert */
   if (t && t.state === "done") {                      /* acknowledge */
      /* WHAT a finished task means is not decided here. The type
         table is asked: travel moves the ship, a scan hands over
         its readout, the next one will do something else. */
      hook = taskHook(t, "onAcknowledge");
      if (hook) { hook(t); }
      clearOngoing(taskId);
      paintProgress(taskId);
      logLine("Process acknowledged and cleared.");
      /* the surrounding screen may describe the process too */
      if ($("nav-actions")) {
         drawMapRoutes();
         drawMapBodies();
         paintNavPanels();
      }
      return;
   }
   if (ui && !ui.enabled) { return; }                  /* nothing selected */
```

```
    startFn(e);
}

function paintProgress(taskId) {
    var ui = PROGRESS_UI[taskId];
    if (!ui) { return; }
    var t = findOngoing(taskId);

    if (!t) {
        ui.wrap.className = "pbtn" + (ui.enabled ? "" : " pbtn-off");
        ui.fill.style.width = "0";

        ui.text.innerHTML = ui.idleLabel;
        return;
    }
    if (t.state === "done") {
        ui.wrap.className = "pbtn pbtn-done";
        ui.fill.style.width = "100%";
        ui.text.innerHTML = ui.doneLabel;
        return;
    }
    ui.wrap.className = "pbtn pbtn-run";
    ui.fill.style.width = ongoingPercent(t) + "%";
    ui.text.innerHTML = t.label;
}

function updateProgressUI() {
    var k;
    for (k in PROGRESS_UI) {
        if (PROGRESS_UI.hasOwnProperty(k)) { paintProgress(k); }
    }
    /* the ring on the chart advances with the bar */
    if ($("map-bodies")) { drawMapBodies(); }
}

function setProgressEnabled(taskId, on) {
    var ui = PROGRESS_UI[taskId];
    if (!ui) { return; }
    ui.enabled = on;
    paintProgress(taskId);
}

/* Convenience wrapper for wiring a live action button. Keeps the
   three concerns - feedback, sound, time - in one place so no
   future action can accidentally skip one. */
/* the cosmetics of an action, split out so a BATCH can leave one
   mark without pretending to be one action */
function actionMark(e) {
    try {
        var m = mousePoint(e);
        spawnActionMark(m.x, m.y);
        playActionSound();
```

```
    } catch (err) { /* a lost sparkle is not a lost action */ }
}

function fireAction(e, detail) {
    /* The cosmetics run first because the mark must appear where
       the hand was, but they are fenced off: a throw in any of
       them used to abort the whole handler and the player lost
       the click they paid for. The action is the only part that
       matters and it is now unconditional. */
    actionMark(e);
    performAction(detail);
}


/* ================================================================
   SECTION D-3 : PROTAGONIST MOUSE
   The cursor is the player's physical presence. A spent action
   must be felt, not just counted.
   ================================================================ */

var FX = {
    size     : 11,    /* mark box in px - keep in sync with .amark */
    hold     : 220,   /* ms it sits still at full strength first */
    rise     : 26,
    maxMarks : 5,     /* live marks; each owns a timer, see below */    /* pixels travelled upward */
    duration : 950,   /* ms of the slow rise and fade */
    frame    : 16     /* ms between frames */
};

function mousePoint(e) {
    var ev = e || window.event;
    var sx = 0, sy = 0;
    if (document.documentElement) {
        sx = document.documentElement.scrollLeft || 0;
        sy = document.documentElement.scrollTop  || 0;
    }
    if (!sx && document.body) { sx = document.body.scrollLeft || 0; }
    if (!sy && document.body) { sy = document.body.scrollTop  || 0; }
    return { x: ev.clientX + sx, y: ev.clientY + sy };
}

/* opacity written twice: the modern property and the Trident
   filter, because which one works depends on the document mode
   the HTA ends up in */
function setOpacity(el, v) {
    if (v < 0) { v = 0; }
    if (v > 1) { v = 1; }
    el.style.opacity = v;
    el.style.filter  = "alpha(opacity=" + Math.round(v * 100) + ")";
}

/* Every mark owns a 16ms timer for over a second. Hammering the
```

```
   mouse used to stack a dozen of them on Trident's single thread,
   which is its own way of losing input. The list is capped: the
   oldest mark is retired early rather than letting them pile up. */
var MARKS = [];

function retireMark(rec) {
    var i;
    if (rec.timer) { window.clearInterval(rec.timer); rec.timer = null; }
    if (rec.el && rec.el.parentNode) { rec.el.parentNode.removeChild(rec.el); }
    for (i = 0; i < MARKS.length; i++) {
        if (MARKS[i] === rec) { MARKS.splice(i, 1); return; }
    }
}

function spawnActionMark(x, y) {
    var half = Math.floor(FX.size / 2);
    var top0 = y - half;

    var el = make("div", "amark",
        '<div class="amark-h"></div><div class="amark-v"></div>');
    el.style.left = (x - half) + "px";
    el.style.top  = top0 + "px";
    document.body.appendChild(el);

    var rec = { el:el, timer:null };
    MARKS[MARKS.length] = rec;
    while (MARKS.length > FX.maxMarks) { retireMark(MARKS[0]); }

    var start = new Date().getTime();
    rec.timer = window.setInterval(function () {
        var elapsed = new Date().getTime() - start;

        /* phase 1 - land and stay put, so the player registers
           where their hand actually was */
        if (elapsed < FX.hold) { return; }

        /* phase 2 - slow drift upward while fading */
        var t = (elapsed - FX.hold) / FX.duration;
        if (t >= 1) {
            retireMark(rec);
            return;
        }
        /* ease-in-out: creeps off, drifts, settles as it vanishes */
        var ease = t * t * (3 - 2 * t);
        el.style.top = (top0 - FX.rise * ease) + "px";
        setOpacity(el, 1 - t);
    }, FX.frame);
}

/* --- audio stub ----------------------------------------------
   No audio file exists yet. When one is dropped beside the .hta,
   set SOUND.enabled to true and this will fire. A swapped bgsound
```

```
   element is used rather than <audio> because it works in every
   document mode an HTA can land in. */
var SOUND = {
    enabled : false,
    file    : "click.wav",
    node    : null
};

function playActionSound() {
    if (!SOUND.enabled) { return; }
    try {
        if (!SOUND.node) {
            SOUND.node = document.createElement("bgsound");
            document.body.appendChild(SOUND.node);
        }
        SOUND.node.src = SOUND.file;
    } catch (err) { /* never let audio break an action */ }
}


/* ================================================================
   SECTION E : IDLE SCREEN - CONSTRUCTION
   Built once, then left alone. Costs nothing to look at.
   ================================================================ */

function buildIdleScreen() {
    var host = $("content");
    clear(host);
    stopNavFX();        /* the map timer must not outlive the map */
    PROGRESS_UI = {};   /* drop stale DOM refs from the last screen */

    var screen = make("div", null, "");
    screen.id = "screen-idle";

    screen.appendChild(buildStatRail());
    screen.appendChild(buildStage());
    screen.appendChild(buildStageNote());
    screen.appendChild(buildNavStrip());
    screen.appendChild(buildInfoRail());
    screen.appendChild(buildFeed());

    host.appendChild(screen);

    bindIdleInteractions();
    renderFeed();
    placeDots();
    layoutIdle();      /* give the feed the leftover vertical space */
}

/* --- upper rail : the four gauge cards ---------------------- */
function buildStatRail() {
    var rail = make("div", "rail"), i, s, cell;
```

```
    for (i = 0; i < STATS.length; i++) {
      s = STATS[i];
      cell = make("div", "rail-cell" +
        (i === STATS.length - 1 ? " rail-last" : ""));
      /* equal share of the rail, floored to 1/100 of a percent so
        Trident's rounding can never push the last cell down */
      cell.style.width = (Math.floor(10000 / STATS.length) / 100) + "%";
      cell.innerHTML =
        '<div class="stat" id="stat-' + s.id + '">' +
          '<div class="stat-label">' + s.label + '</div>' +
          '<div class="stat-value num">' + s.value +
            '<span class="stat-unit">' + s.unit + '</span></div>' +
          bar(s.value, s.colour) +
        '</div>';
      rail.appendChild(cell);
    }
    rail.appendChild(make("div", "clear"));
    return rail;
}

/* --- lower rail : location / contacts / life support ------- */
function buildInfoRail() {
    var rail = make("div", "rail"), i, n, cell;
    rail.style.marginTop = "12px";
    for (i = 0; i < INFOS.length; i++) {
      n = INFOS[i];
      cell = make("div", "rail-cell" +
        (i === INFOS.length - 1 ? " rail-last" : ""));
      cell.style.width = (Math.floor(10000 / INFOS.length) / 100) + "%";
      cell.innerHTML =
        '<div class="info" id="info-' + n.id + '">' +
          '<div class="info-label">' + n.label + '</div>' +
          '<div class="info-value">' + n.value + '</div>' +
        '</div>';
      rail.appendChild(cell);
    }
    rail.appendChild(make("div", "clear"));
    return rail;
}

/* --- the ship cross-section ------------------------------- */
function buildStage() {
    var stage = make("div", null, "");
    stage.id = "stage";

    /* --- the silhouette, straight from the HULL table ----- */
    var i, h, el;
    for (i = 0; i < HULL.length; i++) {
      h  = HULL[i];
      el = make("div", "hpart " + h.cls);
      el.id = "hp-" + h.id;
      el.style.left   = h.x + "px";
```

```
    el.style.top    = h.y + "px";
    el.style.width  = h.w + "px";
    el.style.height = h.h + "px";
    stage.appendChild(el);
    /* intake bars at the forward end of each nacelle */
    if (h.cls === "hp-nacelle") {
        var k, bar;
        for (k = 0; k < 3; k++) {
            bar = make("div", "nac-bar");
            bar.style.left   = (h.x + h.w - 30 - k * 12) + "px";
            bar.style.top    = (h.y + 8) + "px";
            bar.style.width  = "3px";
            bar.style.height = (h.h - 16) + "px";
            stage.appendChild(bar);
        }
    }
}

/* --- three engine nozzles, aft of the main body ------- */
var nz = findById(HULL, "body"), n;
for (i = 0; i < 3; i++) {
    n = make("div", "nozzle");
    n.style.left = (nz.x - 15) + "px";
    n.style.top  = (nz.y + 40 + i * 73) + "px";
    stage.appendChild(n);
}

/* --- the spine corridor ------------------------------ */
var cor = make("div", "corridor");
cor.style.left   = CORRIDOR.x + "px";
cor.style.top    = CORRIDOR.y + "px";
/* border-box: the 1px border is inside the number now, so the
   old -2 compensation would shrink the box by 2px */
cor.style.width  = CORRIDOR.w + "px";
cor.style.height = CORRIDOR.h + "px";
stage.appendChild(cor);


/* .room-tag anchors right/bottom, so those must be released
   before left/top will behave. Setting all four would stretch
   the element across the hull in Trident. */
var corLabel = make("div", "room-tag", "CORRIDOR");
corLabel.style.position = "absolute";
corLabel.style.right    = "auto";
corLabel.style.bottom   = "auto";
corLabel.style.left     = (CORRIDOR.x + 9) + "px";
corLabel.style.top      = (CORRIDOR.y + 7) + "px";
stage.appendChild(corLabel);

/* --- power conduit along the bottom of the main body -- */
var cd = make("div", "conduit");
cd.style.left   = CORRIDOR.x + "px";
```

```
    cd.style.top    = "340px";
    cd.style.width  = CORRIDOR.w + "px";
    cd.style.height = "12px";
    stage.appendChild(cd);

    /* --- rooms, drawn from data --------------------------- */
    var r;
    for (i = 0; i < ROOMS.length; i++) {
      r  = ROOMS[i];
      el = make("div", "room" + (r.small ? " room-small" : ""));
      el.id = "room-" + r.id;
      el.style.left       = r.x + "px";
      el.style.top        = r.y + "px";
      el.style.width      = r.w + "px";
      el.style.height     = r.h + "px";
      el.style.background = r.tone;
      el.innerHTML =
        '<div class="room-name">' + r.name + '</div>' +
        (r.systems ? '<div class="room-sys">' + r.systems + '</div>' : "") +
        '<div class="room-tag">' + r.tag + '</div>';
      stage.appendChild(el);
    }

    /* cosmetic bay dividers inside the cargo hold */
    var cargo = findById(ROOMS, "cargo"), d, k2;
    for (k2 = 1; k2 <= 2; k2++) {
      d = make("div", "divider");
      d.style.left   = (cargo.x + (cargo.w / 3) * k2) + "px";
      d.style.top    = (cargo.y + 26) + "px";
      d.style.height = (cargo.h - 40) + "px";
      stage.appendChild(d);
    }

    /* crew dots */
    var j2, c, dot, lab;
    for (j2 = 0; j2 < CREW.length; j2++) {
      c = CREW[j2];
      dot = make("div", "dot");
      dot.id = "dot-" + c.id;
      dot.style.background = c.colour;
      stage.appendChild(dot);

      lab = make("div", "dot-label", c.name.split(" ")[0]);
      lab.id = "lab-" + c.id;
      stage.appendChild(lab);
    }

    return stage;
  }

  function buildStageNote() {
    return make("div", "stage-note",
```

```
        "Left-click anything here to read what it is. " +
        "Right-click opens its task window.");
}

/* Navigation strip - sits under the ship, opens the map. */
function buildNavStrip() {
    var here = findById(BODIES, MAP.here);
    var t    = findOngoing("travel");
    var main;

    if (t && t.state === "running")   { main = t.label + " - crossing in progress"; }
    else if (t && t.state === "done") { main = "Crossing complete - awaiting acknowledgement"; }
    else { main = "Holding at " + here.name + " - no course plotted"; }

    var strip = make("div", null,
        '<div class="ns-right">Left-click to read - right-click opens it</div>' +
        '<div class="ns-label">Navigation</div>' +
        '<div class="ns-main">' + main + '</div>');
    strip.id = "navstrip";
    return strip;
}

function buildFeed() {
    var wrap = make("div", null, "");
    wrap.id = "feedwrap";
    wrap.appendChild(make("div", "feed-head", "Activity Feed"));
    var box = make("div", null, "");
    box.id = "feed";
    wrap.appendChild(box);
    return wrap;
}


/* ================================================================
   SECTION F : AMBIENT DOT DRIFT
   PURELY COSMETIC. Manifesto 1.5 is explicit that the dots drift
   so the diagram feels alive, and equally explicit that this does
   not advance game state. This timer therefore touches nothing
   except pixel positions. It must never be given game logic.

   In this build a dot drifts only inside its own room, so the
   diagram never contradicts the crew data shown in a task window.
   Room-to-room walks arrive with the crew logic phase.
   ================================================================ */

var DOTS = [];

function placeDots() {
    DOTS = [];
    var i, c, r, d;
    for (i = 0; i < CREW.length; i++) {
        c = CREW[i];
```

```
        r = crewBox(c);
        d = {
           id : c.id,
           el : $("dot-" + c.id),
           lb : $("lab-" + c.id),
           x  : dotSpot(r, "x"),
           y  : dotSpot(r, "y"),
           tx : 0, ty : 0, wait : 0, room : r
        };
        pickTarget(d);
        DOTS[DOTS.length] = d;
        drawDot(d);
     }
}

function pickTarget(d) {
    /* re-read the box every time: a hand laid off mid-game walks
       out of his room on the next drift, with no rebuild */
    var c = findById(CREW, d.id);
    if (c) { d.room = crewBox(c); }
    d.tx = dotSpot(d.room, "x");
    d.ty = dotSpot(d.room, "y");
}

/* A drift target inside a room, safe for any room size. The old
   maths assumed a 112px room and would have thrown a dot clean
   outside the 44px shells on the spine. */
/* THE CORRIDOR IS THE DOLE QUEUE.
   A hand with no job is not at a station, so he is not drawn at
   one. Everybody unemployed drifts up and down the corridor,
   which means the player can read the whole payroll off the
   cross-section without opening anything. */
function crewBox(c) {
    if (!c.job) { return CORRIDOR; }
    var r = findById(ROOMS, c.room);
    return r ? r : CORRIDOR;
}
function crewWhere(c) {
    if (!c.job) { return "Corridor - no job"; }
    var r = findById(ROOMS, c.room);
    return r ? r.name : "Corridor";
}

function dotSpot(r, axis) {
    var lo, span;
    if (axis === "x") {
        lo   = r.x + 22;
        span = r.w - 54;
    } else if (r.h > 70) {
        lo   = r.y + 44;
        span = r.h - 66;
    } else if (r.h > 34) {
```

```
        lo   = r.y + 16;
        span = r.h - 34;
    } else {
        /* the corridor is 26px of headroom - centre the dot in it
           rather than hanging it out of the bottom */
        lo   = r.y + Math.round((r.h - 15) / 2);
        span = 1;
    }
    if (span < 1) { span = 1; }
    return lo + Math.random() * span;
}

function drawDot(d) {
    if (!d.el) { return; }
    d.el.style.left = Math.round(d.x) + "px";
    d.el.style.top  = Math.round(d.y) + "px";
    d.lb.style.left = Math.round(d.x - 8) + "px";
    d.lb.style.top  = Math.round(d.y + 14) + "px";
}

function driftTick() {
    var i, d, dx, dy, dist, step;
    for (i = 0; i < DOTS.length; i++) {
        d = DOTS[i];
        if (d.wait > 0) { d.wait--; continue; }
        dx = d.tx - d.x;
        dy = d.ty - d.y;
        dist = Math.sqrt(dx * dx + dy * dy);
        if (dist < 1.2) {
            d.wait = 20 + Math.floor(Math.random() * 90);
            pickTarget(d);
            continue;
        }
        step = 0.5;
        d.x += (dx / dist) * step;
        d.y += (dy / dist) * step;
        drawDot(d);
    }
}


/* ================================================================
   SECTION G : IDLE-SCREEN MOUSE BINDING
   Right-click is the game's primary verb: it opens the task
   window for a thing - in the background, without moving the
   player. Left-click is the second verb, added later: it opens a
   briefing popup describing the thing and listing the actions its
   window offers. Both are free. Looking must stay harmless.
   ================================================================ */

function bindIdleInteractions() {
    var i, r, s, n, c, el;
```

```
    for (i = 0; i < ROOMS.length; i++) {
        r = ROOMS[i];
        el = $("room-" + r.id);
        if (el) { bindTaskOpen(el, "room", r.id); }
    }
    for (i = 0; i < STATS.length; i++) {
        s = STATS[i];
        el = $("stat-" + s.id);
        if (el) { bindTaskOpen(el, "system", s.id); }
    }
    for (i = 0; i < INFOS.length; i++) {
        n = INFOS[i];
        el = $("info-" + n.id);
        if (el) { bindTaskOpen(el, "info", n.id); }
    }
    for (i = 0; i < CREW.length; i++) {
        c = CREW[i];
        el = $("dot-" + c.id);
        if (el) { bindTaskOpen(el, "crew", c.id); }
        el = $("lab-" + c.id);
        if (el) { bindTaskOpen(el, "crew", c.id); }
    }

    el = $("navstrip");
    if (el) { bindTaskOpen(el, "nav", "main"); }
}

/* THE TWO BUTTONS, ON THE IDLE SCREEN ONLY.

   LEFT  - tells you what the thing is and what could be done to
           it. A briefing. Costs nothing, changes nothing, and
           the idle screen stays in front of you.
   RIGHT - opens the task window for it, and LEAVES YOU HERE.
           The tab appears in the bar and waits. Opening a window
           is not the same as walking into the room, so the game
           no longer drags the player away from what they were
           reading.

   Closure kept in its own function so the loop variable is not
   captured by reference - a classic IE-era trap. */
function bindTaskOpen(el, kind, id) {
    on(el, "click", function (e) {
        var ev = e || window.event;
        openBrief(kind, id);
        /* one click, one briefing - never the parent's as well */
        if (ev) {
            ev.cancelBubble = true;
            if (ev.stopPropagation) { ev.stopPropagation(); }
        }
        return false;
    });
```

```
    rightClick(el, function () { openTask(kind, id, false); });
}


/* ================================================================
   SECTION H : TAB SYSTEM
   Browser-style tabs. Home is permanent and always returns the
   player to the idle screen.
   ================================================================ */

/* hold at full colour, then ease back down. Same smoothstep the
   protagonist mouse uses, so the two feel like one hand. */
var TABFX = { warm:[233, 214, 184], hold:160, fade:1500, frame:50 };

function glowTab(el, isActive) {
    var base = isActive ? [255, 255, 255] : [239, 232, 220];
    var w    = TABFX.warm;
    var t0   = (new Date()).getTime();
    var tid;

    el.style.backgroundColor = "rgb(" + w[0] + "," + w[1] + "," + w[2] + ")";


    tid = setInterval(function () {
        /* the tab bar is rebuilt often; when this element is no
           longer on screen the timer has nothing left to do */
        if (!el.parentNode) { clearInterval(tid); return; }

        var t = ((new Date()).getTime() - t0 - TABFX.hold) / TABFX.fade;
        if (t < 0) { return; }
        if (t >= 1) {
            el.style.backgroundColor = "";   /* hand it back to the CSS */
            clearInterval(tid);
            return;
        }
        var k = t * t * (3 - 2 * t), c = [0, 0, 0], q;
        for (q = 0; q < 3; q++) {
            c[q] = Math.round(w[q] + (base[q] - w[q]) * k);
        }
        el.style.backgroundColor = "rgb(" + c[0] + "," + c[1] + "," + c[2] + ")";
    }, TABFX.frame);
}

function renderTabs() {
    var bar = $("taskbar");
    clear(bar);

    bar.appendChild(makeTab("home", SHIP.name + " (Idle)", "SHIP", true));

    var i, t;
    for (i = 0; i < STATE.tabs.length; i++) {
        t = STATE.tabs[i];
```

```
        bar.appendChild(makeTab(t.key, t.title, t.kindLabel, false));
    }
}

function makeTab(key, title, kindLabel, isHome) {
    var cls = "tab" + (isHome ? " tab-home" : "");
    if (STATE.activeTab === key) { cls += " tab-active"; }

    var tab = make("div", cls, "");
    /* A tab that has just been created announces itself: it comes
       up in coffee cream and cools back to its normal colour over
       about a second and a half. Right-clicking something on the
       idle screen no longer opens a window silently somewhere the
       player is not looking. Purely cosmetic - it touches nothing
       but a background colour and cannot advance the game. */
    if (STATE.freshTabs[key]) {
        STATE.freshTabs[key] = false;
        glowTab(tab, (STATE.activeTab === key));
    }
    tab.innerHTML = '<span class="tab-kind">' + kindLabel + '</span>' + title;

    on(tab, "click", function () { switchTab(key); });

    if (!isHome) {
        var x = make("span", "tab-close", "x");
        on(x, "click", function (e) {
            closeTab(key);
            if (e && e.stopPropagation) { e.stopPropagation(); }
            if (window.event) { window.event.cancelBubble = true; }
        });
        tab.appendChild(x);
    }
    return tab;
}

function switchTab(key) {
    STATE.activeTab = key;
    var t = (key === "home") ? null : refreshTab(findTab(key));
    renderTabs();
    if (key === "home") { buildIdleScreen(); }
    else { buildTaskScreen(t); }
    setStatus();
}

/* A tab stored the description it was born with, and switching
   back rebuilt the screen from that FROZEN copy - so a room
   window could report a condition, a stock or a crew state that
   had since moved on. Re-ask describeTarget every time the screen
   is built. Nav describes itself live and is left alone. */
function refreshTab(t) {
    if (!t || t.kind === "nav") { return t; }
    var fresh = describeTarget(t.kind, t.id), i;
```

```
    fresh.key = t.key;
    for (i = 0; i < STATE.tabs.length; i++) {
        if (STATE.tabs[i].key === t.key) { STATE.tabs[i] = fresh; }
    }
    return fresh;
}

function findTab(key) {
    var i;
    for (i = 0; i < STATE.tabs.length; i++) {
        if (STATE.tabs[i].key === key) { return STATE.tabs[i]; }
    }
    return null;
}

function closeTab(key) {
    var i, keep = [];
    for (i = 0; i < STATE.tabs.length; i++) {
        if (STATE.tabs[i].key !== key) { keep[keep.length] = STATE.tabs[i]; }
    }
    STATE.tabs = keep;
    if (STATE.activeTab === key) { switchTab("home"); }
    else { renderTabs(); setStatus(); }
}

/* Opening a task is free. Only interactions INSIDE a task window
   will ever cost actions, and none exist yet in this build. */
/* focus is opt-IN. Called without it the window opens in the
   background: the tab is created and drawn, and whatever the
   player is looking at stays on screen. Tab clicks pass true,
   because clicking a tab IS the request to go there. */
function openTask(kind, id, focus) {
    var key = kind + ":" + id;

    if (!findTab(key)) {
        var t = describeTarget(kind, id);
        t.key = key;
        STATE.tabs[STATE.tabs.length] = t;
        STATE.freshTabs[key] = true;    /* makeTab will light it up */
    }

    if (focus) { switchTab(key); } else { renderTabs(); }
}


/* ================================================================
   SECTION I : TASK WINDOW CONTENT
   Every task window in this build is a shell. It shows real data
   about the target and lists the interactions that will live there,
   greyed out with their intended action costs. Nothing is clickable
   yet because the action economy does not exist yet.
   ================================================================ */
```

```
/* What record, if any, a tab is watching. Nav carries none and
   says so by returning true. */
function targetEntity(kind, id) {
    if (kind === "room")   { return findById(ROOMS,  id); }
    if (kind === "crew")   { return findById(CREW,   id); }
    if (kind === "system") { return findById(STATS,  id); }
    if (kind === "info")   { return findById(INFOS,  id); }
    if (kind === "event")  { return findById(EVENTS, id); }
    return true;
}

function describeTarget(kind, id) {
    var t = { kind:kind, id:id, kindLabel:"", title:"", sub:"", rows:[], acts:[] };
    var r, c, s, n;

    /* A TAB CAN OUTLIVE WHAT IT WATCHES. refreshTab re-describes
       on every switch, so the moment anything leaves its table -
       the Test Room being deleted, an event being consumed, a
       crew member removed by some future mechanic - every branch
       below would read .name off null and take the screen down
       with it. One guard, all five lookups, no drift. */
    if (!targetEntity(kind, id)) {
        t.kindLabel = "GONE";
        t.title     = "No longer aboard";
        t.sub       = "Whatever this tab was watching is not on the " +
                      "ship any more.";
        t.rows      = [["Was", kind + " " + id],
                       ["Status", "no record"]];
        t.acts      = [act("Close this tab", "free")];
        return t;
    }

    if (kind === "event") {
        var ev = findById(EVENTS, id);
        t.kindLabel = "EVENT";
        t.title = ev ? ev.kind + " report" : "Report";
        t.sub   = "Noticed by the ship. Nobody has looked into it yet.";
        t.rows  = [
            ["Kind",    ev ? ev.kind   : "-"],
            ["Reported by", ev ? ev.source : "-"],
            ["Status",  "Unexamined"],
            ["Detail",  "Not yet established"]
        ];
        t.acts = [
            act("Examine the report", "1 action"),
            act("Send a crew member", "varies"),
            act("Log and ignore", "1 action")
        ];
        return t;
    }
```

```
if (kind === "nav") {
   t.kindLabel = "NAV";
   t.title = "Navigation";
   t.sub   = "Course plotting and system chart";
   return t;   /* nav builds its own screen - see buildNavScreen */
}

if (kind === "room" && id === "test") {
   t.kindLabel = "TEMP";
   t.title = "Test Room";
   t.sub   = "Diagnostic harness - not part of the ship";
   t.rows  = [
      ["Purpose", "Advance the action counter on demand"],
      ["Status",  "Temporary fixture"],
      ["Note",    "Use this to fill a running progress bar"]
   ];
   t.acts  = [ live("Do Work", "1 Action", "Test Room work") ];
   return t;
}

if (kind === "room") {
   r = findById(ROOMS, id);
   t.kindLabel = "ROOM";
   t.title = r.name;
   t.sub   = r.systems;
   t.rows  = [
      ["Designation",   r.tag],
      ["Systems",       r.systems],
      ["Condition",     "Operational"],
      ["Occupants",     occupantsOf(r.id)],
      ["Deck position", "x " + r.x + " / y " + r.y]
   ];
   t.acts = [
      act("Inspect room", "1 action"),
      act("Clean and tidy", "2 actions"),
      act("Assign crew member here", "1 action"),
      act("Begin repair work", "varies")
   ];
   /* the three rooms that hold a store show what is in it,
      above the generic room readings */
   if (r.id === "engine")  { roomEngine(t); }
   if (r.id === "o2")      { roomO2(t); }
   if (r.id === "cargo")   { roomCargo(t); }
   if (r.id === "science") { roomScience(t); }

   /* the engine room writes its own action list above */
   if (r.id !== "engine") {
      t.acts.unshift(act("Run diagnostics", "3 actions"));
   }
} else if (kind === "crew") {
   c = findById(CREW, id);
   r = findById(ROOMS, c.room);
```

```
    t.kindLabel = "CREW";
    t.title = c.name;
    t.sub   = c.role;
    t.rows  = [
      ["Role",     c.role],
      ["Location", r.name],
      ["Activity", c.activity],
      ["Traits",   c.traits],
      ["Health",   c.health + "%"],
      ["Morale",   c.morale + "%"],
      ["Fatigue",  c.fatigue + "%"],
      ["Hunger",   c.hunger + "%"]
    ];
    t.acts = [
      act("Talk", "1 action"),
      act("Assign to a room", "1 action"),
      act("Review service record", "1 action"),
      act("Address a complaint", "2 actions"),
      act("Promote", "3 actions"),
      act("Dismiss from service", "5 actions")
    ];
  } else if (kind === "system") {
    s = findById(STATS, id);
    t.kindLabel = "SYSTEM";
    t.title = s.label;
    t.sub   = "Reading " + s.value + s.unit;
    t.rows  = [
      ["Current reading", s.value + s.unit],
      ["Status",          s.value > 80 ? "Operational" : (s.value > 40 ? "Degraded" : "Damaged")],
      ["Trend",           "Stable"],
      ["Maintained by",   "Jax Moreno"],
      ["Last serviced",   "Not recorded"]
    ];
    t.acts = [

   act("Run diagnostic sweep", "3 actions"),
      act("Begin repair", "20 actions"),
      act("Reroute power", "2 actions"),
      act("Schedule maintenance", "1 action")
    ];
  } else {
    n = findById(INFOS, id);
    t.kindLabel = "READOUT";
    t.title = n.label;
    t.sub   = n.value;
    t.rows  = [
      ["Reading", n.value],
      ["Source",  "Ship sensors"],
      ["Updated", "On arrival"]
    ];
    t.acts = [
      act("Scan surroundings", "4 actions"),
```

```
            act("Plot a course", "2 actions"),
            act("Hail nearby vessels", "1 action")
        ];
    }
    return t;
}

/* ----------------------------------------------------------------
   THE THREE STORE ROOMS
   Each one REPLACES the generic room rows and actions with what
   the store actually holds. Everything is read from the tables in
   the data section, so the window, the briefing popup and the
   sidebar line can never tell three different stories.
   ---------------------------------------------------------------- */

/* ENGINE ROOM - the drive wears with every action spent aboard,
   so the briefing leads with the number that matters. */
function roomEngine(t) {
    var eng = assignedEngineer(), cost = maintCost();
    t.sub  = "Propulsion and power - condition " +
             STATE.engineCondition + "%";
    t.rows = [
        ["Condition",  STATE.engineCondition + "%"],
        ["Wear rate",  "1% per " + ENGINE.wearEvery + " actions spent aboard"],
        ["Maintenance", (STATE.engineCondition >= 100
            ? "Not needed - drive is perfect"
            : cost + " action" + (cost === 1 ? "" : "s") + " - restores to 100%")],
        ["Warning at", ENGINE.warnBelow + "% - a red line joins the sidebar"],
        ["Engineer",   eng ? eng.name + " - acts at " +
                             effectiveJobAt(eng) + "%"
                           : "Nobody holds the job"]
    ];
    /* Dead rows on purpose. A briefing says what the window holds;
       the work itself is committed in the window. A live row here
       would only spend an action and start nothing. */
    t.acts = [
        act("Run Maintenance", (STATE.engineCondition >= 100
              ? "refused at 100%" : cost + " actions")),
        act("The engineer acts alone",
            eng ? "at " + effectiveJobAt(eng) + "%" : "nobody assigned")
    ];
}

/* O2 FILTER ROOM - canisters in the rack, filters on the wall.
   Loading one into the other is manual work, by design. */
function roomO2(t) {
    t.sub  = "Life support - air quality " + airWord();
    t.rows = [
        ["Air quality", airWord()],
        ["Filter life", bankRunning()
            ? STATE.filterLife + " of " + LIFE.lifeMax + " actions"
            : "bank is not running"],
```

```
      ["Slots",       slotsIn("empty") + " bare, " +
                      slotsIn("installed") + " seated, " +
                      slotsIn("active") + " running"],
      ["Canisters",   LIFE.canisters + " of " + LIFE.capacity + " racked"],
      ["Filter hand", (function () {
         var h = assignedFilterHand();
         return h ? h.name + " - acts at " + AIR_STAGES[h.jobAt].word
                  : "nobody assigned";
      }())],
      ["Reward",      "+10 pro-action when all three run"]
    ];
    t.acts = [
      act("Install a canister", "1 action + 2 to seat"),
      act("Turn on the filters", "1 action - needs all three")
    ];
}

/* CARGO HOLD - what is aboard, what space it eats, what it is
   worth. Volume is the constraint the player actually feels. */
function roomCargo(t) {
    var i, g, used = cargoUsed(), rows = [
      ["Hold",     used + " of " + CARGO.capacity + " m3 used"],
      ["Lots",     GOODS.length + " aboard"],
      ["Manifest", cargoWorth() + " shiplets at current prices"]
    ];
    for (i = 0; i < GOODS.length; i++) {
      g = GOODS[i];
      rows[rows.length] = [g.name,
        g.vol + " m3  -  " + g.value + " sh  -  " + g.kind +
        (g.risk ? " (UNREGISTERED)" : "")];
    }
    t.rows = rows;
    t.sub  = "Storage - " + used + " of " + CARGO.capacity + " m3 in use";
    t.acts = [
      act("Re-stow the hold",        "2 actions"),
      act("Inspect a lot",           "1 action"),
      act("Jettison a lot",          "1 action"),
      act("Sell a lot",              "at a station"),
      act("Conceal an unregistered lot", "3 actions")
    ];
}

/* SCIENCE ROOM - the survey record. Data arrives here from deep
   scans, salvage and finished contracts, and leaves as packages
   sold at a station. */
function roomScience(t) {
    var i, v, rows = [
      ["Logs held",     SURVEY.length + " survey records"],
      ["Unsold value",  surveyWorth() + " shiplets"],
      ["Sources",       "Deep scans, salvage, completed contracts"]
    ];
    for (i = 0; i < SURVEY.length; i++) {
```

```
        v = SURVEY[i];
        rows[rows.length] = [v.name,
            v.src + "  -  " + v.value + " sh  -  " + v.state];
    }
    t.rows = rows;
    t.sub  = "Survey records - " + SURVEY.length + " logs on file";
    t.acts = [
        act("Review a survey log",        "1 action"),
        act("Prepare a data package",     "2 actions"),
        act("Cross-reference two logs",   "3 actions"),
        act("Sell a package",             "at a station")
    ];
}

/* Action descriptors. An action is data: a label, a cost and
   whether it is wired yet. Adding a real action later means
   flipping one of these from act() to live(). */
function act(label, costLabel) {
    return { label:label, cost:costLabel, live:false, detail:null };
}

function live(label, costLabel, detail) {
    return { label:label, cost:costLabel, live:true, detail:detail };
}

/* Separate function so the loop variable is not captured by
   reference - the same IE-era closure trap guarded against in
   the right-click binding. */
function wireAction(el, detail) {
    on(el, "click", function (e) { fireAction(e, detail); });
}

function occupantsOf(roomId) {
    var i, names = "";
    for (i = 0; i < CREW.length; i++) {
        if (CREW[i].room === roomId && CREW[i].job) {
            names += (names ? ", " : "") + CREW[i].name;
        }
    }
    return names ? names : "None";
}

function buildTaskScreen(t) {
    var host = $("content");
    clear(host);
    stopNavFX();        /* the map timer must not outlive the map */
    PROGRESS_UI = {};   /* drop stale DOM refs from the last screen */
    if (!t) { return; }

    if (t.kind === "nav") { buildNavScreen(host); return; }
    if (t.kind === "room" && t.id === "engine") {
        buildEngineScreen(host, t); return;
```

```
}
if (t.kind === "room" && t.id === "o2") {
   buildO2Screen(host, t); return;
}
if (t.kind === "room" && t.id === "quarters") {
   buildCrewScreen(host, t); return;
}

var scr = make("div", "task-screen");

var head = make("div", "task-head",
   '<div class="task-kind">' + t.kindLabel + '</div>' +
   '<div class="task-title">' + t.title + '</div>' +
   '<div class="task-sub">' + t.sub + '</div>');
scr.appendChild(head);

var left = make("div", "col-l");
var pnl  = make("div", "panel");
pnl.appendChild(make("div", "panel-head", "Status"));
var body = make("div", "panel-body"), html = "", i;
for (i = 0; i < t.rows.length; i++) {
   html += '<div class="kv"><div class="kv-k">' + t.rows[i][0] +
           '</div><div class="kv-v">' + t.rows[i][1] +
           '</div><div class="clear"></div></div>';
}
body.innerHTML = html;
pnl.appendChild(body);
left.appendChild(pnl);

var right = make("div", "col-r");
var pnl2  = make("div", "panel");
pnl2.appendChild(make("div", "panel-head", "Available Actions"));
var body2 = make("div", "panel-body"), a, row, dead = 0;
for (i = 0; i < t.acts.length; i++) {
   a   = t.acts[i];
   row = make("div", "act" + (a.live ? " act-live" : ""),
             '<span class="act-cost">' + a.cost + '</span>' + a.label);
   if (a.live) { wireAction(row, a.detail); }
   else { dead++; }
   body2.appendChild(row);
}
if (dead > 0) {
   body2.appendChild(make("div", "notice",
      'Greyed interactions are not wired yet. Costs shown are the ' +
      'intended design values.'));
}
pnl2.appendChild(body2);
right.appendChild(pnl2);

scr.appendChild(left);
scr.appendChild(right);
scr.appendChild(make("div", "clear"));
```

```
    host.appendChild(scr);
}


/* ================================================================
   SECTION I-2 : NAVIGATION SCREEN
   Map dead centre and full width, information panels above and
   below it.

   THE NO-FARMING RULE, ENFORCED HERE:
   Selecting a body, zooming, reading panels and collecting a
   finished readout are all free. None of them can reach
   performAction. The player may study the whole chart all
   afternoon and the game will not move. Exactly two things on
   this screen commit - "Start Travel" and "Run Deep Scan" - and
   both do it for the same reason: they ask the ship to do work.
   Any button added to the action band must go through fireAction
   or be free. There is no third option.
   ================================================================ */

function buildNavScreen(host) {
    var scr = make("div", "task-screen");

    scr.appendChild(make("div", "task-head",
        '<div class="task-kind">NAV</div>' +
        '<div class="task-title">Navigation</div>' +
        '<div class="task-sub">Select a destination to read it. ' +
        'Reading costs nothing.</div>'));

    var wrap = make("div", "navwrap");

    /* ------------------------------------------------------
       THE FRAME. One row of three columns with the chart in the
       middle, then the commitments
 full width underneath.

          [ Target         ] [           ] [           ]
          [ Classification ] [   CHART   ] [ Scan Data ]
          [ Status         ] [           ] [           ]
                           [Details][Distance]
          [           Target Actions              ]

       Identity reads down the left, sensor output reads down the
       right, and the chart is never pushed off centre by either.
       ------------------------------------------------------ */
    var main = make("div", "navmain");

    /* --- left column : who the target is --- */
    var cl = make("div", "navcell navcell-l");
    cl.appendChild(navPanel("Target",         "nav-name",    ""));
    cl.appendChild(navPanel("Classification", "nav-class",   ""));
```

```
    cl.appendChild(navPanel("Status",         "nav-status", "np-last"));
    main.appendChild(cl);

    /* --- middle column : the chart, and the two panels that
           describe the leg of the journey it is showing --- */
    var cc  = make("div", "navcell navcell-c");
    var mid = make("div", "navcenter");
    mid.appendChild(buildMap());

    var under = make("div", "navrow");
    under.appendChild(navPanel("Target Details", "nav-details", "np-half"));
    under.appendChild(navPanel("Distance",       "nav-dist",    "np-half-last"));
    mid.appendChild(under);

    cc.appendChild(mid);
    main.appendChild(cc);

    /* --- right column : what the sensors have to say. Free to
           grow downward for as long as it likes - nothing to its
           left depends on its height. --- */
    var cr = make("div", "navcell navcell-r");
    cr.appendChild(navPanel("Scan Data", "nav-scan", "np-last"));
    main.appendChild(cr);

    wrap.appendChild(main);

    /* --- the commitments, full width at the very bottom --- */
    var r3 = make("div", "navrow");
    r3.appendChild(navPanel("Target Actions", "nav-actions", "np-full"));
    wrap.appendChild(r3);

    scr.appendChild(wrap);
    host.appendChild(scr);

    /* size the chart to the window BEFORE anything is plotted on
       it, so the first paint is already correct */
    layoutNav();
    paintLayerButtons();
    applyPan();
    paintNavPanels();
    if (MAP.layer === "system") { startNavFX(); }
}

function navPanel(title, bodyId, cls) {
    var p = make("div", "navpanel" + (cls ? " " + cls : ""));
    p.appendChild(make("div", "navpanel-head", title));
    var b = make("div", "navpanel-body", "");
    b.id = bodyId;
    p.appendChild(b);
    return p;
}
```

```
/* Layers keep redraws cheap and independent: each draw routine
   owns exactly one layer and clears only that. The raster is a
   CSS background on #map itself, so there is no grid layer. */
function buildMap() {
    var m = make("div", null, "");
    m.id = "map";

    /* ORDER MATTERS AND IS A BUG FIX, NOT A PREFERENCE.
       The fx layer holds drifting rocks and a satellite that
       orbits Tolme at 17px - i.e. directly across its dot. Above
       the bodies layer, those 3px cosmetics intermittently ate
       clicks aimed at a real target. Cosmetics go underneath. */
    var names = ["map-routes", "map-fx", "map-bodies"], i, L;
    for (i = 0; i < names.length; i++) {
        L = make("div", "maplayer", "");
        L.id = names[i];
        m.appendChild(L);
    }

    var z  = make("div", "mapzoom");
    var zi = make("div", "zbtn", "+");
    var zo = make("div", "zbtn", "-");
    on(zi, "click", function () { zoomMap(1); });
    on(zo, "click", function () { zoomMap(-1); });
    z.appendChild(zi);
    z.appendChild(zo);
    m.appendChild(z);

    var sc = make("div", "mapscale", "");
    sc.id = "map-scale";
    m.appendChild(sc);

    /* the layer stack, directly under the zoom pair */
    var lb = make("div", "maplayers"), k, key, el;
    for (k = 0; k < LAYER_ORDER.length; k++) {
        key = LAYER_ORDER[k];
        el  = make("div", "lbtn", LAYERS[key].label);
        el.id = "lbtn-" + key;
        bindLayer(el, key);
        lb.appendChild(el);
    }
    m.appendChild(lb);

    m.appendChild(make("div", "maphint", "Drag to pan. Selecting is free"));

    bindMapDrag(m);
    return m;
}

/* closure guard, same reason as every other binder in this file */
function bindLayer(el, key) {
    on(el, "click", function () { setLayer(key); });
```

```
}

/* ---------------- layers ---------------- */

function curLayer()  { return LAYERS[MAP.layer]; }
function curBodies() { return curLayer().bodies; }

/* ids are unique across every layer, so a lookup does not need to
   know which map the player is looking at */
function findBody(id) {
    var i, b;
    for (i = 0; i < LAYER_ORDER.length; i++) {
        b = findById(LAYERS[LAYER_ORDER[i]].bodies, id);
        if (b) { return b; }
    }
    return null;
}

/* Only the system layer is inside the sensor envelope. Everything
   else can be looked at and nothing else. */
function inRange(b) { return !(b && b.range === false); }

/* where the ship is, expressed at the scale currently on screen */
function homeBody() {
    var lay = curLayer(), i;
    if (lay.id === "system") { return findById(BODIES, MAP.here); }
    for (i = 0; i < lay.bodies.length; i++) {
        if (lay.bodies[i].home) { return lay.bodies[i]; }
    }
    return null;
}

/* FREE. Switching scale is looking, and looking never costs. Each
   layer keeps its own zoom, pan and selection so coming back finds
   the map where it was left. */
function setLayer(key) {
    var old = curLayer();
    if (!LAYERS[key] || MAP.layer === key) { return; }

    old.zoom = MAP.zoom;
    old.sel  = STATE.navTarget;

    MAP.layer      = key;
    MAP.zoom       = curLayer().zoom;
    STATE.navTarget = curLayer().sel;

    /* the drifting rocks and the station beacon belong to the
       system view; they must not keep running over a galaxy */
    stopNavFX();

    paintLayerButtons();
    applyRaster();
```

```
    applyPan();
    drawMapRoutes();
    drawMapBodies();
    paintNavPanels();

    if (MAP.layer === "system") { startNavFX(); }
}

function paintLayerButtons() {
    var i, key, el;
    for (i = 0; i < LAYER_ORDER.length; i++) {
        key = LAYER_ORDER[i];
        el  = $("lbtn-" + key);
        if (el) { el.className = "lbtn" + (MAP.layer === key ? " lbtn-on" : ""); }
    }
}

/* ---------------- panning ---------------- */

/* Pan is applied by moving the three layer anchors, NOT by
   redrawing. The anchors are 0x0 positioned boxes and every dot,
   label and route is absolutely positioned inside them, so one
   style write on each moves the entire chart. Redrawing several
   hundred elements on every mousemove would be unusable in
   Trident. The raster stays put on purpose: it is a backdrop
   suggesting depth, not a coordinate grid, and it is drawn
   no-repeat so panning it would expose bare corners. */
var DRAG = { on:false, sx:0, sy:0, ox:0, oy:0, moved:0, bound:false };

function bindMapDrag(m) {
    on(m, "mousedown", mapDown);
    if (DRAG.bound) { return; }
    /* move and up go on the document so a drag survives the
       pointer leaving the map, and are bound exactly once even
       though the map itself is rebuilt on every screen change */
    DRAG.bound = true;
    on(document, "mousemove", mapMove);
    on(document, "mouseup",   mapUp);
}

function mapDown(e) {
    var ev = e || window.event, lay = curLayer();
    if (!ev || ev.button === 2) { return true; }   /* right-click is not a drag */
    DRAG.on    = true;
    DRAG.moved = 0;
    DRAG.sx    = ev.clientX;
    DRAG.sy    = ev.clientY;
    DRAG.ox    = lay.px;
    DRAG.oy    = lay.py;
    if (ev.preventDefault) { ev.preventDefault(); }
    return false;
}
```

```
function mapMove(e) {
   if (!DRAG.on) { return true; }
   var ev = e || window.event, lay = curLayer();
   if (!ev) { return true; }
   var dx = ev.clientX - DRAG.sx, dy = ev.clientY - DRAG.sy;

   DRAG.moved = Math.abs(dx) + Math.abs(dy);
   lay.px = DRAG.ox + dx;
   lay.py = DRAG.oy + dy;
   clampPan(lay);
   applyPan();
   return false;
}

function mapUp() {
   DRAG.on = false;
   return true;
}

/* the chart may be pulled half a screen out of place, no further,
  or it can be lost off the edge and look broken */
function clampPan(lay) {
   var mx = Math.round(MAP.vw * 0.5), my = Math.round(MAP.vh * 0.5);
   if (lay.px >  mx) { lay.px =  mx; }
   if (lay.px < -mx) { lay.px = -mx; }
   if (lay.py >  my) { lay.py =  my; }
   if (lay.py < -my) { lay.py = -my; }
}

function applyPan() {
   var lay = curLayer(), ids = ["map-routes", "map-fx", "map-bodies"], i, el;
   for (i = 0; i < ids.length; i++) {
      el = $(ids[i]);
      if (el) {
         el.style.left = lay.px + "px";
         el.style.top  = lay.py + "px";
      }
   }
}

/* a drag that happens to end on a star must not also select it */
function dragged() { return DRAG.moved > 4; }

/* World coordinate -> screen coordinate. Two steps: zoom about the
  centre of the world, then stretch the world onto the live size of
  the map box. Every star, route dot, label, rock and ring goes
  through this pair, which is why the chart can be any size at all
  and nothing comes loose from anything else. */
function mapX(wx) { return (MAP.cx + (wx - MAP.cx) * MAP.zoom) * (MAP.vw / MAP.w); }
function mapY(wy) { return (MAP.cy + (wy - MAP.cy) * MAP.zoom) * (MAP.vh / MAP.h); }
```

```
/* ---------------- routes ---------------- */

/* Dot trails rather than lines: no CSS transforms, no canvas, no
   VML. Renders identically in every document mode. */
function drawMapRoutes() {
    var L = $("map-routes"), lay = curLayer(), here, i, b, lk, a2, b2;
    if (!L) { return; }
    clear(L);

    /* SYSTEM: routes are derived - they radiate from wherever the
       ship actually is, so they stay honest after it moves. */
    if (lay.id === "system") {
        here = findById(BODIES, MAP.here);
        for (i = 0; i < BODIES.length; i++) {
            b =
 BODIES[i];
            if (b.id === MAP.here) { continue; }
            routeDots(L, here, b, STATE.navTarget === b.id, false);
        }
        return;
    }

    /* WIDE LAYERS: lanes are authored, because a shipping lane is
       something people agreed on, not a straight line between two
       points. Drawn fainter than a plotted course - they are
       context, not a route this ship can fly. */
    for (i = 0; i < lay.links.length; i++) {
        lk = lay.links[i];
        a2 = findById(lay.bodies, lk[0]);
        b2 = findById(lay.bodies, lk[1]);
        if (a2 && b2) { routeDots(L, a2, b2, false, true); }
    }
    drawClusters(L, lay);
}

/* Star clusters around a region marker. Scattered from a fixed
   seed, never Math.random, or they would jump to new positions on
   every redraw, zoom and pan. */
function drawClusters(L, lay) {
    var i, j, b, rnd, a, r, d;
    for (i = 0; i < lay.bodies.length; i++) {
        b = lay.bodies[i];
        if (!b.cluster) { continue; }
        rnd = seeded(i + 7);
        for (j = 0; j < b.cluster; j++) {
            a = rnd() * 6.283;
            r = 16 + rnd() * 34;
            d = make("div", "cdot");
            d.style.left = Math.round(mapX(b.x + Math.cos(a) * r)) + "px";
            d.style.top  = Math.round(mapY(b.y + Math.sin(a) * r * 0.62)) + "px";
            L.appendChild(d);
```

```
      }
    }
}

/* tiny deterministic generator - same seed, same sky, forever */
function seeded(seed) {
    var s = seed * 9301 + 49297;
    return function () {
        s = (s * 9301 + 49297) % 233280;
        return s / 233280;
    };
}

function routeDots(layer, a, b, selected, faint) {
    var x1 = mapX(a.x), y1 = mapY(a.y), x2 = mapX(b.x), y2 = mapY(b.y);
    var dx = x2 - x1, dy = y2 - y1;
    var len = Math.sqrt(dx * dx + dy * dy);
    var gap = selected ? 8 : (faint ? 15 : 11);
    var n   = Math.floor(len / gap);
    if (n > 90) { n = 90; }

    var i, t, d;
    for (i = 1; i < n; i++) {
        t = i / n;
        d = make("div", selected ? "rdot rdot-sel"
                                 : (faint ? "rdot rdot-faint" : "rdot"));
        d.style.left = Math.round(x1 + dx * t) + "px";
        d.style.top  = Math.round(y1 + dy * t) + "px";
        layer.appendChild(d);
    }
}


/* ---------------- bodies + the travelling marker ---------------- */

/* Where the green ring sits: at the origin normally, and part way
   along the route in proportion to travel progress. It advances
   only when the progress bar does, which is to say only when the
   player spends actions. */
function travelMarker() {
    var here = findById(BODIES, MAP.here);
    var t    = findOngoing("travel");
    if (!t || !t.target) { return { x:here.x, y:here.y }; }
    var tb = findById(BODIES, t.target);
    if (!tb) { return { x:here.x, y:here.y }; }
    var p = ongoingPercent(t) / 100;
    return { x: here.x + (tb.x - here.x) * p,
             y: here.y + (tb.y - here.y) * p };
}

function drawMapBodies() {
    var L = $("map-bodies");
```

```
if (!L) { return; }
clear(L);

var lay = curLayer(), bodies = lay.bodies;
var j, b, size, dot, lab, mk, hb, ring;

/* THE SHIP'S POSITION RING IS DRAWN FIRST, NOT LAST.
   It is 26px of transparent, positioned element and it sits
   exactly on top of the body the ship is parked at. Painted
   last it swallowed every click aimed at the current
   location - the same failure recorded in Entry 015. Drawn
   first, the dot and its label sit above it and stay
   reachable, and the ring is bound as well so clicking the
   ship itself selects where the ship is. */
/* On the system layer the ring is the ship and slides along
   its course. On the wider layers there is no ship to draw at
   that scale, so it rings the system or region the ship is
   inside - "you are in here" rather than "you are here". */
hb = homeBody();
if (hb) {
    mk   = (lay.id === "system") ? travelMarker() : { x:hb.x, y:hb.y };
    ring = make("div", "maphere");
    ring.style.left   = (mapX(mk.x) - 13) + "px";
    ring.style.top    = (mapY(mk.y) - 13) + "px";
    ring.style.width  = "26px";
    ring.style.height = "26px";
    bindSelect(ring, hb.id);
    L.appendChild(ring);
}

for (j = 0; j < bodies.length; j++) {
    b = bodies[j];
    /* generous hit targets - these are the primary control
       of the whole screen */
    /* outer sizes now: border-box means the 2px white ring is
       inside the number, so these are 4 larger than before and
       the dot lands exactly on its coordinate */
    size = (b.type === "station") ? 18 : 16;
    if (b.type === "origin") { size = 14; }
    if (b.type === "system") { size = 18; }
    if (b.type === "region") { size = 22; }

    dot = make("div", "mapdot" +
        (b.type === "station" ? " mapdot-station" : "") +
        (b.type === "system"  ? " mapdot-sys"     : "") +
        (b.type === "region"  ? " mapdot-region"  : "") +
        (STATE.navTarget === b.id ? " mapdot-sel" : ""));
    dot.style.left   = (mapX(b.x) - size / 2) + "px";
    dot.style.top    = (mapY(b.y) - size / 2) + "px";
    dot.style.width  = size + "px";
    dot.style.height = size + "px";
    bindSelect(dot, b.id);
```

```
        L.appendChild(dot);

        lab = make("div", "maplabel", b.name);
        lab.style.top   = (mapY(b.y) - 6) + "px";
        lab.style.width = "92px";
        /* flip the label inward near the right edge or the map's
           overflow clips it */
        if (mapX(b.x) > MAP.vw * 0.78) {
            lab.style.left      = (mapX(b.x) - 100) + "px";
            lab.style.textAlign = "right";
        } else {
            lab.style.left      = (mapX(b.x) + 10) + "px";
            lab.style.textAlign = "left";
        }
        bindSelect(lab, b.id);
        L.appendChild(lab);
    }

    var sc = $("map-scale");
    if (sc) { sc.innerHTML = lay.name + " - zoom x" + MAP.zoom.toFixed(1); }
}

/* closure guard, same pattern as the other binders */
function bindSelect(el, bodyId) {
    on(el, "click", function () {
        /* the click that ends a pan is not a choice of target */
        if (dragged()) { return; }
        selectBody(bodyId);
    });
}

/* FREE. No performAction, no spawnActionMark, no tick. */
function selectBody(bodyId) {
    STATE.navTarget   = bodyId;
    curLayer().sel    = bodyId;
    drawMapRoutes();
    drawMapBodies();
    paintNavPanels();
}

/* FREE. */
function zoomMap(dir) {
    MAP.zoom += dir * MAP.zstep;
    if (MAP.zoom < MAP.zmin) { MAP.zoom = MAP.zmin; }
    if (MAP.zoom > MAP.zmax) { MAP.zoom = MAP.zmax; }
    MAP.zoom = Math.round(MAP.zoom * 10) / 10;

    /* keep the raster in step with the chart. Harmless no-op in
       any mode that ignores background-size. */
    applyRaster();

    drawMapRoutes();
```

```
    drawMapBodies();
}


/* ================================================================
   SECTION I-3 : MAP AMBIENT MOTION
   Cosmetic only, exactly like the crew dots. Drifting rock,
   an orbiting satellite and a blinking station beacon. This
   timer must never touch game state. It runs only while the
   Navigation screen is on show and is killed on any screen
   change, so it cannot leak.
   ================================================================ */

var NAVFX = { timer:null, rocks:[], sat:null, beacon:null, phase:0 };

function startNavFX() {
    stopNavFX();
    var L = $("map-fx");
    if (!L) { return; }

    var i, r, el;
    for (i = 0; i < 9; i++) {
        el = make("div", "rock");
        r = {
            el : el,
            x  : Math.random() * MAP.w,
            y  : 96 + Math.random() * 120,
            vx : 0.10 + Math.random() * 0.22,
            s  : 2 + Math.floor(Math.random() * 3)
        };
        el.style.width  = r.s + "px";
        el.style.height = r.s + "px";
        L.appendChild(el);
        NAVFX.rocks[NAVFX.rocks.length] = r;
    }

    var tolme = findById(BODIES, "tolme");
    el = make("div", "sat");
    L.appendChild(el);
    NAVFX.sat = { el:el, host:tolme, r:17, ang:0, speed:0.022 };

    el = make("div", "beacon");
    L.appendChild(el);
    NAVFX.beacon = { el:el, host:tolme };

    NAVFX.timer = window.setInterval(navFxTick, 60);
}

function stopNavFX() {
    if (NAVFX.timer) { window.clearInterval(NAVFX.timer); }
    NAVFX.timer  = null;
    NAVFX.rocks  = [];
```

```
    NAVFX.sat    = null;
    NAVFX.beacon = null;
}

function navFxTick() {
    if (!$("map-fx")) { stopNavFX(); return; }
    var i, r;

    for (i = 0; i < NAVFX.rocks.length; i++) {
        r = NAVFX.rocks[i];
        r.x += r.vx;
        if (mapX(r.x) > MAP.vw + 12) { r.x = MAP.cx - (MAP.cx + 20) / MAP.zoom; }
        r.el.style.left = Math.round(mapX(r.x)) + "px";
        r.el.style.top  = Math.round(mapY(r.y)) + "px";
    }

    if (NAVFX.sat) {
        NAVFX.sat.ang += NAVFX.sat.speed;
        var sx = NAVFX.sat.host.x + Math.cos(NAVFX.sat.ang) * NAVFX.sat.r;
        var sy = NAVFX.sat.host.y + Math.sin(NAVFX.sat.ang) * (NAVFX.sat.r * 0.55);
        NAVFX.sat.el.style.left = Math.round(mapX(sx)) + "px";
        NAVFX.sat.el.style.top  = Math.round(mapY(sy)) + "px";
    }

    NAVFX.phase++;
    if (NAVFX.beacon) {
        var b = NAVFX.beacon.host;
        NAVFX.beacon.el.style.left = Math.round(mapX(b.x) - 1) + "px";
        NAVFX.beacon.el.style.top  = Math.round(mapY(b.y) - 9) + "px";
        setOpacity(NAVFX.beacon.el, (NAVFX.phase % 22) < 11 ? 1 : 0.15);
    }
}


/* ================================================================
   SECTION I-4 : DERIVED DISTANCES AND PANEL PAINTING
   ================================================================ */

/* Distance is measured, never authored, so it stays truthful
   after the ship relocates. */
function navMetrics(b) {
    var here = findById(BODIES, MAP.here);

   var dx = b.x - here.x, dy = b.y - here.y;
    var px = Math.sqrt(dx * dx + dy * dy);

    var dist = Math.round(px / MAP.pxPerAction);
    if (b.id === MAP.here) { dist = 0; }
    else if (dist < 3) { dist = 3; }

    var ly   = dist * MAP.lyPerAction;
    var e13  = (ly * 9.46) / 10;        /* 1 ly = 9.46 x 10^12 km */
```

```
    var days = dist * 19;
    var mon  = Math.floor(days / 30);
    var rem  = days - mon * 30;

    return {
      dist    : dist,
      /* what the crossing ACTUALLY costs the player: the action
         that commits it, plus the distance drained afterwards.
         The panel used to print dist alone, so a 21 action trip
         quietly took 22 clicks. */
      total   : dist + 1,
      ly      : ly.toFixed(1) + " ly",
      km      : e13.toFixed(1) + " x 10^13 km",
      transit : mon > 0 ? "~" + plur(mon, "month") + ", " + plur(rem, "day")
                        : "~" + plur(days, "day")
    };
}

function plur(n, word) { return n + " " + word + (n === 1 ? "" : "s"); }

function paintNavPanels() {
    var b       = STATE.navTarget ? findBody(STATE.navTarget) : null;
    var here    = findById(BODIES, MAP.here);
    var running = findOngoing("travel");

    if (!b) {
      setPanel("nav-name",    '<span class="dim">No target selected</span>');
      setPanel("nav-class",   '<span class="dim">Select a body on the chart.</span>');
      setPanel("nav-status",  '<span class="dim">Idle</span>');
      setPanel("nav-details", '<span class="dim">No target.</span>');
      setPanel("nav-scan",    '<span class="dim">No scan data.</span>');
      setPanel("nav-dist",    '<span class="dim">No route plotted.</span>');
    } else if (!inRange(b)) {
      /* Looking is always free, even across a galaxy. What the
         player gets for free is the chart entry - a name and a
         classification. Anything the SHIP would have to measure
         is refused, because the ship cannot reach it. */
      setPanel("nav-name",
        '<div class="navname">' + b.name + '</div>' +
        '<div class="navsub">Long range contact - ' +
        curLayer().name + ' chart</div>');
      setPanel("nav-class",   b.klass);
      setPanel("nav-status",  b.home ? "Current position"
                                     : "Out of sensor range");
      setPanel("nav-details", OUT_OF_RANGE);
      setPanel("nav-scan",    OUT_OF_RANGE);
      setPanel("nav-dist",    OUT_OF_RANGE);

    } else {
      var m = navMetrics(b);

      setPanel("nav-name",
```

```
        '<div class="navname">' + b.name + '</div>' +
        '<div class="navsub">' + (b.id === MAP.here
            ? "Current position"
            : "From " + here.name) + '</div>');

    setPanel("nav-class", b.klass);

    setPanel("nav-status", b.id === MAP.here
        ? "Current position"
        : (running ? "Crossing under way" : "Reachable"));

    /* target details, origin included here per design */
    var i, html = "";
    for (i = 0; i < b.details.length; i++) {
        html += field(b.details[i][0], b.details[i][1]);
    }
    html += field("Origin", here.name);
    setPanel("nav-details", html);

    setPanel("nav-scan", scanReadout(b));

    setPanel("nav-dist", b.id === MAP.here
        ? '<span class="dim">You are already here.</span>'
        : field("Distance", m.ly) +
          field("", m.km) +
          field("Transit", m.transit) +
          field("Action cost", plur(m.total, "action")) +
          field("", "1 to commit, " + plur(m.dist, "action") +
                " under way"));
}

/* --- the action band owns the commit buttons, so it is
   rebuilt from scratch on every paint --- */
var actions = $("nav-actions");
if (!actions) { return; }
clear(actions);

/* the previous paint's buttons are gone from the document;
   drop their refs or updateProgressUI keeps painting orphans */
PROGRESS_UI = {};

var reach = !!(b && inRange(b));

actions.appendChild(make("div", "actionbar",
    reach && b.id !== MAP.here
        ? "Course from " + here.name + " to " + b.name + "."
        : (b && !reach
            ? '<span class="dim">No course can be plotted to a ' +
              'contact this far out.</span>'
            : '<span class="dim">Plot a course by selecting a ' +
              'destination.</span>')));
```

```
    var row  = make("div", "actionrow");
    var scan = findOngoing("scan");

    row.appendChild(actionCell(buildProgressButton(
        "travel", "Start Travel", startTravel,
        !!(reach && b.id !== MAP.here)
    )));

    /* A scan running on a DIFFERENT body must not look like it
       belongs to this one, so that case gets a dead task id of its
       own: nothing is found for it, and it paints as inert. One
       set of sensors, one scan at a time. */
    if (scan && scan.state === "running" && b && scan.target !== b.id) {
        row.appendChild(actionCell(buildProgressButton(
            "scan-busy", "Sensors busy", startScan, false, "Scan Complete"
        )));
    } else {
        row.appendChild(actionCell(buildProgressButton(
            "scan", "Run Deep Scan", startScan, reach, "Scan Complete"
        )));
    }

    actions.appendChild(row);

    actions.appendChild(make("div", "actionnote",
        running
            ? '<span class="dim">A crossing is already under way.</span>'
            : '<span class="dim">Committing costs one action now, then the ' +
              'distance in actions spent elsewhere aboard the ship. A deep ' +
              'scan costs one action now and ' + SCAN_COST + ' spent elsewhere.</span>'));
}

/* one slot in the action row - keeps the buttons a normal width
   instead of letting them span the panel */
function actionCell(btn) {
    var c = make("div", "actioncell");
    c.appendChild(btn);
    return c;
}

/* The scan panel is a reward, not a description. Chart data -
   name, class, distance - is free because looking is free. What
   the sensors found is not, and it stays blank until a deep scan
   has been run AND the player has gone back to read it. */
function scanReadout(b) {
    var st = findOngoing("scan"), i, html;

    /* the sensors have a horizon, and it is not negotiable */
    if (!inRange(b)) { return OUT_OF_RANGE; }

    if (!STATE.scanned[b.id]) {
        if (st && st.target === b.id) {
```

```
            return st.state === "done"
                ? '<span class="dim">Deep scan complete. Readout waiting.</span>'
                : '<span class="dim">Deep scan under way.</span>';
        }
        return '<span class="dim">No detailed data. Sensors idle.</span>';
    }

    /* tiered: what this scan found, then what it could not reach */
    html = "";
    for (i = 0; i < b.scan.length; i++) {
        html += '<div class="scanline">' + b.scan[i] + '</div>';
    }
    for (i = 0; i < b.scanLocked.length; i++) {
        html += '<div class="scanlock">' + b.scanLocked[i] +
                ' - sensors insufficient</div>';
    }
    return html;
}

function setPanel(id, html) {
    var el = $(id);
    if (el) { el.innerHTML = html; }
}

function field(k, v) {
    return '<div class="navfield"><div class="navfield-k">' + k +
           '</div><div class="navfield-v">' + v +
           '</div><div class="clear"></div></div>';
}

/* THE COMMITMENT. The only thing on this screen that costs. */
function startTravel(e) {
    var b = STATE.navTarget ? findBody(STATE.navTarget) : null;
    if (!b || !inRange(b) || b.id === MAP.here) { return; }
    if (findOngoing("travel")) { return; }

    var m = navMetrics(b);

    /* Order matters. The initiating action is spent FIRST, so the
       tick it causes cannot eat a step of the journey it starts.
       Manifesto 1.3: one click to initiate, then the distance. */
    fireAction(e, "Course locked - " + b.name);

    addOngoing("travel", "Under way to " + b.name, m.dist, "travel", b.id);

    paintProgress("travel");
    paintNavPanels();
    drawMapBodies();
}

/* THE SECOND COMMITMENT. Reading the chart is free, but asking
   the sensors for more is work, so it costs exactly like a
```

```
  crossing does: one action now, the rest spent elsewhere. */
function startScan(e) {
    var b = STATE.navTarget ? findBody(STATE.navTarget) : null;
    if (!b || !inRange(b)) { return; }
    if (findOngoing("scan")) { return; }

    /* same ordering rule as startTravel: spend the initiating
       action FIRST, so the tick it causes cannot eat a step of
       the work it is starting */
    fireAction(e, "Deep scan started - " + b.name);

    addOngoing("scan", "Deep scan - " + b.name, SCAN_COST, "scan", b.id);

    paintProgress("scan");
    paintNavPanels();
}

/* THE REVEAL. Called from the type table when the player clicks a
   finished scan. The data existed all along; the player earning
   the right to see it is the mechanic. */
function revealScan(bodyId) {
    var b = findById(BODIES, bodyId);
    if (!b) { return; }
    STATE.scanned[b.id] = true;
    fileSurvey(b);              /* the readout becomes a saleable record */
    /* look at what was just read, in case the player wandered off
       to another body while the sensors worked */
    STATE.navTarget = b.id;
    logLine("Deep scan readout filed: " + b.name +
            ". Survey record stored in the Science Room.");
}

/* Arrival. Called when the player acknowledges a finished
   crossing. The ship genuinely relocates, which is safe because
   every distance is measured from MAP.here rather than authored. */
function arriveAt(bodyId) {
    var b = findById(BODIES, bodyId);
    if (!b) { return; }
    MAP.here = b.id;
    findById(INFOS, "location").value = b.name;
    logLine("Arrived at " + b.name + ".");
    setStatus();
}


/* ================================================================

   SECTION I-3 : ENGINE ROOM
   The second task window built on the house frame: a fixed stage
   dead centre, reading panels around it, commitments full width
   underneath. The stage here is a machine rather than a chart.
```

```
   THE MACHINE is pure markup - absolutely positioned divs inside
   a fixed 560x300 box. No canvas, no SVG, no images: a single
   .hta has nowhere to keep an image and Trident cannot be
   trusted with the rest. Every part is drawn from a table below,
   so the illustration is data and not three hundred lines of
   hand-written HTML.

   ALL OF IT IS FLAVOUR EXCEPT ONE READING. The condition meter
   is the only thing on the machine connected to state; the
   gauges, valves, cylinders and statistics exist to make the
   drive feel like a machine that could go wrong.
   ============================================================ */

function buildEngineScreen(host, t) {
   var scr = make("div", "task-screen");

   scr.appendChild(make("div", "task-head",
      '<div class="task-kind">ROOM</div>' +
      '<div class="task-title">Engine Room</div>' +
      '<div class="task-sub">Propulsion and power - the drive wears ' +
      'with every action spent aboard.</div>'));

   var wrap = make("div", "navwrap");
   var main = make("div", "navmain");

   /* --- left : what the drive is doing ------------------- */
   var cl = make("div", "navcell navcell-l");
   cl.appendChild(navPanel("Engine Condition", "eng-cond",  ""));
   cl.appendChild(navPanel("Wear",             "eng-wear",  ""));
   cl.appendChild(navPanel("Drive",            "eng-drive", "np-last"));
   main.appendChild(cl);

   /* --- centre : the machine ----------------------------- */
   var cc  = make("div", "navcell navcell-c");
   var mid = make("div", "navcenter");
   mid.appendChild(buildMachine());

   var under = make("div", "navrow");
   under.appendChild(navPanel("Maintenance", "eng-maint", "np-half"));
   under.appendChild(navPanel("Readings",    "eng-read",  "np-half-last"));
   mid.appendChild(under);

   cc.appendChild(mid);
   main.appendChild(cc);

   /* --- right : who is in here --------------------------- */
   var cr = make("div", "navcell navcell-r");
   cr.appendChild(navPanel("Engineering Crew", "eng-crew", "np-last"));
   main.appendChild(cr);

   wrap.appendChild(main);
```

```
    var r3 = make("div", "navrow");
    r3.appendChild(navPanel("Engine Actions", "eng-actions", "np-full"));
    wrap.appendChild(r3);

    scr.appendChild(wrap);
    host.appendChild(scr);

    paintEngine();
}

/* ---------------------------------------------------------------
   THE MACHINE, AS DATA
   Each row is [class, left, top, width, height, html]. Height or
   width of 0 means the rule supplies it. Anything decorative gets
   pointer-events:none through .mach-part, because scenery painted
   over a control has swallowed a click three separate times in
   this project already.
   --------------------------------------------------------------- */
function buildMachine() {
    var m = make("div", null);
    m.id = "machine";
    m.style.width  = STAGE.engine.w + "px";
    m.style.height = STAGE.engine.h + "px";

    var k, p, i2, a, x, y, cx, cy, r;

    /* --- chassis corner brackets --------------------------- */
    var BRK = [[16, 16, 1, 1], [634, 16, -1, 1],
               [16, 374, 1, -1], [634, 374, -1, -1]];
    for (k = 0; k < BRK.length; k++) {
      x = BRK[k][0]; y = BRK[k][1];
      m.appendChild(mpart("mach-brk", BRK[k][2] > 0 ? x : x - 20, y, 20, 2, ""));
      m.appendChild(mpart("mach-brk", x, BRK[k][3] > 0 ? y : y - 20, 2, 20, ""));
    }

    /* --- main pipe run across the top, riveted, five valves - */
    m.appendChild(mpart("mach-pipe", 30, 26, 576, 10, ""));
    for (k = 0; k < 20; k++) {
      m.appendChild(mpart("mach-rivet", 40 + k * 28, 29, 4, 4, ""));
    }
    for (k = 0; k < 5; k++) {
      x = 56 + k * 114;
      m.appendChild(mpart("mach-valve", x, 18, 16, 26, ""));
      m.appendChild(mpart("mach-cap", x - 8, 4, 32, 10, "V" + (k + 1)));
    }

    /* --- the coolant circuit, a closed loop ---------------- */
    m.appendChild(mpart("mach-pipe", 602,  26,  10, 268, ""));  /* right fall   */
    m.appendChild(mpart("mach-pipe", 124, 284, 488,  10, ""));  /* cold return  */
    m.appendChild(mpart("mach-pipe", 124, 250,  10,  34, ""));  /* core elbow   */
    m.appendChild(mpart("mach-pipe", 420,  36,  10,  60, ""));  /* exch. feed   */
    m.appendChild(mpart("mach-pipe", 420, 192,  10,  92, ""));  /* exch. drain  */
```

```
for (k = 0; k < 5; k++) {
   m.appendChild(mpart("mach-flow", 176 + k * 84, 287, 5, 4, ""));
}

/* --- reactor core : rings, encoder ticks, bolts, mounts - */
cx = 208; cy = 182;
m.appendChild(mpart("mach-mount", 112, 174, 12, 16, ""));
m.appendChild(mpart("mach-mount", 292, 174, 12, 16, ""));
m.appendChild(mpart("mach-mount", 200,  86, 16, 12, ""));
m.appendChild(mpart("mach-ring",  124,  98, 164, 164, ""));
m.appendChild(mpart("mach-ring2", 148, 122, 118, 118, ""));
for (k = 0; k < 24; k++) {          /* inner encoder ring */
   a = k * Math.PI / 12;
   m.appendChild(mpart("mach-tick",
      Math.round(cx + 52 * Math.cos(a)) - 2,
      Math.round(cy + 52 * Math.sin(a)) - 2, 3, 3, ""));
}
for (k = 0; k < 12; k++) {          /* bolts on the band  */
   a = k * Math.PI / 6;
   m.appendChild(mpart("mach-bolt",
      Math.round(cx + 72 * Math.cos(a)) - 4,
      Math.round(cy + 72 * Math.sin(a)) - 4, 8, 8, ""));
}
m.appendChild(mpart("mach-hub",  190, 164, 34, 34, ""));
m.appendChild(mpart("mach-core", 201, 175, 14, 14, ""));
m.appendChild(mpart("mach-cap",  124, 268, 164, 12, "PRIMARY CORE"));

/* --- linkage, flywheel and governor ---------------------- */
m.appendChild(mpart("mach-rod", 292, 150, 22, 6, ""));
m.appendChild(mpart("mach-rod", 292, 230, 26, 6, ""));

m.appendChild(mpart("mach-wheel", 306, 112, 76, 76, ""));
m.appendChild(mpart("mach-ring2", 322, 128, 46, 46, ""));
m.appendChild(mpart("mach-spoke", 306, 151, 80, 2, ""));
m.appendChild(mpart("mach-spoke", 345, 112, 2, 80, ""));
for (k = 0; k < 8; k++) {           /* blades */
   a = k * Math.PI / 4 + Math.PI / 8;
   m.appendChild(mpart("mach-tick",
      Math.round(346 + 30 * Math.cos(a)) - 3,
      Math.round(152 + 30 * Math.sin(a)) - 3, 6, 6, ""));
}
m.appendChild(mpart("mach-hub2", 338, 144, 16, 16, ""));

m.appendChild(mpart("mach-rod",   344, 188, 4, 20, ""));   /* belt */
m.appendChild(mpart("mach-wheel", 318, 206, 52, 52, ""));
m.appendChild(mpart("mach-spoke", 318, 233, 56, 2, ""));
m.appendChild(mpart("mach-spoke", 345, 206, 2, 56, ""));
for (k = 0; k < 6; k++) {
   a = k * Math.PI / 3;
   m.appendChild(mpart("mach-tick",
      Math.round(346 + 20 * Math.cos(a)) - 2,
      Math.round(234 + 20 * Math.sin(a)) - 2, 5, 5, ""));
```

```
}
m.appendChild(mpart("mach-hub2", 340, 228, 12, 12, ""));
m.appendChild(mpart("mach-mini", 300, 266, 96, 10, "TURBINE STAGE"));

/* --- heat exchanger, finned ---------------------------- */
m.appendChild(mpart("mach-box",  396,  96, 62, 96, ""));
m.appendChild(mpart("mach-mini", 401, 100, 56, 10, "EXCHANGER"));
for (k = 0; k < 9; k++) {
   m.appendChild(mpart("mach-fin", 402 + k * 6, 114, 4, 66, ""));
}

/* --- two dial gauges down the left ---------------------- */
var G = [["PRS", 68, "68 bar - nominal"], ["TMP", 41, "41 pct - nominal"]];
for (k = 0; k < G.length; k++) {
   p  = 52 + k * 98;
   cx = 64; cy = p + 48;
   m.appendChild(mpart("mach-box",  16, p, 96, 92, ""));
   m.appendChild(mpart("mach-mini", 22, p + 5, 88, 10, G[k][0]));
   m.appendChild(mpart("mach-dial", cx - 26, cy - 26, 50, 50, ""));
   for (i2 = 0; i2 < 11; i2++) {    /* 270 degree tick sweep */
      a = (135 + i2 * 27) * Math.PI / 180;
      m.appendChild(mpart("mach-tick",
         Math.round(cx + 21 * Math.cos(a)) - 1,
         Math.round(cy + 21 * Math.sin(a)) - 1, 2, 2, ""));
   }
   a = (135 + G[k][1] * 2.7) * Math.PI / 180;
   for (i2 = 0; i2 < 3; i2++) {     /* the needle, as three dots */
      r = 7 + i2 * 5;
      m.appendChild(mpart("mach-nd",
         Math.round(cx + r * Math.cos(a)) - 2,
         Math.round(cy + r * Math.sin(a)) - 2, 4, 4, ""));
   }
   m.appendChild(mpart("mach-hub2", cx - 4, cy - 4, 8, 8, ""));
   m.appendChild(mpart("mach-mini2", 22, p + 74, 88, 10, G[k][2]));
}

/* --- the indicator bus : eight lamps -------------------- */
var LAMP = ["on", "on", "amb", "on", "on", "", "on", "on"];
m.appendChild(mpart("mach-box",  16, 248, 96, 48, ""));
m.appendChild(mpart("mach-mini", 22, 252, 88, 10, "BUS A / B"));
for (k = 0; k < LAMP.length; k++) {
   m.appendChild(mpart("mach-lamp" +
      (LAMP[k] ? " mach-lamp-" + LAMP[k] : ""),
      24 + (k % 4) * 20, 266 + (k > 3 ? 14 : 0), 8, 8, ""));
}

/* --- statistics panel ---------------------------------- */
var R = [["Output",    "4.21 TJ"],  ["Flux",      "0.98"],
        ["Coolant",   "STABLE"],   ["Flow",      "83 pct"],
        ["Bearing",   "0.07 mm"],  ["Cycles",    "18,402"],
        ["Manifold",  "2 of 2"],   ["Tolerance", "+/- 0.4%"],
        ["Last svc",  "412 hr"]];
```

```
    m.appendChild(mpart("mach-box",  470, 56, 130, 152, ""));
    m.appendChild(mpart("mach-mini", 476, 60, 118, 10, "DRIVE STATISTICS"));
    for (k = 0; k < R.length; k++) {
      m.appendChild(mpart("mach-stat", 476, 74 + k * 14, 118, 13,
        '<span class="mach-sk">' + R[k][0] + '</span>' +
        '<span class="mach-sv">' + R[k][1] + '</span>'));
    }

    /* --- cylinder
  bank ------------------------------------ */
    m.appendChild(mpart("mach-mini", 470, 214, 130, 10, "CYLINDER BANK 1-16"));
    for (k = 0; k < 16; k++) {
      m.appendChild(mpart("mach-cyl", 470 + k * 8, 226 + (k % 2 ? 10 : 0),
                  6, (k % 2 ? 34 : 44), ""));
    }

    /* --- THE ONE LIVE READING ------------------------------ */
    m.appendChild(mpart("mach-box",  16, 310, 618, 64, ""));
    m.appendChild(mpart("mach-mini", 24, 316, 160, 10, "ENGINE CONDITION"));
    m.appendChild(mpart("mach-track2", 24, 330, 420, 14, ""));
    var el = mpart("mach-fill2", 24, 330, 420, 14, "");
    el.id = "eng-meter";
    m.appendChild(el);
    for (k = 0; k < 11; k++) {
      m.appendChild(mpart("mach-scale", 24 + k * 42, 348, 1, 5, ""));
    }
    m.appendChild(mpart("mach-plate", 24, 358, 340, 10,
      "DRIVE UNIT K-4 . SER " + SHIP.registry + "-E . BANK 1-16"));
    el = mpart("mach-pct", 470, 314, 158, 32, "100%");
    el.id = "eng-pct";
    m.appendChild(el);
    el = mpart("mach-word", 470, 350, 158, 14, "NOMINAL");
    el.id = "eng-word";
    m.appendChild(el);

    return m;
}

/* one absolutely positioned part of the illustration */
function mpart(cls, x, y, w, h, html) {
    var el = make("div", "mach-part " + cls, html);
    el.style.left = x + "px";
    el.style.top  = y + "px";
    if (w) { el.style.width  = w + "px"; }
    if (h) { el.style.height = h + "px"; }
    return el;
}

/* ------------------------------------------------------------
   PAINTING THE ENGINE
   Called after every action and after every repair. Safe to call
   when the window is not open - every lookup is guarded.
```

```
  ---------------------------------------------------------------- */
function paintEngine() {
  if (!$("eng-actions")) { return; }

  var c    = STATE.engineCondition;
  var cost = maintCost();
  var eng  = assignedEngineer();
  var run  = maintRunning();
  var tone, word;

  if (c >= 75)      { tone = "#7f9161"; word = "NOMINAL"; }
  else if (c >= 50) { tone = "#c08a3e"; word = "SERVICEABLE"; }
  else if (c >= 25) { tone = "#c08a3e"; word = "DEGRADED"; }
  else              { tone = "#b05a42"; word = "CRITICAL"; }

  /* the meter on the machine */
  if ($("eng-meter")) {
    $("eng-meter").style.width      = Math.round(420 * (c / 100)) + "px";
    $("eng-meter").style.background = tone;
  }
  if ($("eng-pct"))  { $("eng-pct").innerHTML  = c + "%"; }
  if ($("eng-word")) {
    $("eng-word").innerHTML = word;
    $("eng-word").style.color = tone;
  }

  setPanel("eng-cond",
    '<div class="navname" style="color:' + tone + ';">' + c + '%</div>' +
    '<div class="navsub">' + word + '</div>' +
    field("Threshold", ENGINE.warnBelow + "% - warning line"));

  setPanel("eng-wear",
    field("Rate", "1% per " + ENGINE.wearEvery + " actions") +
    field("Next step", (ENGINE.wearEvery - STATE.wearTick) +
          " action" + (ENGINE.wearEvery - STATE.wearTick === 1 ? "" : "s")) +
    field("Full cycle", (100 * ENGINE.wearEvery) + " actions"));

  setPanel("eng-drive",
    field("Class", "Kestrel light hauler") +
    field("Core", "Primary - single") +
    field("Cylinders", "12 in bank 1"));

  setPanel("eng-maint",
    (c >= 100
      ? '<span class="dim">Engine is at full condition. ' +
        'Nothing to maintain.</span>'
      : field("Cost now", cost + " action" + (cost === 1 ? "" : "s")) +
        field("Formula", "1 action per 10% missing") +
        field("Restores", "to 100%")));

  setPanel("eng-read",
    field("Output", "4.21 TJ") +
```

```
      field("Coolant", "Stable") +
      field("Bearing wear", (Math.round((100 - c) * 0.7) / 10) + " mm"));

    setPanel("eng-crew",
      (eng
        ? '<div class="navname">' + eng.name + '</div>' +
          '<div class="navsub">' + eng.role + ' - maintenance job</div>' +
          field("Skill",    eng.skill) +
          field("Acts at",  effectiveJobAt(eng) + "% condition" +
              (effectiveJobAt(eng) !== eng.jobAt
                ? " (air: was " + eng.jobAt + ")" : "")) +
          field("Activity", eng.activity) +
          '<div class="navnote">He holds the maintenance job and will ' +
          'service the drive unasked - but not before it falls to ' +
          eng.jobAt + '%, which is long past the warning. Training at ' +
          'a station, or a better engineer, raises that.</div>'
        : '<span class="dim">Nobody holds the maintenance job. The ' +
          'commander runs it personally, or nobody does.</span>'));

    paintEngineActions(c, cost, eng, run);
}

/* the commitment band, rebuilt whenever the situation changes */
function paintEngineActions(c, cost, eng, run) {
    var band = $("eng-actions");
    if (!band) { return; }
    clear(band);

    /* the old buttons are gone - drop their DOM refs or the
       progress painter will write into detached elements */
    PROGRESS_UI["maint"]     = null;
    PROGRESS_UI["maint-eng"] = null;

    band.appendChild(make("div", "actionbar",
      c >= 100
        ? '<span class="dim">The drive is at full condition. ' +
          'Maintenance cannot be run on a perfect engine.</span>'
        : "Maintenance will cost " + cost + " action" +
          (cost === 1 ? "" : "s") + " spent anywhere aboard, and " +
          "restores the drive to 100%."));

    var row = make("div", "actionrow");

    /* ONE maintenance slot. If the engineer took the job on himself
       the same slot shows HIS progress - the player watches it fill
       rather than owning it.

       THIS ROW IS A LIST, NOT A PAIR. Every future engine job -
       overhaul a cylinder, purge the coolant loop, retune the
       flux, strip the turbine - appends one more actionCell here
       and one more TASK_TYPES entry. The machine already has the
       parts drawn for them. No layout change will be needed:
```

```
      .actionrow centres whatever it is given. */
    if (findOngoing("maint-eng")) {
        row.appendChild(actionCell(buildProgressButton(
            "maint-eng", "Engineer working", startMaintSelf, false)));
    } else {
        row.appendChild(actionCell(buildProgressButton(
            "maint", "Run Maintenance", startMaintSelf, c < 100)));
    }

    band.appendChild(row);
    band.appendChild(make("div", "clear"));
}

function startMaintSelf(e) { startMaint(e, "maint", "the commander"); }

/* Refuses a perfect engine, refuses a second spanner, otherwise
   commits the work and charges the action that started it. */
function startMaint(e, id, who) {
    if (STATE.engineCondition >= 100) {
        logLine("Maintenance refused - the drive is already at full condition.");
        return;
    }
    if (maintRunning()) { return; }

    var cost = maintCost();
    MAINT_CLAIM = true;
    try {
        fireAction(e, "Engine maintenance begun by " + who);
    } catch (err) { MAINT_CLAIM = false; throw err; }
    MAINT_CLAIM = false;

    var t = addOngoing(id, "Maintenance - " + who, cost, "maint", "engine");
    if (t) { t.by = who; }

    paintEngine();
}


/* ================================================================
   SECTION I-4 : THE O2 FILTER ROOM
   Same frame as Navigation and the Engine Room: a fixed stage
   centre-top, reading columns either side, paired panels beneath
   it, one full-width band of commitments at the bottom.
   The only things on this screen that are not scenery are the
   state of the three slots and the charge left in the bank.
   ================================================================ */
function buildO2Screen(host, t) {
    var scr = make("div", "task-screen");

    scr.appendChild(make("div", "task-head",
        '<div class="task-kind">ROOM</div>' +
        '<div class="task-title">O2 Filter Room</div>' +
```

```
      '<div class="task-sub">Air is the only thing aboard that ' +
      'nobody can do without.</div>'));

    var wrap = make("div", "navwrap");
    var main = make("div", "navmain");

    var cl = make("div", "navcell navcell-l");
    cl.appendChild(navPanel("Air Quality",     "o2-air-p",  ""));
    cl.appendChild(navPanel("Filter Lifespan", "o2-life",   ""));
    cl.appendChild(navPanel("Slots",           "o2-slotsp", "np-last"));
    main.appendChild(cl);

    var cc  = make("div", "navcell navcell-c");
    var mid = make("div", "navcenter");
    mid.appendChild(buildFilterMachine());

    var under = make("div", "navrow");
    under.appendChild(navPanel("System Status",    "o2-status", "np-half"));
    under.appendChild(navPanel("Pro-Action Burst", "o2-burst",  "np-half-last"));
    mid.appendChild(under);

    cc.appendChild(mid);
    main.appendChild(cc);

    var cr = make("div", "navcell navcell-r");
    cr.appendChild(navPanel("Canister Inventory", "o2-rack", ""));
    cr.appendChild(navPanel("Life Support Crew", "o2-crew", "np-last"));
    main.appendChild(cr);

    wrap.appendChild(main);

    var r3 = make("div", "navrow");
    r3.appendChild(navPanel("Filter Actions", "o2-actions", "np-full"));
    wrap.appendChild(r3);

    scr.appendChild(wrap);
    host.appendChild(scr);

    paintO2();
}

/* ----------------------------------------------------------------
   THE MACHINE
   Positioned divs again. Scenery carries .o2-part and therefore
   pointer-events:none - the three install buttons are the only
   live things in the box and they are appended LAST so nothing is
   ever painted over them. Fourth time that rule has earned its
   keep in this project.
   ---------------------------------------------------------------- */
var DUST = [[48,66],[96,88],[150,62],[204,94],[262,70],[318,100],
            [372,64],[430,92],[486,72],[534,96],[72,102],[128,74],
            [186,106],[244,60],[300,84],[356,104],[414,68],[510,60]];
```

```
function
 buildFilterMachine() {
    var m = make("div", null);
    m.id = "o2mach";
    m.style.width  = STAGE.o2.w + "px";
    m.style.height = STAGE.o2.h + "px";

    var k, sx, el;

    /* --- intake duct across the top ---------------------- */
    m.appendChild(opart("o2-duct", 20, 14, 560, 30, ""));
    m.appendChild(opart("o2-mini", 26, 19, 60, 10, "INTAKE"));
    for (k = 0; k < 7; k++) {
       m.appendChild(opart("o2-louvre", 100 + k * 66, 20, 6, 18, ""));
    }

    /* --- the cabin air, rebuilt by paintO2 --------------- */
    el = opart("o2-air", 20, 52, 560, 66, "");
    el.id = "o2-air";
    m.appendChild(el);

    /* --- three slot housings ----------------------------- */
    for (k = 0; k < 3; k++) {
       sx = 20 + k * 190;
       m.appendChild(opart("o2-slot", sx, 126, 180, 152, ""));
       m.appendChild(opart("o2-mini", sx + 9, 132, 80, 10, "SLOT " + (k + 1)));

       el = opart("o2-well", sx + 30, 152, 120, 84, "");
       el.id = "o2-well" + k;
       m.appendChild(el);

       el = opart("o2-can", sx + 46, 162, 88, 64, "");
       el.id = "o2-can" + k;
       m.appendChild(el);
       el = opart("o2-neck", sx + 78, 154, 24, 8, "");
       el.id = "o2-neck" + k;
       m.appendChild(el);

       el = opart("o2-state", sx + 8, 242, 164, 14, "EMPTY");
       el.id = "o2-state" + k;
       m.appendChild(el);

       m.appendChild(opart("o2-track", sx + 30, 260, 120, 8, ""));
       el = opart("o2-fill", sx + 30, 260, 0, 8, "");
       el.id = "o2-bar" + k;
       m.appendChild(el);

       /* the live control - deliberately NOT a .o2-part */
       el = make("div", "o2-btnwrap");
       el.id = "o2-btnwrap" + k;
       el.style.left  = (sx + 5) + "px";
```

```
        el.style.top   = "286px";
        el.style.width = "170px";
        m.appendChild(el);
    }

    /* --- outlet manifold ---------------------------------- */
    m.appendChild(opart("o2-duct", 20, 324, 560, 26, ""));
    m.appendChild(opart("o2-mini", 26, 331, 70, 10, "TO DECKS"));
    for (k = 0; k < 8; k++) {
        m.appendChild(opart("o2-flow", 120 + k * 56, 332, 10, 10, ""));
    }
    return m;
}

/* one absolutely positioned piece of scenery */
function opart(cls, x, y, w, h, html) {
    var el = make("div", "o2-part " + cls, html);
    el.style.left = x + "px";
    el.style.top  = y + "px";
    if (w) { el.style.width  = w + "px"; }
    if (h) { el.style.height = h + "px"; }
    return el;
}

/* ----------------------------------------------------------------
   PAINTING
   Everything visible about the bank is derived from LIFE.slots and
   STATE.filterLife. Nothing is stored twice.
   ---------------------------------------------------------------- */
function paintO2() {
    if (!$("o2-actions")) { return; }

    var i, sl, el, run = bankRunning(), tone = airTone(), word = airWord();
    var pct = Math.round((STATE.filterLife / LIFE.lifeMax) * 100);

    paintAir(run, word, tone);

    for (i = 0; i < LIFE.slots.length; i++) {
        sl = LIFE.slots[i];

        el = $("o2-well" + i);
        if (el) {
            el.className = "o2-part o2-well" +
                (sl.state === "empty" ? " o2-well-bare" : "");
        }
        el = $("o2-can" + i);
        if (el) {
            el.style.display = (sl.state === "empty") ? "none" : "block";
            el.className = "o2-part o2-can" +
                (sl.state === "active" ? " o2-can-live" : "");
        }
        el = $("o2-neck" + i);
```

```
    if (el) { el.style.display = (sl.state === "empty") ? "none" : "block"; }

    el = $("o2-state" + i);
    if (el) {
      el.innerHTML = (sl.state === "empty" ? "EMPTY"
                  : (sl.state === "installed" ? "INSTALLED" : "ACTIVE"));
      el.style.color = (sl.state === "active") ? "#7f9161"
                   : ((sl.state === "installed") ? "#8c6f4e" : "#b05a42");
    }
    el = $("o2-bar" + i);
    if (el) {
      el.style.width = (sl.state === "active"
        ? Math.round(120 * (pct / 100)) : 0) + "px";
    }
}

paintO2Buttons();

setPanel("o2-air-p",
    '<div class="navname" style="color:' + tone + ';">' + word + '</div>' +
    '<div class="navsub">' + (run
      ? "Scrubbed and circulating"
      : "Unfiltered - the bank is not running") + '</div>' +
    barLine(airPercent(), tone) +
    field("Cabin air", run ? "Replenished"
                       : (STATE.airHold + " of " + AIR.holdMax)) +
    field("Declines", "one step per " + AIR.step + " actions") +
    field("Crew", airPenalty()) +
    (airStage() < 3
      ? '<div class="navnote">Bad air does not kill anyone today - ' +
        'it makes the crew slow. Work they run themselves takes ' +
        'longer, and they wait later before they act at all.</div>'
      : ""));

setPanel("o2-life",
    '<div class="navname">' + STATE.filterLife +
    ' <span class="navsub">of ' + LIFE.lifeMax + '</span></div>' +
    '<div class="navsub">actions of clean air</div>' +
    barLine(pct, tone) +
    field("Burn rate", LIFE.decay + " per action") +
    field("Full charge", (LIFE.lifeMax / LIFE.decay) + " actions"));

var sh = "", st;
for (i = 0; i < LIFE.slots.length; i++) {
    st = LIFE.slots[i].state;
    sh += field(LIFE.slots[i].name,
      st === "empty" ? "bare" : (st === "installed" ? "seated" : "running"));
}
setPanel("o2-slotsp", sh);

/* "3 of 10" read as a fraction of something the player
   thought he had ten of. The count is the count; the rack
```

```
      size is its own line. */
    setPanel("o2-rack",
      '<div class="navname">' + LIFE.canisters + '</div>' +
      '<div class="navsub">canisters in the rack</div>' +
      field("Rack holds", LIFE.capacity) +
      field("Per install", "1 canister") +
      field("Full bank", LIFE.slots.length + " canisters") +
      '<div class="navnote">A canister leaves the rack the moment an ' +
      'install is committed. Stations sell them; nothing aboard makes ' +
      'them.</div>');

    var hand = assignedFilterHand();
    setPanel("o2-crew",
      (hand
        ? '<div class="navname">' + hand.name + '</div>' +
          '<div class="navsub">' + hand.role + ' - filter job</div>' +
          field("Skill",    hand.skill) +
          field("Acts at",  AIR_STAGES[hand.jobAt].word + " air") +
          field("Activity", hand.activity) +
          field("Seating",  crewCost(2) + " actions per slot") +
          '<div class="navnote">He holds the filter job and works the ' +
          'bank unasked - but not until the air is already ' +
          AIR_STAGES[hand.jobAt].word + ', by which time the crew have ' +
          'been labouring for a while. He is slowed by the same bad ' +
          'air he is there to fix.</div>'
        : '<span class="dim">Nobody holds the filter job. The ' +
          'commander seats every canister personally.</span>'));

    setPanel("o2-status",
      field("Intake", "Open") +
      field("Manifold", run ? "Circulating" : "Static") +
      field("Scrubber", run ? "Running" : "Idle") +
      field("Seated", (slotsIn("installed") + slotsIn("active")) +
          " of " + LIFE.slots.length));

    setPanel("o2-burst",
      '<div class="navnote" style="border:none;padding-top:0;margin-top:0;">' +
      'Starting the bank costs one ordinary action and hands ten back: ' +
      'every job already running jumps forward by ten actions. The ' +
      'counter does not move and nothing wears. It is the only thing ' +
      'aboard that gives time instead of taking it.</div>');

    paintO2Band(run);
}

/* the chamber: specks of dirt, or the clean streaks of moving air */
/* THE INDICATOR ABOVE THE SLOTS.
   One bar, full and green while the bank runs, then draining and
   shifting green -> yellow -> orange -> red in four steps of five
   actions once there is nothing left to filter with. The grime in
   the chamber thickens with every step, so the state is readable
   even out of the corner of an eye. */
```

```
function paintAir(run, word, tone) {
   var air = $("o2-air"), i, el, n;
   if (!air) { return; }
   var st = airStage(), pct = airPercent();

   clear(air);
   air.className = "o2-part o2-air" + (st === 4 ? " o2-air-clean" : "");

   air.appendChild(make("div", "o2-airlab", "CABIN AIR"));

   el = make("div", "o2-airword", word);
   el.style.color = tone;
   air.appendChild(el);

   air.appendChild(make("div", "o2-airsub", run
      ? (STATE.filterLife + " filtered")
      : (STATE.airHold + " of " + AIR.holdMax + " left")));

   air.appendChild(make("div", "o2-airtrack"));
   el = make("div", "o2-airfill");
   el.style.width      = Math.round(544 * (pct / 100)) + "px";
   el.style.background = tone;
   air.appendChild(el);

   /* four step marks, so the drop can be seen coming */
   for (i = 1; i < 4; i++) {
      el = make("div", "o2-airnotch");
      el.style.left = Math.round(544 * (i / 4)) + 8 + "px";
      air.appendChild(el);
   }

   if (run) {
      for (i = 0; i < 4; i++) {
         el = make("div", "o2-streak");
         el.style.left = (60 + i * 130) + "px";
         el.style.top  = "52px";
         air.appendChild(el);
      }
   } else {
      n = (4 - st) * 4;          /* the worse it gets, the dirtier it looks */
      for (i = 0; i < n; i++) {
         el = make("div", "o2-dust");
         el.style.left = (14 + i * 33) + "px";
         el.style.top  = (48 + (i % 3) * 5) + "px";
         air.appendChild(el);
      }
   }
}

/* the three install buttons, rebuilt whenever anything changes */
function paintO2Buttons() {
   var i, wrap, sl, lab, live;
```

```
    for (i = 0; i < LIFE.slots.length; i++) {

   wrap = $("o2-btnwrap" + i);
       if (!wrap) { continue; }
       clear(wrap);
       PROGRESS_UI["o2s" + i] = null;   /* drop the detached ref first */

       sl   = LIFE.slots[i];
       live = false;
       if (findOngoing("o2s" + i))        { lab = "Install Canister"; }
       else if (sl.state === "active")    { lab = "Running"; }
       else if (sl.state === "installed") { lab = "Seated"; }
       else if (LIFE.canisters <= 0)      { lab = "No canisters"; }
       else { lab = "Install Canister"; live = true; }

       wrap.appendChild(buildProgressButton("o2s" + i, lab, installFn(i), live));
    }
}

function paintO2Band(run) {
    var band = $("o2-actions");
    if (!band) { return; }
    clear(band);
    PROGRESS_UI["o2-run"] = null;

    var ready = bankReady() && !run;
    band.appendChild(make("div", "actionbar",
       run
          ? '<span class="dim">The bank is running. ' + STATE.filterLife +
            ' actions of clean air left.</span>'
          : (ready
             ? "All three slots are seated. Starting the bank costs one " +
               "action and advances every running job by ten."
             : '<span class="dim">Seat a canister in all three slots ' +
               'before the bank can be started.</span>')));

    var row = make("div", "actionrow");
    row.appendChild(actionCell(buildProgressButton(
       "o2-run", run ? "Filters running" : "Turn on Filter",
       turnOnFilter, ready)));
    band.appendChild(row);
    band.appendChild(make("div", "clear"));
}

/* a plain proportion bar for a reading panel */
function barLine(pct, tone) {
    var w = pct;
    if (w < 0)   { w = 0; }
    if (w > 100) { w = 100; }
    return '<div class="navbar"><div class="navbar-in" style="width:' + w +
          '%;background:' + tone + ';"></div></div>';
}
```

```
/* ================================================================
   SECTION I-5 : CREW QUARTERS
   The order sheet. Looking at a crew member is free, writing an
   order down is free, changing your mind is free. Nothing is
   spent until the whole sheet is handed out at the bottom of the
   screen - one mark, one feed line, one payment.
   ================================================================ */
function buildCrewScreen(host, t) {
    var scr = make("div", "task-screen");

    scr.appendChild(make("div", "task-head",
        '<div class="task-kind">ROOM</div>' +
        '<div class="task-title">Crew Quarters</div>' +
        '<div class="task-sub">Shared bunks, lockers, and the only ' +
        'place aboard where orders are written down.</div>'));

    var wrap = make("div", "navwrap");
    var main = make("div", "navmain");

    var cl = make("div", "navcell navcell-l");
    cl.appendChild(navPanel("Crew", "cq-roster", "np-last"));
    main.appendChild(cl);

    var cc  = make("div", "navcell navcell-c");
    var mid = make("div", "navcenter");
    mid.appendChild(buildHallway());

    var trow = make("div", "navrow");
    trow.appendChild(navPanel("The Figure", "cq-talk", "np-full"));
    mid.appendChild(trow);

    var under = make("div", "navrow");
    under.appendChild(navPanel("Condition", "cq-cond", "np-half"));
    under.appendChild(navPanel("Standing",  "cq-stand", "np-half-last"));
    mid.appendChild(under);

    cc.appendChild(mid);
    main.appendChild(cc);

    var cr = make("div", "navcell navcell-r");
    cr.appendChild(navPanel("Orders", "cq-orders", ""));
    cr.appendChild(navPanel("Order Sheet", "cq-sheet", "np-last"));
    main.appendChild(cr);

    wrap.appendChild(main);

    var r3 = make("div", "navrow");
    r3.appendChild(navPanel("Assignments", "cq-band", "np-full"));
    wrap.appendChild(r3);
```

```
    scr.appendChild(wrap);
    host.appendChild(scr);

    paintCrewScreen();
}

/* ---------------------------------------------------------------
   THE HALLWAY
   Lockers down both walls, a door at the far end, a light strip
   overhead. All scenery, all .cq-part, no clicks.
   --------------------------------------------------------------- */
function buildHallway() {
    var m = make("div", null);
    m.id = "hallway";
    m.style.width  = STAGE.crew.w + "px";
    m.style.height = STAGE.crew.h + "px";

    var k, x, el;

    m.appendChild(cpart("cq-wall",  0, 0, 560, 230, ""));
    m.appendChild(cpart("cq-floor", 0, 230, 560, 70, ""));
    for (k = 0; k < 4; k++) {           /* floorboards, widening */
        m.appendChild(cpart("cq-board", 0, 236 + k * (8 + k * 4), 560, 1, ""));
    }
    m.appendChild(cpart("cq-skirt", 0, 226, 560, 4, ""));

    /* ceiling light */
    m.appendChild(cpart("cq-light", 236, 0, 88, 9, ""));
    m.appendChild(cpart("cq-glow",  216, 9, 128, 4, ""));

    /* the door at the far end */
    m.appendChild(cpart("cq-door",  236, 64, 88, 162, ""));
    m.appendChild(cpart("cq-dpane", 248, 78, 64, 52, ""));
    m.appendChild(cpart("cq-dhand", 306, 150, 8, 14, ""));
    m.appendChild(cpart("cq-mini",  236, 52, 88, 10, "DECK A"));

    /* two banks of lockers */
    for (k = 0; k < 8; k++) {
        x = (k < 4) ? (16 + k * 40) : (384 + (k - 4) * 40);
        m.appendChild(cpart("cq-lock", x, 70, 38, 156, ""));
        m.appendChild(cpart("cq-lvent", x + 8, 80, 22, 2, ""));
        m.appendChild(cpart("cq-lvent", x + 8, 85, 22, 2, ""));
        m.appendChild(cpart("cq-lvent", x + 8, 90, 22, 2, ""));
        m.appendChild(cpart("cq-lhand", x + 28, 140, 4, 16, ""));
        m.appendChild(cpart("cq-lnum",  x + 6, 208, 26, 10,
                            "0" + (k + 1)));
    }

    /* the figure stands here */
    m.appendChild(cpart("cq-shadow", 248, 222, 64, 10, ""));
    /* ONE LAYER FOR THE WHOLE SCENE. The figure is no longer
       alone in a 72x84 box - a cot, a bench or a crate stands
```

```
      beside it, so the layer is the full stage and everything
      in it is placed in hallway coordinates. Scenery, so it
      takes no clicks. */
    el = make("div", "cq-scene");
    el.id = "crew-scene";
    m.appendChild(el);

    return m;
}
function cpart(cls, x, y, w, h, html) {
    var el = make("div", "cq-part " + cls, html);
    el.style.left = x + "px";
    el.style.top  = y + "px";
    if (w) { el.style.width  = w + "px"; }
    if (h) { el.style.height = h + "px"; }
    return el;
}

/* ------------------------------------------------------------------
   THE SPRITE
   Sixteen rows of twelve, five pixels each, built as divs. No
   image file, no font, no canvas - the same discipline as every
   other picture in this build. Letters are colour keys:
     h hair   s skin   k line   c clothing   d clothing shadow
     b boots  . nothing
   ------------------------------------------------------------------ */
var SPRITE = [
    ".......kkk........",
    "......khhk........",
    ".......khhk.......",
    "........khk.......",
    "....kkkkkkkkkk....",
    "..kkhhhhhhhhhhkk..",
    ".khhHHhhhhhhhhhk.",
    ".khhHhhhhhhhhhhk.",
    ".khhhsssssssshhhk.",
    ".khhhsssssssshhhk.",
    ".khhhseesseeshhhk.",
    ".khhhseesseeshhhk.",
    ".khhhsssssssshhhk.",
    "..khhsssssssshhk..",
    "...kkssssssssk...",
    "...kkccccccccckk...",
    "..kccccccccccck..",
    "..kscccyyccccsk..",
    "...kkddddddddkk...",
    "...kkdddkkdddkk...",
    "...kkbbbkkbbbkk..."
];
var SP_BLANK = "................";

/* Lying down - used twice: on a cot at the medical bay, and on
```

```
   the deck by a hand with nothing to do. Head to the left. */
var LIE = [
    "......kkkk.................",
    "....kkhhhhhk..kkkkkkkkk...",
    "...khhhhhhhhhkkccccccccckk..",
    "..khhsssssshkccccccccccckk.",
    "..khseessshkcccccccccddk.",
    "..khhssssshkkcccccccccddk.",
    "...khhhhhhkk.kkkkkkkkkddk.",
    "....kkkkkk.........kbbbbk.",
    ".....................kkkkk."
];

/* Dozing: sat down, head tipped into the right shoulder, eyes
   shut. Flat on the deck read as collapsed rather than asleep -
   the Commander's word was "squished". 18 x 16. */
var SIT = [
    "......kkkkkkkk....",
    "....kkhhhhhhhhkk..",
    "...khhhhhhhhhhhk.",
    "...khhHHhhhhhhhhk.",
    "...khhhsssssshhk.",
    "..khhhhsssssshhk.",
    "..khhhhkkssskkshk.",
    "..khhhhsssssshhk.",
    "..kkhhhsssssshk.",
    "...kkssssssskk..",
    "....kkccccccckk..",
    "..kkccccccccck..",
    "..kscccyyccccsk..",
    "..kkddddddddkk...",
    ".kddddddddddddddk.",
    ".kbbkkkkkkkkkbbk."
];

/* ------------------------------------------------------------
   PROPS
   The order sheet changes what the figure is WEARING; these say
   what it is about to DO. Same pixel language, same 4px, all of
   them standing on the deck line. A new order on the sheet means
   a new prop here and one line in crewScene - nothing else.
   ------------------------------------------------------------ */
var PROPS = {
    desk : [ ".....kkkkkkkkkk.....", ".....kggggggggk.....",
             ".....kggggggggk.....", ".....kggggggggk.....",
             ".....kggggggggk.....", ".....kkkkkkkkkk.....",
             ".......kkkkkk.......", "......kkkkkkkk......",
             "kkkkkkkkkkkkkkkkkkkk", "kddddddddddddddddddddk".substring(0,20),
             "kkkkkkkkkkkkkkkkkkkk", "..k..............k..",
             "..k..............k..", "..k..............k..",
             "..k..............k..", "..k..............k.." ],
    bed  : [ "..wwwwww..................", ".kwwwwwwkkkkkkkkkkkkkkkkk.",
```

```
            ".kwwwwwwkmmmmmmmmmmmmmmk.",
".kkkkkkkkmmmmmmmmmmmmmmk.",
            ".kkkkkkkkkkkkkkkkkkkkkkk.", ".k.....................k.",
            ".k.....................k.", ".kk.....................kk." ],
    cross:
[ "wwwwwww","wwwrwww","wwwrwww","wrrrrrw","wwwrwww","wwwrwww","wwwwwww"
],
    zzz  : [ ".........kkkk",".........k.k.",".........kk..",".........k.k..",
            ".........kkkk","..kkkk.......","...k.k.......","..kk.........",
            "..kkkk.......","kkk..........","k.k..........","kk...........",

            "kkk.........." ],
    wrench:[ "..kk...kk..","..kkk.kkk..","...kkkkk...","....kkk....",
            "....mkm....","....mkm....","....mkm....","....mkm....",
            "....mkm....","...kkkkk..." ],
    mop  : [ ".....k.....",".....k.....",".....k.....",".....k.....",
            ".....k.....",".....k.....","...kkkkk...","..kmmmmmk..",
            "..kmmmmmk..","...kkkkk..." ],
    chart: [ "kkkkkkkkk","kwwwwwwwk","kwkwwwwwk","kwwkwwwwk","kwwwkwwwk",
            "kwwwwkwwk","kwwwwwwwk","kkkkkkkkk" ],
    tank : [ "...kk...","...kk...","..kkkk..",".kmmmmk.","kmmmmmmk",
            "kmmmmmmk","kmmmmmmk","kmmmmmmk","kmmmmmmk",".kkkkkk." ],
    crate: [ "kkkkkkkkkk","kcccccccck","kcckkkkcck","kcccccccck",
            "kcckkkkcck","kcccccccck","kkkkkkkkkk" ]
};
var JOB_TOOL = { maintenance:"wrench", cleaner:"mop",
                 navigation:"chart",  filters:"tank" };

/* HATS. Worn over the head, drawn after the figure, "j" taking
   the job's own colour. A hat reads across the room faster than
   a tunic does - at 4px the yellow helmet and the white cap are
   telling you the trade before you have focused on the figure.
   The slate used for life support is deliberately muted: the
   palette has no true blue in it and is not getting one. */
var HATS = {
    helmet : [ "...kkkkkkkk...", "..kjjjjjjjk..", ".kjjjjjjjjk.",
               ".kjjjjjjjjk.", "kkkkkkkkkkkkkk" ],
    cap    : [ "...kkkkkkkk...", "..kjjjjjjjk..", ".kjjjyjjjjk.",
               ".kkkkkkkkkkk.", "..kkkkkkkkkk.." ],
    bandana: [ "...kkkkkkkk...", "..kjjjjjjjk..", ".kjjjjjjjjk.",
               ".kkjjjjjjjkk.", "..k.kk..kk.k.." ]
};
var JOB_HAT = { maintenance:["helmet", "#e0b53a"],   /* hard hat   */
                navigation :["cap",    "#faf7f2"],   /* peaked cap */
                cleaner    :["bandana","#7f9161"],   /* kerchief   */
                filters    :["helmet", "#6e8491"] }; /* slate helm */
var FLOORY = 230;      /* the deck line inside the hallway */

/* ONE BODY, FOUR HEADS.
   A style is a sparse set of row overrides on SPRITE - only the
   rows that differ are stored, so a new haircut is four short
   strings and not a second sprite to keep in step with the
```

```
   first. Chibi has no neck: the jaw at row 14 sits straight on
   the collar at row 15. */
var HAIR_STYLES = {
    cowlick : {},                    /* the base head */
    crop    : { 0:SP_BLANK, 1:SP_BLANK, 2:SP_BLANK, 3:SP_BLANK },
    bun     : { 0:SP_BLANK,
                1:".......kkk........",
                2:"......khHk........",
                3:"......khhk........" },
    long    : { 0:SP_BLANK, 1:SP_BLANK, 2:SP_BLANK, 3:SP_BLANK,
                14:"..khkssssssskhk..",
                15:"..khkccccccckhk..",
                16:".khccccccccccchk.",
                17:".khscccyyccccshk." },
    /* no hair at all - the dome is skin and only the outline
       stays. Kept for hires as much as for the crew aboard. */
    bald    : { 0:SP_BLANK, 1:SP_BLANK, 2:SP_BLANK, 3:SP_BLANK,
                5:"..kkssssssssssskk.",  6:".kssssssssssssssk.",
                7:".kssssssssssssssk.",  8:".kssssssssssssssk.",
                9:".kssssssssssssssk.", 10:".kssseessseessssk.",
               11:".kssseessseessssk.", 12:".kssssssssssssssk.",
               13:"..kssssssssssssk.." },
    /* a little left, round the sides */
    thin    : { 0:SP_BLANK, 1:SP_BLANK, 2:SP_BLANK, 3:SP_BLANK,
                5:"..kkssssssssssskk.", 6:".kssssssssssssssk.",
                7:".khsssssssssssshk." }
};

/* Row 9 rebuilt with a taller eye and a lash tick at each outer
   corner. SPLICED into whatever row 9 the style left behind -
   write it as a fixed string and a bald head grows hair again. */
function femaleRow(r9) {
    return r9.substring(0, 5) + "eeesseee" + r9.substring(13);
}
var PX = 4;

/* ---------------------------------------------------------------
   THE SCENE
   What is standing around the figure says what the sheet is
   about to do to them. Priority is what matters most: a body in
   the medical bay first, then schooling, then a change of trade,
   then nobody wanting them at all.
   --------------------------------------------------------------- */
function crewScene(c) {
    var o   = STATE.pendingOrders[c.id] || {};
    var sc  = { pose:"stand", props:[] };
    var jid, tool;

    if (o.doc || atDoc(c)) {                 /* on the cot */
        sc.pose = "lie";
        sc.props[sc.props.length] = ["bed",   226, onDeck(PROPS.bed)];
        sc.props[sc.props.length] = ["cross", 252, 138];
```

```
        sc.lieOn = onDeck(PROPS.bed) - LIE.length * PX + 12;
        return sc;
    }
    if (o.learnOn || o.learnSt) {              /* at the bench */
        sc.props[sc.props.length] = ["desk", 120, onDeck(PROPS.desk)];
        return sc;
    }
    /* THE TRADE THEY WILL BE DOING - staged if the sheet changes
       it, otherwise the one they already hold. A working hand
       should look like one; only the crate marks it as NEW. */
    jid = (o.job !== undefined) ? o.job : (c.job || "");
    if (jid) {
        tool = JOB_TOOL[jid];
        if (o.job !== undefined && o.job !== (c.job || "")) {
            sc.props[sc.props.length] = ["crate", 170, onDeck(PROPS.crate)];
        }
        if (tool) {
            sc.props[sc.props.length] =
                [tool, 312, FLOORY - 14 - PROPS[tool].length * PX];
        }
        return sc;
    }
    if (!c.job) {                              /* nobody wants him */
        sc.pose = "sit";
        sc.props[sc.props.length] = ["zzz", 300, 118];
        return sc;
    }
    return sc;
}
function onDeck(rows) { return FLOORY - rows.length * PX; }

/* redrawn only when it would look different - this layer is
   several hundred divs and paintCrewScreen runs every action */
var FIGURE_SIG = "";

function paintSprite(c) {
    var host = $("crew-scene"), i, sc, look, sig;
    if (!host) { return; }
    if (!c) { clear(host); FIGURE_SIG = ""; return; }

    sc   = crewScene(c);
    look = spriteLook(c);
    sig  = c.id + "|" + sc.pose + "|" + look.cloth + "|" +
           (look.hat || "-") + "|" + c.hairStyle;
    for (i = 0; i < sc.props.length; i++) { sig += "|" + sc.props[i][0]; }
    if (sig === FIGURE_SIG) { return; }
    FIGURE_SIG = sig;

    clear(host);
    for (i = 0; i < sc.props.length; i++) {
        drawPix(host, PROPS[sc.props[i][0]], sc.props[i][1],
                sc.props[i][2], c, look);
```

```
    }
    if (sc.pose === "lie") {
        drawPix(host, LIE, 230, sc.lieOn, c, look);
    } else if (sc.pose === "sit") {
        drawPix(host, SIT, 244, onDeck(SIT), c, look);   /* eyes shut */
    } else {
        drawPix(host, figureRows(c, !!look.hatKind), 244,
                onDeck(SPRITE), c, look);
        if (look.hatKind) {       /* the hat goes on last, over the hair */
            drawPix(host, HATS[look.hatKind], 252,
                    onDeck(SPRITE) + 12, c, look);
        }
    }
}

/* The standing figure: haircut, then the hat, then the face.
   NOTHING POKES OUT OF A HELMET - rows 0 to 3 are the cowlick
   and the bun, and a hat blanks them outright. */
function figureRows(c, hatted) {
    var st = HAIR_STYLES[c.hairStyle || "cowlick"] || {}, out = [], r;
    for (r = 0; r < SPRITE.length; r++) {
        out[r] = (st[r] !== undefined) ? st[r] : SPRITE[r];
    }
    if (hatted) {
        for (r = 0; r < 4; r++) { out[r] = SP_BLANK; }
    }
    if (c.sex === "f") { out[9] = femaleRow(out[9]); }
    return out;
}

/* one grid of pixels, placed in hallway coordinates */
function drawPix(host, rows, ox, oy, c, look) {
    var r, q, ch, el, col;
    for (r = 0; r < rows.length; r++) {
        for (q = 0; q < rows[r].length; q++) {
            ch = rows[r].charAt(q);
            if (ch === ".") { continue; }
            col = (ch === "h") ? (c.hair  || "#5e3622")
                : (ch === "H") ? (c.hair2 || "#804e2e")
                : (ch === "s") ? (c.skin  || "#f7e4c0")
                : (ch === "e") ? "#3a1c2e"
                : (ch === "k") ? "#1a1614"
                : (ch === "b") ? "#1a1614"
                : (ch === "w") ? "#faf7f2"
                : (ch === "r") ? "#b05a42"
                : (ch === "g") ? "#bfcdba"
                : (ch === "m") ? "#cbb89c"
                : (ch === "j") ? (look.hat || "#cbb89c")
                : (ch === "y") ? (look.btn || "#e2b63c")
                : (ch === "c") ? look.cloth
                : (ch === "d") ? look.dark
                : "#5c5145";
```

```
            el = make("div", "cq-px");
            el.style.left       = (ox + q * PX) + "px";
            el.style.top        = (oy + r * PX) + "px";
            el.style.width      = PX + "px";
            el.style.height     = PX + "px";
            el.style.background = col;
            host.appendChild(el);
        }
    }
}

/* what the figure is wearing: the STAGED job, not the saved one,
   so the player sees the change before paying for it */
function spriteLook(c) {
    var o = STATE.pendingOrders[c.id];
    if ((o && o.doc) || atDoc(c)) {
        /* patient gown - even the buckle goes quiet */
        return { cloth:"#e8e2d6", dark:"#cbb89c", btn:"#cbb89c" };
    }
    var jid = (o && o.job !== undefined) ? o.job : (c.job || "");
    var j = findById(JOBS, jid), hat = JOB_HAT[jid];
    j = j ? j : JOBS[0];
    return { cloth:j.cloth, dark:j.dark,
             hatKind:(hat ? hat[0] : null),
             hat:    (hat ? hat[1] : null) };
}

/* ---------------- selection is free ------------------------- */
function selectCrew(id) {
    STATE.crewSel = id;
    paintCrewScreen();          /* no action, no mark, no tick */
}
function selCrew() {
    var c = findById(CREW, STATE.crewSel);
    return c ? c : (CREW.length ? CREW[0] : null);
}

/* ---------------- staging ----------------------------------- */
function stageJob(id, jobId) {
    var c = findById(CREW, id);
    if (!c ||
 ordersRunning() || crewBusy(c)) { return; }
    var o = STATE.pendingOrders[id] || {};
    if ((c.job || "") === jobId) { delete o.job; }   /* no change, no charge */
    else { o.job = jobId; }
    keepOrder(id, o);
    paintCrewScreen();
}
function toggleOrder(id, oid) {
    /* the UI does not wire this row without a dock, but the rule
       belongs with the data, not with the button */
    if (oid === "learnSt" && !STATE.docked) { return; }
```

```
    if (ordersRunning())                  { return; }
    if (crewBusy(findById(CREW, id)))     { return; }
    var o = STATE.pendingOrders[id] || {};
    if (o[oid]) { delete o[oid]; } else { o[oid] = true; }
    keepOrder(id, o);
    paintCrewScreen();
}
function keepOrder(id, o) {
    var k, any = false;
    for (k in o) { if (o.hasOwnProperty(k)) { any = true; } }
    if (any) { STATE.pendingOrders[id] = o; }
    else     { delete STATE.pendingOrders[id]; }
}
/* What a single line on the sheet costs. It is never a flat
   number: handing someone the life support job is not the same
   size of decision as handing them a mop, and teaching them
   either one costs what that knowledge is worth. */
/* THE PRICE OF ONE ORDER, FOR ONE JOB.
   Every caller must say which job it means. Twice now a caller
   has been allowed to assume, and twice it assumed the job the
   crew member is leaving instead of the one he is taking:
   Entry 048 charged a cleaner's price for an engineer's
   schooling, and then the per-person task lengths disagreed
   with the sheet total they were cut from. There is one price
   list and it answers questions - it does not guess them. */
/* The ACTIONS an order costs are the crew's time. The SHIPLETS
   are what it costs the ship - and only schooling costs those:
   somebody has to be paid to teach. Aboard is cheap and slow, a
   station has instructors and charges for them. Charged once,
   when the sheet is handed over, never while it is being
   written. */
function orderFeeFor(jid, oid) {
    var j = findById(JOBS, jid);
    if (oid === "learnOn") { return j && j.feeOn ? j.feeOn : 0; }
    if (oid === "learnSt") { return j && j.feeSt ? j.feeSt : 0; }
    return 0;
}
function orderFee(c, oid) {
    var o   = STATE.pendingOrders[c.id];
    var jid = (o && o.job !== undefined) ? o.job : (c.job || "");
    return orderFeeFor(jid, oid);
}
function crewFeeOf(c) {
    var o = STATE.pendingOrders[c.id], i, n = 0;
    if (!o) { return 0; }
    for (i = 0; i < ORDERS.length; i++) {
        if (o[ORDERS[i].id]) { n += orderFee(c, ORDERS[i].id); }
    }
    return n;
}
function ordersFee() {
    var id, c, n = 0;
```

```
    for (id in STATE.pendingOrders) {
      if (!STATE.pendingOrders.hasOwnProperty(id)) { continue; }
      c = findById(CREW, id);
      if (c) { n += crewFeeOf(c); }
    }
    return n;
}

function orderCostFor(jid, oid) {
    var j = findById(JOBS, jid);
    if (oid === "doc")     { return 1; }
    if (oid === "fire")    { return 5; }
    if (oid === "learnOn") { return j && j.learnOn ? j.learnOn : 1; }
    if (oid === "learnSt") { return j && j.learnSt ? j.learnSt : 2; }
    return 1;
}

/* what the pad shows: priced against the job STAGED on the sheet
   above it, falling back to the one they already hold */
function orderCost(c, oid) {
    var o   = STATE.pendingOrders[c.id];
    var jid = (o && o.job !== undefined) ? o.job : (c.job || "");
    return orderCostFor(jid, oid);
}

function jobCost(jid) {
    var j = findById(JOBS, jid);
    return j ? j.cost : 1;
}
function crewCostOf(c) {
    var o = STATE.pendingOrders[c.id], i, n = 0;
    if (!o) { return 0; }
    if (o.job !== undefined) { n += jobCost(o.job); }
    for (i = 0; i < ORDERS.length; i++) {
      if (o[ORDERS[i].id]) { n += orderCost(c, ORDERS[i].id); }
    }
    return n;
}
function ordersCost() {
    var id, c, n = 0;
    for (id in STATE.pendingOrders) {
      if (!STATE.pendingOrders.hasOwnProperty(id)) { continue; }
      c = findById(CREW, id);
      if (c) { n += crewCostOf(c); }
    }
    return n;
}
function orderedCount() {
    var id, n = 0;
    for (id in STATE.pendingOrders) {
      if (STATE.pendingOrders.hasOwnProperty(id)) { n++; }
    }
```

```
    return n;
}

/* ---------------- the batch ---------------------------------- */
function commitOrders(e) {
    var n = ordersCost();
    if (n <= 0) { return; }

    /* the schooling is paid for here or the sheet does not go out
       at all - a half-paid order would teach half a lesson */
    var fee = ordersFee();
    if (fee > STATE.shiplets) {
        logLine("Refused - the sheet needs " + fee + " sh in fees and " +
                "the purse holds " + STATE.shiplets + ".");
        return;
    }

    var sheet = STATE.pendingOrders;
    STATE.pendingOrders = {};           /* the sheet is handed over */

    /* ONE CLICK IS ONE ACTION. Handing the sheet over is a single
       act and costs a single action, exactly like locking a
       course or starting a scan. The price of the orders is not
       charged here in a lump - it is the WORK, and the crew
       spend it carrying the orders out while the ship goes on
       around them. Charging it all at once made one click move
       every other countdown aboard by four or five steps, which
       is not an action economy, it is a teleport. */
    fireAction(e, "Crew assignments issued");

    /* queued AFTER that action, or the sheet would eat its own
       first step - the same trap as the engineer's */
    if (fee > 0) {
        STATE.shiplets -= fee;
        paintResources();
        logLine("Training fees paid - " + fee + " sh. Purse " +
                STATE.shiplets + " sh.");
    }
    queueOrders(sheet, n);
    logLine("Crew assignments updated. " + n + " actions of work.");

    refreshLifeInfo();
    paintIndicators();
    paintCrewScreen();
}

/* One task per crew member, however many lines they were given.
   The sidebar grows a line for each, and the roster locks them
   out until it is finished. */
/* ONE task for the whole sheet, carrying the whole sheet, under
   the id of the button at the bottom - so that button IS the
   progress bar, exactly like every other commitment in the game.
```

```
   Until it fills, nobody takes new orders. If the player left
   somebody off the sheet, that is a decision he has made. */
function queueOrders(sheet, total) {
    var id, c, t, n;

    /* the bar at the bottom: the whole sheet, all of it */
    t = addOngoing("crew-assign", "Carrying out orders",
                total, "crewOrder", "crew");
    if (t) { t.sheet = sheet; }

    /* and one task each, sized to what THAT person was asked to
       do, so nobody is held by somebody else's workload */
    for (id in sheet) {
        if (!sheet.hasOwnProperty(id)) { continue; }
        c = findById(CREW, id);
        if (!c) { continue; }
        n = sheetCost(c, sheet[id]);
        t = addOngoing("cw-" + id, "Orders - " + c.name.split(" ")[0],
                    n, "crewJob", id);
        if (t) { t.who = id; t.orders = sheet[id]; }
        c.activity = sheet[id].doc ? "On the way to the medical bay"
                                   : "Carrying out orders";
    }
    paintIndicators();
}

/* the same sum the sheet showed, read from a sheet already
   handed over */
function sheetCost(c, o) {
    var i, n = 0, jid;
    if (!o) { return 1; }
    /* the sheet has already been handed over, so STATE.pending-
       Orders is empty: the staged job must come out of the sheet
       itself or these tasks will not add up to the bar the
       player was charged for */
    jid = (o.job !== undefined) ? o.job : (c.job || "");
    if (o.job !== undefined) { n += jobCost(o.job); }
    for (i = 0; i < ORDERS.length; i++) {
        if (o[ORDERS[i].id]) { n += orderCostFor(jid, ORDERS[i].id); }
    }
    return n < 1 ? 1 : n;
}
/* what the order was actually FOR, applied when it is finished */
function applyOrderPayload(c, o) {
    if (!o) { return; }
    /* FIRST, because nothing else on the sheet means anything to
       somebody who is being put off the ship. Doing this last
       left a medical countdown running for a man who was not
       aboard. */
    if (o.fire) { fireCrew(c); return; }
    if (o.job !== undefined) { applyJob(c, o.job); }
    if (o.doc) {
```

```
      /* the sheet is done; the doctor is not. This is the one
         order whose consequence outlives the order itself. */
      var d = addOngoing("doc-" + c.id, "Medical - " + c.name.split(" ")[0],
                         DOC_AWAY, "crewDoc", c.id);
      if (d) { d.who = c.id; }
      c.activity = "In the medical bay";
      logLine(c.name + " went into the medical bay - about " +
            DOC_AWAY + " actions.");
    }
    if (o.learnOn) { trainCrew(c, 1, "aboard"); }
    if (o.learnSt && STATE.docked) { trainCrew(c, 2, "at the station"); }
    if (!o.doc)    { c.activity = "Back on station"; }
}

/* the sheet as a whole is either out or it is not */
function ordersRunning() { return findOngoing("crew-assign"); }

/* a hand is unavailable if the running sheet has their name on
   it, or if the doc still has them */
function crewBusy(c) {
    if (!c) { return null; }
    var d = findOngoing("doc-" + c.id);
    if (d) { return d; }
    return findOngoing("cw-" + c.id);
}
function atDoc(c) {
    return c ? findOngoing("doc-" + c.id) : null;
}

/* A unique job belongs to one pair of hands. Handing it over
   takes it off whoever had it, which is the only honest way to
   do it - two engineers both "on call" would just race. */
function applyJob(c, jid) {
    var j = findById(JOBS, jid), i;
    if (j && j.unique) {
        for (i = 0; i < CREW.length; i++) {
            if (CREW[i] !== c && CREW[i].job === jid) {
                CREW[i].job = "";
                logLine(CREW[i].name + " was relieved of the " +
                      j.label.toLowerCase() + " job.");

     }
        }
    }
    c.job = jid;
    if (jid === "") {
        logLine(c.name + " has no standing job.");
        return;
    }
    if (!c.skill) { c.skill = "Untrained"; }
    c.jobAt = jobBase(jid);
    logLine(c.name + " now holds the " + j.label.toLowerCase() + " job.");
```

```
}
function jobBase(jid) {
   if (jid === "maintenance") { return 30; }
   if (jid === "filters")     { return 1; }
   if (jid === "navigation")  { return 1; }
   return 0;
}
/* steps = how far this session moves them. A station has
  instructors and a simulator; the ship has a manual. */
function trainCrew(c, steps, where) {
   var j = findById(JOBS, c.job || ""), k;
   c.skill = (c.skill === "Untrained" || !c.skill) ? "Trained" : "Practised";
   if (j && j.trainStep) {
      for (k = 0; k < steps; k++) {
         c.jobAt = Math.min(j.trainCap,
                        (c.jobAt || jobBase(c.job)) + j.trainStep);
      }
      logLine(c.name + " trained " + where + " - now acts at " +
            c.jobAt + j.unit + ".");
   } else {
      logLine(c.name + " trained " + where +
            ", but holds no job to use it on.");
   }
}
/* The order the Commander calls "fire" - unemployment, not an
  airlock. He stays aboard, stops drawing a wage, stops doing
  the job, and joins the others drifting in the corridor. */
function fireCrew(c) {
   if (!c.job) {
      logLine(c.name + " holds no job to be taken off.");
      return;
   }
   layOff(c, "dismissed by the commander");
}

/* ---------------- painting ---------------------------------- */
function paintCrewScreen() {
   if (!$("cq-band")) { return; }
   var c = selCrew();

   paintRoster();
   paintSprite(c);
   paintTalk(c);
   paintCrewPanels(c);
   paintOrders(c);
   paintSheet();
   paintCrewBand();
}

function paintRoster() {
   var host = $("cq-roster"), i, c, row, o;
   if (!host) { return; }
```

```
    clear(host);

    /* Four numbers that mean four different things. Crammed onto
       one line they read as one number with decoration - the
       Commander could not tell the purse from the bill. */
    host.appendChild(make("div", "cq-pay",
        '<div class="cq-payh">PAYROLL</div>' +
        payLine("Wages owed", payrollBill() + " sh") +
        payLine("Every",      PAY.every + " actions") +
        payLine("Next pay",   "in " + payrollDue()) +
        payLine("Purse",      STATE.shiplets + " sh")));
    for (i = 0; i < CREW.length; i++) {
        c   = CREW[i];
        o   = STATE.pendingOrders[c.id];
        var bt = crewBusy(c);
        row = make("div", "cq-row" +
             (c.id === STATE.crewSel ? " cq-row-sel" : "") +
             (bt ? " cq-row-busy" : ""));
        row.innerHTML =
           '<span class="cq-dot" style="background:' + c.colour + ';"></span>' +
           '<span class="cq-nm">' + c.name + '</span>' +
           (bt ? '<span class="cq-flag cq-flag-busy">' + bt.left + '</span>'
               : (o ? '<span class="cq-flag">order</span>' : "")) +
           '<div class="cq-rl">' + (bt
               ? (atDoc(c) ? "In the medical bay" : "Carrying out orders")
               : (shortRole(c.role) +
                  (c.job ? " - " + jobLabel(c.job) : ""))) + '</div>';
        wireSelect(row, c.id);
        host.appendChild(row);
    }
}
function payLine(k, v) {
    return '<div class="cq-payrow"><span class="cq-payk">' + k +
           '</span><span class="cq-payv">' + v +
           '</span><div class="clear"></div></div>';
}
function wireSelect(el, id) {
    on(el, "click", function (e) {
        selectCrew(id);
        if (e && e.stopPropagation) { e.stopPropagation(); }
        if (window.event) { window.event.cancelBubble = true; }
        return false;
    });
}
function jobLabel(jid) {
    var j = findById(JOBS, jid);
    return j ? j.label : "None";
}

/* TALK IS NOT AN ORDER. It is one action, spent on the spot,
   and what it buys is information: what this person actually
   needs, in their own words. Everything else on this screen is
```

```
   the commander deciding; this is the crew answering. */
function paintTalk(c) {
    var host = $("cq-talk"), row, btn;
    if (!host) { return; }
    clear(host);
    if (!c) { return; }

    row = make("div", "cq-talkrow");
    if (atDoc(c)) {
        btn = make("div", "cq-talkbtn cq-talkbtn-off",
                c.name.split(" ")[0] + " is in the medical bay");
    } else {
        btn = make("div", "cq-talkbtn",
                "Talk to " + c.name.split(" ")[0]);
        wireTalk(btn);
    }
    row.appendChild(btn);
    row.appendChild(make("div", "cq-says", c.said
        ? "" + c.said + ""
        : '<span class="dim">You have not asked them anything yet.</span>'));
    row.appendChild(make("div", "clear"));
    host.appendChild(row);
}
function wireTalk(el) {
    on(el, "click", function (e) { talkTo(e); });
}
/* FREE, and it hands out nothing but words. A button that costs
   one action and gives a small reward is a button the player
   will hammer - so this one costs nothing and rewards nothing.
   All it does is let the crew answer. */
function talkTo(e) {
    var c = selCrew();
    if (!c || atDoc(c)) { return; }
    c.said = crewNeed(c);
    logLine(c.name + ': "' + c.said + '"');
    paintCrewScreen();
}

/* ----------------------------------------------------------------
   WHAT THEY SAY
   Ask once and you get the loudest thing. Ask again and you get
   another line about the same trouble - people repeat themselves
   differently, and a player who keeps asking should hear a
   person, not a recording. The order of the tests is the order
   of what hurts: a sick body first, then the air, then work they
   were never taught, then the softer complaints.
   Every line points at something fixable on another screen.
   ---------------------------------------------------------------- */
var NEED_LINES = {
    ill : [
        "I am not right. I should see the doc.",
        "Been feeling rough for a while now. Do not make me say it twice.",
```

```
      "I can work. I would rather not, but I can work.",
      "Whatever I picked up, it has not let go of me yet."
    ],
    air : [
      "The air in here is filthy. Sort the filters and I will sort everything else.",
      "Every breath tastes of the last one. How long has the bank been dead?",
      "My head is splitting and it is not the work.",
      "You can smell the ship going stale. That is not supposed to happen."
    ],
    untrained : [
      "I am doing the {job} work on guesswork. Teach me properly and I will catch things sooner.",
      "Nobody ever showed me the {job} job. I copy what I saw the last one do.",
      "I will keep guessing at {job} if you like. It has held so far.",
      "Send me to learn the {job} work. I would rather know than hope."
    ],
    morale : [
      "Put me on something else. This job is wearing on me.",
      "I am not asking for much. Just something different for a while.",
      "Same station, same hours, same walls. You see the problem.",
      "I have stopped looking forward to my shifts."
    ],
    tired : [
      "I need a shift off my feet.",
      "I have been awake longer than the drive has.",
      "Give me an hour in the bunk and I will be twice the use."
    ],
    hungry : [
      "When did we last eat properly?",
      "Rations again. I am starting to dream about the mess hall.",
      "Feed the crew, commander. It is cheaper than replacing us."
    ],
    idle : [
      "Give me something to be responsible for.",
      "I will take any job going. Standing about is worse than work.",
      "You have a spare pair of hands here doing nothing."
    ],
    fine : [
      "Nothing to report. The {job} work is in hand.",
      "All quiet. I will tell you the moment it is not.",
      "Ship is behaving. Long may it last.",
      "No complaints today, and that is rare enough to mention."
    ]
};

function needKey(c) {
    if (c.health < 75)                          { return "ill"; }
    if (airStage() <= 1)                        { return "air"; }
    if (c.job && (c.skill === "Untrained" || !c.skill)) { return "untrained"; }
    if (c.morale < 60)                          { return "morale"; }
    if (c.fatigue > 70)                         { return "tired"; }
    if (c.hunger  > 70)                         { return "hungry"; }
    if (!c.job)                                 { return "idle"; }
```

```
    return "fine";
}

function crewNeed(c) {
    var key   = needKey(c);
    var lines = NEED_LINES[key];
    var j     = findById(JOBS, c.job || "");
    var text;

    /* a different line each time, in order, so asking twice is
       worth something and asking ten times is not a jackpot */
    if (c.saidKey !== key) { c.saidKey = key; c.saidN = 0; }
    else { c.saidN = (c.saidN || 0) + 1; }

    text = lines[c.saidN % lines.length];
    while (text.indexOf("{job}") >= 0) {
        text = text.replace("{job}", j ? j.label.toLowerCase() : "ship");
    }
    return text;
}

function paintCrewPanels(c) {
    if (!c) { setPanel("cq-cond", ""); setPanel("cq-stand", ""); return; }
    setPanel("cq-cond",
        statLine("Health",  c.health,  false) +
        statLine("Morale",  c.morale,  false) +
        statLine("Fatigue", c.fatigue, true) +
        statLine("Hunger",  c.hunger,  true));

    var j = findById(JOBS, c.job || ""), thr = "";
    if (c.job === "maintenance") { thr = effectiveJobAt(c) + "% condition"; }
    else if (c.job === "filters") { thr = AIR_STAGES[c.jobAt].word + " air"; }
    else { thr = "-"; }

    setPanel("cq-stand",
        field("Role",     c.role) +
        field("Job",      j ? j.label : "None") +
        field("Skill",    c.skill || "Untrained") +
        field("Acts at",  thr) +
        field("Pay",      wageOf(c)
                          ? wageOf(c)
 + " sh each payday"
                          : "Unpaid - holds no job") +
        field("Station",  crewWhere(c)) +
        field("Doing",    c.activity) +
        '<div class="navnote">' + c.traits + '</div>');
}
/* A high number is not always a good number: 90 health is
   excellent and 90 hunger is a problem. good=false reads the bar
   upside down for the tone, while the bar itself still shows the
   real value. */
function statLine(k, v, bad) {
```

```
    var g = bad ? (100 - v) : v;
    var tone = (g >= 70) ? "#7f9161" : (g >= 40 ? "#c08a3e" : "#b05a42");
    return '<div class="navfield"><div class="navfield-k">' + k +
           '</div><div class="navfield-v"><span class="cq-num">' + v +
           '</span>' + barLine(v, tone) + '</div><div class="clear"></div></div>';
}
function roomName(id) {
    var r = findById(ROOMS, id);
    return r ? r.name : "-";
}

function paintOrders(c) {
    var host = $("cq-orders"), i, o, sel, op, row, jid;
    if (!host) { return; }
    clear(host);
    if (!c) { return; }

    host.appendChild(make("div", "cq-sub", c.name));

    /* While the sheet is out, the pad is shut - for everybody,
       not only for the names on it. The pad is not there to be
       argued with; it is simply closed. */
    var busy = crewBusy(c), out = ordersRunning();
    if (busy || out) {
        host.appendChild(make("div", "cq-closed",
            (atDoc(c) ? "In the medical bay."
                      : (busy ? "Carrying out orders." : "Orders in hand.")) +
            "<br>" + (busy || out).left + " action" +
            ((busy || out).left === 1 ? "" : "s") + " remaining."));
        return;
    }

    o   = STATE.pendingOrders[c.id] || {};
    jid = (o.job !== undefined) ? o.job : (c.job || "");

    host.appendChild(make("div", "cq-lab", "Standing job"));
    sel = make("select", "cq-sel");
    for (i = 0; i < JOBS.length; i++) {
        op = document.createElement("option");
        op.value = JOBS[i].id;
        op.text  = JOBS[i].label;
        if (JOBS[i].id === jid) { op.selected = true; }
        sel.appendChild(op);
    }
    wireJob(sel, c.id);
    host.appendChild(sel);
    host.appendChild(make("div", "cq-cost",
        (o.job !== undefined
            ? ("staged - " + jobCost(jid) + " action" +
               (jobCost(jid) === 1 ? "" : "s"))
            : ("costs " + jobCost(jid) + " to hand over"))));
```

```
    host.appendChild(make("div", "cq-lab", "Orders"));
    var od, cost, locked;
    for (i = 0; i < ORDERS.length; i++) {
      od     = ORDERS[i];
      cost   = orderCost(c, od.id);
      locked = (od.id === "learnSt" && !STATE.docked);
      row = make("div", "cq-ord" + (o[od.id] ? " cq-ord-on" : "") +
                 (locked ? " cq-ord-off" : ""));
      var fee = orderFee(c, od.id);
      row.innerHTML =
        '<span class="cq-box">' + (o[od.id] ? "x" : "") + '</span>' +
        '<span class="cq-ordl">' + od.label + '</span>' +
        '<span class="cq-ordc">' + cost +
        (fee ? ' &middot; ' + fee + ' sh' : '') + '</span>' +
        '<div class="cq-ordn">' +
        (locked ? "no station in reach" : od.note) + '</div>';
      if (!locked) { wireOrder(row, c.id, od.id); }
      host.appendChild(row);
    }

}
function wireJob(sel, id) {
    on(sel, "change", function () { stageJob(id, sel.value); });
}
function wireOrder(el, cid, oid) {
    on(el, "click", function (e) {
      toggleOrder(cid, oid);
      if (e && e.stopPropagation) { e.stopPropagation(); }
      if (window.event) { window.event.cancelBubble = true; }
      return false;
    });
}

function paintSheet() {
    var host = $("cq-sheet"), id, o, c, i, bits, html = "";
    if (!host) { return; }
    for (id in STATE.pendingOrders) {
      if (!STATE.pendingOrders.hasOwnProperty(id)) { continue; }
      o = STATE.pendingOrders[id];
      c = findById(CREW, id);
      if (!c) { continue; }
      bits = [];
      if (o.job !== undefined) { bits[bits.length] = jobLabel(o.job); }
      for (i = 0; i < ORDERS.length; i++) {
        if (o[ORDERS[i].id]) { bits[bits.length] = ORDERS[i].label; }
      }
      html += field(c.name.split(" ")[0],
                    bits.join(", ") + ' <span class="cq-sub2">' +
                    crewCostOf(c) + '</span>');
    }
    setPanel("cq-sheet", html ||
      '<span class="dim">Nothing written down yet.</span>');
```

```
}

function paintCrewBand() {
   var band = $("cq-band"), n = ordersCost(), who = orderedCount();
   var out  = ordersRunning();
   if (!band) { return; }
   clear(band);
   PROGRESS_UI["crew-assign"] = null;

   /* No instructions. A fact, or nothing. */
   band.appendChild(make("div", "actionbar",
      out
         ? '<span class="dim">Orders in hand.</span>'
         : (n > 0
            ? ("Orders written for " + who + " crew member" +
              (who === 1 ? "" : "s") + "." +
              (ordersFee() ? " Training fees " + ordersFee() +
                        " sh of " + STATE.shiplets + "." : ""))
            : '<span class="dim">No orders written.</span>')));

   /* THE SAME BUTTON, THE SAME BAR. buildProgressButton finds
      the "crew-assign" task on its own and fills green as the
      ship spends actions; while it fills it is inert, and no
      new orders can be written by anyone. */
   var row = make("div", "actionrow");
   row.appendChild(actionCell(buildProgressButton(
      "crew-assign", "Assign Tasks (" + n + " Action" +
      (n === 1 ? "" : "s") + ")", commitOrders, n > 0)));
   band.appendChild(row);
   band.appendChild(make("div", "clear"));
}


/* ================================================================
  SECTION J : SIDE PANEL
  ================================================================ */

function buildSidePanel() {
   var sp = $("sidepanel"), i, html;
   if (!sp) { return; }
   clear(sp);

   /* --- vessel identity and spec sheet : folded ---------- */
   html = '<div class="sp-ship">' + SHIP.name + '</div>' +
         '<div class="sp-sub">' + SHIP.className +
         '<br>Registry ' + SHIP.registry + '</div>';
   /* Grouped, and ruled between groups. Anything whose value is
      a sentence gets the label on its own line with the text
      beneath it - cramming those into a right-hand column was
      what made the card look like a jumble. */
   html += spPair("Built", SHIP.built) + '<div class="sp-hr"></div>' +
         spLong("Yard",  SHIP.yard) +
```

```
        spLong("Refit", SHIP.refit) +
        spLong("Drive", SHIP.drive) + '<div class="sp-hr"></div>' +
        spPair("Length", SHIP.length) + spPair("Beam",   SHIP.beam) +
        spPair("Height", SHIP.height) + spPair("Mass",   SHIP.mass) +
        '<div class="sp-hr"></div>' +
        spPair("Decks",  SHIP.decks)  + spPair("Berths", SHIP.berths) +
        spPair("Rooms",  ROOMS.length + " aboard");
  sp.appendChild(dropBlock("vessel", "Vessel", html));

  /* --- resources : folded ------------------------------ */
  html = "";
  for (i = 0; i < RESOURCES.length; i++) {
    html += '<div class="sp-row"><span class="sp-key">' + RESOURCES[i].label +
        '</span><span class="sp-val num" id="res-' + i + '">' +
        resourceValue(RESOURCES[i]) + '</span></div>';
  }
  sp.appendChild(dropBlock("resources", "Resources", html));

  /* --- crew : folded ----------------------------------- */
  html = "";
  for (i = 0; i < CREW.length; i++) {
    html += '<div class="sp-row"><span class="sp-key">' + CREW[i].name +
        '</span><span class="sp-val-dim">' + shortRole(CREW[i].role) +
        '</span></div>';
  }
  sp.appendChild(dropBlock("crew", "Crew", html));

  /* --- events : folded. Kind only - the rest is behind the
       task window, which is still a shell. ------------- */
  sp.appendChild(dropBlockEl("events", "Events", buildEventsBody()));

  /* --- indicators : never folded, this is the glance ---- */
  var ind = make("div", "sp-block", '<div class="sp-title">Indicators</div>');
  var strip = make("div", "ind-strip");
  strip.id = "ind-strip";
  ind.appendChild(strip);
  sp.appendChild(ind);
  paintIndicators();
}

/* A block whose body the player folds away. The open/shut state
   lives in STATE, so a rebuild never springs them all open. */
/* Same block, but the body is an element rather than a string -
   the events list carries buttons, and a button needs a handler,
   which innerHTML cannot give it. */
function spPair(k, v) {
  return '<div class="sp-row"><span class="sp-key">' + k +
      '</span><span class="sp-val-dim">' + v + '</span></div>';
}
function spLong(k, v) {
  return '<div class="sp-long"><div class="sp-longk">' + k +
      '</div><div class="sp-longv">' + v + '</div></div>';
```

```
}

function dropBlockEl(key, title, bodyEl) {
    var open = !!STATE.spOpen[key];
    var b    = make("div", "sp-block");

    var head = make("div", "sp-drop" + (open ? " sp-drop-open" : ""), title);
    head.appendChild(make("div", "sp-caret" + (open ? " sp-caret-open" : "")));
    bindDrop(head, key);
    b.appendChild(head);

    if (open) {
        var body = make("div", "sp-body");
        body.appendChild(bodyEl);
        b.appendChild(body);
    }
    return b;
}

/* One row per event: the KIND, and a button that opens its task.
   No detail, no countdown, no severity - the player is told that
   something of this sort has been noticed, and must go and look.
   Opening it is free; only what happens INSIDE will cost. */
function buildEventsBody() {
    var wrap = make("div", null), i, ev, row, btn;
    if (EVENTS.length === 0) {
        wrap.appendChild(make("div", "sp-dim", "Nothing reported."));
        return wrap;
    }
    for (i = 0; i < EVENTS.length; i++) {
        ev
  = EVENTS[i];
        row = make("div", "sp-evt");
        row.appendChild(make("div", "sp-evt-kind", ev.kind));
        btn = make("div", "sp-evt-btn", "Open");
        wireEventOpen(btn, ev.id);
        row.appendChild(btn);
        row.appendChild(make("div", "clear"));
        wrap.appendChild(row);
    }
    return wrap;
}
function wireEventOpen(el, id) {
    on(el, "click", function (e) {
        openTask("event", id, false);   /* tab only - no jump, no action */
        if (e && e.stopPropagation) { e.stopPropagation(); }
        if (window.event) { window.event.cancelBubble = true; }
        return false;
    });
}

function dropBlock(key, title, innerHtml) {
```

```
    var open = !!STATE.spOpen[key];
    var b    = make("div", "sp-block");

    var head = make("div", "sp-drop" + (open ? " sp-drop-open" : ""), title);
    head.appendChild(make("div", "sp-caret" + (open ? " sp-caret-open" : "")));
    bindDrop(head, key);
    b.appendChild(head);

    if (open) { b.appendChild(make("div", "sp-body", innerHtml)); }
    return b;
}

/* closure guard, same pattern as every other binder here */
function bindDrop(el, key) {
    on(el, "click", function () {
        STATE.spOpen[key] = !STATE.spOpen[key];
        buildSidePanel();          /* free - the sidebar costs nothing */
    });
}

/* ----------------------------------------------------------------
   THE INDICATOR STRIP
   One line per running process, drawn bottom-up.

   Green lines are the player's own committed work and SHRINK as
   they finish - a short line means "nearly ready". Red lines are
   ambient ship processes and GROW - a tall red line is the ship
   telling the player something is getting away from them.

   Repainted whenever the task list changes; never labelled.
   ---------------------------------------------------------------- */
function indicatorLines() {
    var out = [], i, t, pct;

    /* THE ENGINE WARNING. Silent above the threshold; below it a
       red line appears and climbs as the condition falls, so the
       worse it gets the harder it is to miss. Leftmost, because
       it is the one line that means trouble rather than work. */
    if (STATE.engineCondition < ENGINE.warnBelow) {
        out[out.length] = {
            pct  : Math.round(((ENGINE.warnBelow - STATE.engineCondition) /
                               ENGINE.warnBelow) * 100),
            grow : true
        };
    }

    /* THE AIR. Silent while the bank runs or the room is still
       clean; below that a second red line grows as it declines. */
    if (airStage() < 4) {
        out[out.length] = {
            pct  : Math.round(((4 - airStage()) / 4) * 100),
            grow : true
```

```
        };
    }

    for (i = 0; i < STATE.ongoing.length; i++) {
        t   = STATE.ongoing[i];
        pct = (t.total > 0) ? Math.round((t.left / t.total) * 100) : 0;
        if (t.state !== "running") { pct = 0; }
        out[out.length] = { pct:pct, grow:false };
    }
    for (i = 0; i < INDICATORS.length; i++) {
        out[out.length] = { pct:INDICATORS[i].pct, grow:true };
    }
    return out;
}

var IND = { h:104, gap:5 };   /* strip height and line pitch */

function paintIndicators() {
    var strip = $("ind-strip");
    if (!strip) { return; }
    clear(strip);

    var lines = indicatorLines(), i, ln, h, x, el;
    /* IND.h must match .ind-strip in the CSS. Closer together
       than before as well - the strip has to hold a busy ship. */
    if (lines.length === 0) {
        strip.appendChild(make("div", "ind-none", "-"));
        return;
    }

    for (i = 0; i < lines.length; i++) {
        ln = lines[i];
        h  = Math.round(IND.h * (ln.pct / 100));
        if (h < 1)     { h = 1; }
        if (h > IND.h) { h = IND.h; }
        x  = i * IND.gap;

        el = make("div", "ind-ghost");
        el.style.left   = x + "px";
        el.style.height = (IND.h - h) + "px";
        strip.appendChild(el);

        el = make("div", "ind-line" + (ln.grow ? " ind-grow" : ""));
        el.style.left   = x + "px";
        el.style.height = h + "px";
        strip.appendChild(el);
    }
}

/* ---------------------------------------------------------------
   THE PINNED FOOT
   Ship functions as symbols, then the build marker last. Save and
```

```
   Load are one door, so they are one button.
   ------------------------------------------------------------- */
function buildSideFoot() {
   var sf = $("sidefoot");
   if (!sf) { return; }
   clear(sf);

   var row = make("div", "sp-icons");
   row.appendChild(iconButton("manifest", "Ship Manifest",
      ["ico-mast", "ico-sail", "ico-hull"]));
   row.appendChild(iconButton("log", "Activity Log",
      ["ico-line1", "ico-line2", "ico-line3"]));
   row.appendChild(iconButton("files", "Save / Load Game",
      ["ico-disk", "ico-shut", "ico-lab"]));
   row.appendChild(iconButton("settings", "Settings",
      ["ico-ring", "ico-t1", "ico-t2", "ico-t3", "ico-t4"]));
   sf.appendChild(row);

   sf.appendChild(make("div", "sp-build", "Version 1 - interface shell"));
}

function iconButton(key, title, parts) {
   var b = make("div", "sp-ico"), i;
   b.title = title;                    /* the only label they get */
   for (i = 0; i < parts.length; i++) {
      b.appendChild(make("div", "ico-part " + parts[i]));
   }
   on(b, "click", function () { openPopup(key); });
   return b;
}

function shortRole(role) {
   var cut = role.indexOf(" /");
   return cut > 0 ? role.substring(0, cut) : role;
}

/* -------------------------------------------------------------
   THE STORES
   Every total is computed, never stored twice. A room window, a
   briefing and the sidebar all call these, so they cannot
   disagree with each other.
   ------------------------------------------------------------- */
/* The sidebar used to be built once and then left to rot: the
   purse could empty and the rack could run dry while the panel
   still showed the boot figures. This rewrites the VALUES only -
   no rebuild, no flicker, and it does nothing at all while the
   block is folded away. */
function paintResources() {
   var i, el;
   for (i = 0; i < RESOURCES.length; i++) {
      el = $("res-" + i);
      if (el) { el.innerHTML = resourceValue(RESOURCES[i]); }
```

```
    }
}

function resourceValue(r) {
    if (r.from === "purse")  { return STATE.shiplets; }
    if (r.from === "o2")     { return LIFE.canisters; }
    if (r.from === "goods")  { return cargoUsed() + " m3"; }
    if (r.from === "survey") { return SURVEY.length + " logs"; }
    return r.value;
}

function cargoUsed() {
    var i, v = 0;
    for (i = 0; i < GOODS.length; i++) { v += GOODS[i].vol; }
    return v;
}

function cargoWorth() {
    var i, v = 0;
    for (i = 0; i < GOODS.length; i++) { v += GOODS[i].value; }
    return v;
}

function surveyWorth() {
    var i, v = 0;
    for (i = 0; i < SURVEY.length; i++) { v += SURVEY[i].value; }
    return v;
}

/* how many filters still have something in them */
function filtersLive() {
    return bankRunning() ? LIFE.slots.length : 0;
}

function lifeSummary() {
    if (bankRunning()) {
        return "Filters running - " + STATE.filterLife + " of " +
               LIFE.lifeMax + " actions of air";
    }
    if (STATE.airHold > 0) {
        return airWord() + " - " + STATE.airHold +
               " actions of cabin air left";
    }
    return "FOUL - no filtration, no reserve";
}

/* A deep scan is worth data. Filed under the body's name so the
   Science Room can show where each log came from. */
function fileSurvey(b) {
    var i, worth;
    for (i = 0; i < SURVEY.length; i++) {
        if (SURVEY[i].id === "sv-" + b.id) { return; }   /* already held */
```

```
    }
    worth = SURVEY_WORTH[b.type];
    if (!worth) { worth = 60; }
    SURVEY[SURVEY.length] = {
       id:"sv-" + b.id, name:b.name, src:"Deep scan",
       value:worth, state:"filed"
    };
}


/* ================================================================
   SECTION K : FLOATING POPUPS
   Side panel items open here rather than replacing the main area,
   so the idle screen stays readable behind them.
   ================================================================ */

function openPopup(key) {
    var spec = popupSpec(key);
    showPopup(spec.title, spec.html, key);
}

/* Anything written to the log reaches an open log popup at once.
   A record the player is staring at while it goes stale is worse
   than no record: it says, wrongly, that nothing happened. */
function refreshLogPopup() {
    var b = $("pop-log-body");
    if (!b) { return; }
    b.innerHTML = popupSpec("log").html;
}

/* The window chrome itself. Every floating panel in the game is
   built here - side panel items and idle-screen briefings alike -
   so they can never drift apart in look or behaviour. */
/* key is optional: a popup that names itself can be found again
   and refreshed in place. The Activity Log needs that - it is
   carried from screen to screen and must keep telling the truth
   while it sits there, not freeze at the moment it was opened. */
function showPopup(title, html, key) {
    var pop  = make("div", "pop");
    if (key) { pop.id = "pop-" + key; }
    var off  = (STATE.popups % 5) * 22;
    STATE.popups++;

    pop.style.left = (300 + off) + "px";
    pop.style.top  = (90 + off) + "px";

    var head = make("div", "pop-head", title);
    var x    = make("span", "pop-x", "x");
    on(x, "click", function () { pop.parentNode.removeChild(pop); });
    head.appendChild(x);

    pop.appendChild(head);
```

```
    var body = make("div", "pop-body", html);
    if (key) { body.id = "pop-" + key + "-body"; }
    pop.appendChild(body);
    document.body.appendChild(pop);

    makeDraggable(pop, head);
}

function popupSpec(key) {
    var html = "", i, c, r;

    if (key === "manifest") {
        html = '<div class="panel-head" style="margin:-11px -13px 10px -13px;">Crew</div>';

        for (i = 0; i < CREW.length; i++) {
            c = CREW[i];
            r = findById(ROOMS, c.room);
            html += '<div class="kv"><div class="kv-k">' + c.name +
                    '</div><div class="kv-v">' + c.role + ' - ' + r.name +
                    '</div><div class="clear"></div></div>';
        }
        html += '<div class="panel-head" style="margin:14px -13px 10px
-13px;">Compartments</div>';
        for (i = 0; i < ROOMS.length; i++) {
            html += '<div class="kv"><div class="kv-k">' + ROOMS[i].name +
                    '</div><div class="kv-v">' + ROOMS[i].systems +
                    '</div><div class="clear"></div></div>';
        }
        return { title:"Ship Manifest", html:html };
    }

    if (key === "log") {
        for (i = STATE.feed.length - 1; i >= 0; i--) {    /* newest first */
            html += '<div class="feed-line"><span class="feed-time num">' +
                    STATE.feed[i].t + '</span>' + STATE.feed[i].s + '</div>';
        }
        return { title:"Activity Log", html:html };
    }

    if (key === "save" || key === "load" || key === "files") {
        return {
            title: (key === "files" ? "Save / Load Game"
                                    : (key === "save" ? "Save Game" : "Load Game")),
            html : '<div class="notice">Persistence is not built yet. ' +
                   'It arrives in a later phase and will write the whole ' +
                   'game state as JSON to a .txt file sitting beside this ' +
                   '.hta file.</div>'
        };
    }

    return {
        title:"Settings",
```

```
      html :'<div class="kv"><div class="kv-k">Build</div><div class="kv-v">Version 1 -
interface shell</div><div class="clear"></div></div>' +
           '<div class="kv"><div class="kv-k">Engine</div><div class="kv-v">Trident /
HTA</div><div class="clear"></div></div>' +
           '<div class="kv"><div class="kv-k">Action economy</div><div class="kv-v">Live - 1
action wired</div><div class="clear"></div></div>' +
           '<div class="kv"><div class="kv-k">Event engine</div><div class="kv-v">Not yet
present</div><div class="clear"></div></div>' +
           '<div class="notice" style="margin-top:10px;">No settings to change yet.</div>'
   };
}

/* ----------------------------------------------------------------
   IDLE-SCREEN BRIEFINGS
   A left click anywhere on the idle screen answers two questions
   and nothing else: what am I looking at, and what could be done
   to it. The costs shown are the same descriptors the task window
   renders, read from describeTarget, so a briefing can never
   drift out of step with the window it describes.

   Strictly free. It reads state, writes none, and spends no
   action - the idle screen remains the one place that costs the
   player nothing.
   ---------------------------------------------------------------- */
function openBrief(kind, id) {
   var b = briefSpec(kind, id);
   showPopup(b.title, b.html);
}

function briefSpec(kind, id) {
   if (kind === "nav") { return navBrief(); }

   var t = describeTarget(kind, id), html = "", i, a, lim;

   html += '<div class="notice" style="margin:0 0 10px 0;">' +
         t.sub + '</div>';

   lim = t.rows.length;
   if (lim > 5) { lim = 5; }          /* a briefing, not the file */
   for (i = 0; i < lim; i++) {
      html += kvRow(t.rows[i][0], t.rows[i][1]);
   }

   html += popHead("Actions in this window");
   for (i = 0; i < t.acts.length; i++) {
      a = t.acts[i];
      html += '<div class="act"><span class="act-cost">' + a.cost +
            '</span>' + a.label + (a.live ? " - wired" : "") + '</div>';
   }

   html += '<div class="notice" style="margin-top:10px;">' +
         'Right-click to open the ' + t.title + ' window. It waits in ' +
```

```
          'the tab bar - you are not taken there.</div>';

    return { title: t.kindLabel + " - " + t.title, html: html };
}

/* The chart has no describeTarget rows - it builds its own screen -
   so its briefing is written out here. */
function navBrief() {
    var html =
        '<div class="notice" style="margin:0 0 10px 0;">The ship chart. ' +
        'Everything that is only LOOKING at it is free.</div>' +
        kvRow("Scales",     "System, Sector and Galaxy") +
        kvRow("Switching",  "The three buttons on the chart - free") +
        kvRow("Panning",    "Hold the left button and drag the chart") +
        kvRow("Selecting",  "Click any marker to read it - free") +
        kvRow("Out of range", "Sector and galaxy contacts can be named " +
                              "and classified, nothing more") +
        popHead("Actions in this window") +
        '<div class="act"><span class="act-cost">varies</span>' +
        'Start Travel - wired</div>' +
        '<div class="act"><span class="act-cost">' + SCAN_COST +
        ' actions</span>Run Deep Scan - wired</div>' +
        '<div class="notice" style="margin-top:10px;">Travel costs one ' +
        'action per step of charted distance, so the price is the ' +
        'crossing itself. Both run in the background and drain from ' +
        'actions spent anywhere on the ship.</div>' +
        '<div class="notice" style="margin-top:6px;">Right-click to open ' +
        'the Navigation window. It waits in the tab bar - you are not ' +
        'taken there.</div>';
    return { title:"NAV - Navigation", html:html };
}

/* the two scraps of markup every popup repeats */
function kvRow(k, v) {
    return '<div class="kv"><div class="kv-k">' + k +
           '</div><div class="kv-v">' + v +
           '</div><div class="clear"></div></div>';
}

function popHead(text) {
    return '<div class="panel-head" style="margin:14px -13px 10px -13px;">' +
           text + '</div>';
}

/* simple drag, written without modern event APIs */
function makeDraggable(pop, handle) {
    var dragging = false, ox = 0, oy = 0;

    on(handle, "mousedown", function (e) {
        var ev = e || window.event;
        dragging = true;
        ox = ev.clientX - pop.offsetLeft;
```

```
      oy = ev.clientY - pop.offsetTop;
      if (ev.preventDefault) { ev.preventDefault(); }
      return false;
    });

    on(document, "mousemove", function (e) {
      if (!dragging) { return; }
      var ev = e || window.event;
      pop.style.left = (ev.clientX - ox) + "px";
      pop.style.top  = (ev.clientY - oy) + "px";
    });

    on(document, "mouseup", function () { dragging = false; });
}


/* ================================================================
   SECTION L : STATUS BAR
   ================================================================ */

/* THE ONLY STRIP OF SCREEN VISIBLE FROM EVERY WINDOW.
   The activity feed lives on the idle screen, so a player who
   seats three canisters and starts the bank - all of it done
   inside the filter room - saw nothing happen anywhere. The
   last line of the log now rides here, so the ship can always
   answer "what just happened" without changing screens. */
function setStatus() {
    var bar = $("statusbar");
    if (!bar) { return; }
    var open = STATE.tabs.length;
    var last = STATE.feed.length
            ? STATE.feed[STATE.feed.length - 1].s : "";
    bar.innerHTML =
      SHIP.name + "  -  " + findById(INFOS, "location").value +
      "   |   Actions <span class='num'>" + actionCounter + "</span>" +
      "   |   Open tasks <span class='num'>" + open + "</span>" +
      (last ? "   |   <span class='st-last'>" + last + "</span>" : "");
}


/* ================================================================
   SECTION M : RESERVED - FUTURE PHASES
   Deliberately empty. Marked now so the file has obvious homes for
   the systems the manifesto requires, and so later sessions extend
   rather than restructure.
   ================================================================ */

/* --- ACTION ECONOMY ----------------------------------------
   IMPLEMENTED. See Section D-2. performAction() is the only way
   time passes; tickWorld() is where the engine will hook in. */

/* --- EVENT ENGINE ------------------------------------------
```

```
   Data-driven. Events will be plain objects in a table with
   trigger conditions, costs, countdowns, outcomes and chain links.
   The engine reads the table; adding an event must never mean
   editing engine code. Not implemented. */

/* --- SHIP SYSTEMS LOGIC ---------------------------------------
   Degradation per action, status thresholds, power priority and
   cascade rules. Not implemented. */

/* --- CREW LOGIC -----------------------------------------------
   Needs, stat drift, traits, relationships, room-to-room movement.
   Not implemented. */

/* --- STATIONS, QUESTS, ECONOMY --------------------------------
   Docking, market, hiring, shipyard, tavern, mission board.
   Not implemented. */

/* --- SAVE / LOAD ----------------------------------------------
   FileSystemObject via ActiveX, JSON text into a .txt beside this
   file. Trident may lack native JSON in some document modes, so a
   fallback serialiser may be needed. Not implemented. */


/* ================================================================
   SECTION N : INITIALISATION
   ================================================================ */

function init() {
    try {
        window.resizeTo(1440, 940);
        window.moveTo(
            Math.max(0, Math.floor((screen.availWidth  - 1440) / 2)),
            Math.max(0, Math.floor((screen.availHeight -  940) / 2))
        );
    } catch (err) { /* window ops can fail if the host blocks them */ }

    document.oncontextmenu = function () { return false; };

    /* the layout follows the window from here on */
    window.onresize = onWindowResize;

    /* the life support readout is the filter rack, not a slogan */
    findById(INFOS, "life").value = lifeSummary();

    buildSidePanel();

   buildSideFoot();
    renderTabs();
    buildIdleScreen();

    logLine("Command dashboard online. All primary systems reporting.");
    logLine("Rell Aventine assumed the bridge watch.");
```

```
    logLine("Jax Moreno logged a maintenance note: coolant line, port manifold.");
    logLine("Shield generator operating at 62%.");
    logLine("Suri Tal is preparing rations in the mess hall.");
    logLine("No contacts within scan range. Holding position at Kessler Drift.");

    setStatus();

    /* cosmetic only - see Section F */
    window.setInterval(driftTick, 55);
}

window.onload = init;

</script>
</body>
</html>
```